esprit
Se former autrement

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : DATA SCIENCE

Design of a Multi-Agent AI Pipeline for Evidence Collection from Illegal Streaming Sites

By: *Ahmed Arfaoui*

Academic supervisor: *Nadine Maazoun*

Corporate Internship Supervisor: *Wael Hkiri*

SOFT STARS

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Ahmed Arfaoui

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Wael Hkiri

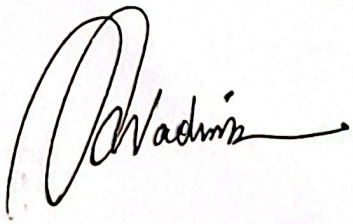
Cachet & Signature / Stamp & Signature

IDEATION SERVICES
IU:1452857P

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Nadine Maazoun

Signature / Signature



Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

Abstract

This report presents Open Web Catcher (OWC), an evidence-collection platform designed during a PFE internship at Soft Stars in a beIN SPORTS rights-protection context. The project addresses a practical problem: illegal streaming evidence often appears only after a browser renders the page, handles popups, traverses iframes, activates players, and observes media requests. The work produced two connected artifacts: an n8n landing-agent workflow used as an operational backup for dynamic landing pages, and the broader OWC platform built with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, specialist agents, traces, screenshots, provider enrichment, and telemetry. The evaluation focuses on reviewability rather than automatic enforcement: the system must preserve enough evidence for a human reviewer to understand what happened and what remains uncertain.

Keywords: Open Web Catcher; multi-agent system; browser automation; illegal streaming; evidence collection; stream extraction; channel detection; observability.

Résumé

Ce rapport présente Open Web Catcher (OWC), une plateforme de collecte de preuves conçue pendant un stage PFE chez Soft Stars dans un contexte de protection des droits lié à beIN SPORTS. Le projet répond à un problème concret: les preuves de streaming illégal apparaissent souvent seulement après le rendu navigateur, la gestion des popups, l'inspection des iframes, l'activation du lecteur et l'observation des requêtes média. Le travail a produit deux réalisations complémentaires: un workflow n8n utilisé comme solution de secours pour les pages dynamiques, puis la plateforme OWC construite avec FastAPI, Next.js, PostgreSQL, des outils MCP, Puppeteer/Chrome, des agents spécialisés, des traces, des captures d'écran, l'enrichissement fournisseur et la télémétrie. L'évaluation met l'accent sur la revue humaine et la traçabilité des preuves.

Mots clés: Open Web Catcher; système multi-agent; automatisation navigateur; streaming illégal; collecte de preuves; extraction de flux; détection de canaux; observabilité.

To the people who supported me throughout this work

Dedication

To my parents, for their love, patience, and constant support.

To my family, for giving me strength when the work became heavy.

To my friends, for the encouragement, the late conversations, and the reminders that I was not doing this alone.

This work is dedicated to you.

Acknowledgements

with gratitude

At the end of this internship and report, I would like to express my sincere gratitude to the people and institutions who supported this work and helped turn it into a complete engineering project.

To ESPRIT

I warmly thank the Higher Private School of Engineering and Technology, ESPRIT, and especially the Department of Computer Science, for the academic framework, resources, and support provided throughout my studies. This environment made it possible to connect a concrete professional mission with a rigorous engineering methodology.

To my academic supervisor

I am deeply grateful to my academic supervisor, Nadine Maazoun, for her availability, guidance, and precise feedback. Her remarks helped me keep the report structured, technically grounded, and focused on the real contribution of the project.

To my professional supervisor

I also thank my professional supervisor, Wael Hkiri, for his trust, technical direction, and continuous follow-up during the internship. His feedback on operational constraints, browser tooling, deployment choices, and evidence review had a direct impact on the project.

To the Soft Stars team

My sincere thanks go to the Soft Stars team for welcoming me, sharing their experience, and giving me the opportunity to work on a concrete rights-protection problem in a professional environment.

To the jury members

Finally, I respectfully thank the members of the jury for the time and attention they will devote to evaluating this work. I hope that this report reflects the effort invested in the project and responds clearly to their expectations.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Meaning and use in the report
AI	Artificial Intelligence; model-assisted browser reasoning and evidence interpretation.
API	Application Programming Interface; FastAPI routes and operator-console endpoints.
ASN	Autonomous System Number; network ownership identifier used during provider enrichment.
ASB	Azure Service Bus; queue path used by the deployed n8n landing-agent backup.
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart; challenge category that can block automated evidence collection.
CDN	Content Delivery Network; hosting layer for assets, challenge pages, and stream delivery.
CDP	Chrome DevTools Protocol; browser network and runtime observation interface.
DASH	Dynamic Adaptive Streaming over HTTP; shorthand used for MPEG-DASH playback and manifests.
DB	Database; short label for persisted run, trace, stream, and review records.
DMCA	Digital Millennium Copyright Act; takedown-email context for provider notifications.
DNS	Domain Name System; source of host, name-resolution, and provider-enrichment evidence.
DOM	Document Object Model; rendered browser tree inspected by the agents.
DSRM	Design Science Research Methodology; research method used to structure the artifact work.
FP	Fingerprint profile; browser-identity profile shorthand used in the fingerprint-rotation diagram.
FR	Functional Requirement; requirement identifiers used in Chapter 2.
HLS	HTTP Live Streaming; playlist protocol represented by `.m3u8` evidence.
HTML	Hypertext Markup Language; static source compared with rendered browser state.
HTTP	Hypertext Transfer Protocol; request layer for pages, APIs, manifests, and media.
IP	Internet Protocol; network address used for stream-host and provider attribution.
JSON	JavaScript Object Notation; format used for agent outputs, traces, and settings.
KPI	Key Performance Indicator; success, stream-yield, cost, token, and tool-call measures.
LLM	Large Language Model; reasoning runtime used by the specialist agents.
M3U8	UTF-8 M3U Playlist; HLS playlist extension captured as stream evidence.
MCP	Model Context Protocol; tool interface used by the browser agents.

List of Acronyms (continued)

Acronym	Meaning and use in the report
MENA	Middle East and North Africa; regional coverage context used in the n8n prototype evaluation.
MPD	Media Presentation Description; MPEG-DASH manifest captured as stream evidence.
MPEG-DASH	MPEG standard for Dynamic Adaptive Streaming over HTTP; full standard name behind DASH.
NFR	Non-Functional Requirement; reliability, cost, security, and observability constraints.
OCR	Optical Character Recognition; future direction for screenshot and channel-name reading.
OWC	Open Web Catcher; reference platform built during the internship.
PFE	Projet de Fin d'Etudes; graduation internship project and report context.
RDAP	Registration Data Access Protocol; ownership metadata source used for provider enrichment.
RFC	Request for Comments; standards publication source used for HLS.
RPD	Requests Per Day; provider or model rate-limit unit.
RPM	Requests Per Minute; model and API throughput unit.
SQL	Structured Query Language; query layer used by PostgreSQL.
SSE	Server-Sent Events; one-way event stream used by the MCP client session model.
TLS	Transport Layer Security; protocol layer discussed with HTTP headers and browser fingerprinting.
TPM	Tokens Per Minute; LLM throughput-limit unit.
UA	User-Agent; browser identity string and client-hint shorthand used in fingerprinting controls.
UI	User Interface; operator-console screens and controls.
URL	Uniform Resource Locator; targets, candidates, stream links, and evidence links.
XML	Extensible Markup Language; markup family used by MPEG-DASH MPD manifests.
XHR	XMLHttpRequest; dynamic browser request pattern observed during page inspection.

CHAPTER 1

General Introduction

1.1 Introduction

This PFE report presents the work carried out at Soft Stars in the context of illegal sports-streaming evidence collection for beIN SPORTS. The project started from a concrete operational need: suspicious pages could be discovered, but the evidence needed for review was often hidden behind browser behavior.

In practice, that evidence could depend on rendered interfaces, popups, iframe players, server buttons, redirects, and short-lived media manifests. The internship focused on a clear goal: design an evidence workflow that observes the page like a browser user, preserves the trace, and returns reviewable evidence instead of only a final URL.

This opening chapter explains why the internship mattered, who the work served, and how the project was scoped. It distinguishes the existing company workflow, the deployed n8n landing-agent backup, and Open Web Catcher as the reference platform built for the academic study. It also defines the problem, the mission deliverables, the project boundaries, and the report structure used by the following chapters.

1.2 Host Organization and Internship Context

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship took place in a sports media-protection context connected to beIN SPORTS. In this domain, the practical problem is not only detecting that a suspicious website exists. A rights-protection workflow also needs enough evidence to answer review questions after the browser session has ended:

- what was visible on the page;
- which channel, event, or match appeared;
- which page led to the player;
- which technical host or provider was involved;
- whether the result is trustworthy enough for human review.

1.2.1 Target Users and Business Context

The project was made for a concrete rights-protection setting. For a broadcaster such as beIN SPORTS, illegal live streams weaken the value of exclusive broadcasting rights, create unauthorized alternatives during high-value events, and make enforcement difficult when the useful evidence disappears after the browser session. The business need is not only to find suspicious domains. It is also to shorten the path from a reported URL to reviewable technical evidence: page screenshots, channel or event context, stream URLs, provider information, and a trace explaining how the result was obtained.

The main target users are the teams who need to transform a suspicious page into a reviewable case. Rights-protection operators need faster triage and fewer manual browser repetitions. Technical analysts need enough runtime detail to understand why the system followed a specific page, server button, iframe, or media request. Legal and compliance reviewers need evidence that can be checked by a human before any external action is taken. Business stakeholders need visibility on whether the monitoring workflow supports the protection of licensed content and operational prioritization.

Target user group	Practical need	Project contribution
Rights-protection operators	Triage suspicious pages quickly and avoid repeating fragile manual browser steps.	Browser-guided evidence collection, screenshots, candidate URLs, and clear run status.
Technical analysts	Understand dynamic pages, popups, iframes, server buttons, redirects, and media requests.	Specialist agents, structured traces, tool-call records, and separated page-role logic.
Legal and compliance reviewers	Review evidence before escalation or takedown preparation.	Human-reviewable screenshots, stream evidence, provider enrichment, and draft email artifacts.
Business and client stakeholders	Protect the value of licensed sports content and prioritize enforcement effort.	A clearer monitoring workflow that connects suspicious pages to auditable evidence and operational reporting.

Table 1.1: Target users and business value of the evidence workflow.

Figure 1.1: Visual context for the host organization and the sports media-rights domain of the internship.

1.3 Existing Solutions and Project Positioning

Before this PFE work, the company already had an operational workflow for monitoring and processing suspicious streaming websites. That workflow remained the primary company process. It was useful for broad discovery, recurring checks, and normal cases where the page structure was stable enough for crawler-style logic.

The limitation appeared on pages where useful evidence was created only at browser runtime:

- JavaScript-rendered match cards or schedules;
- server buttons and popups that had to be handled in order;
- nested iframes and player activation steps;
- short-lived `m3u8` or `mpd` media requests.

In this report, the company workflow means the existing Soft Stars operational monitoring and crawler-based processing path that remains the primary company work.

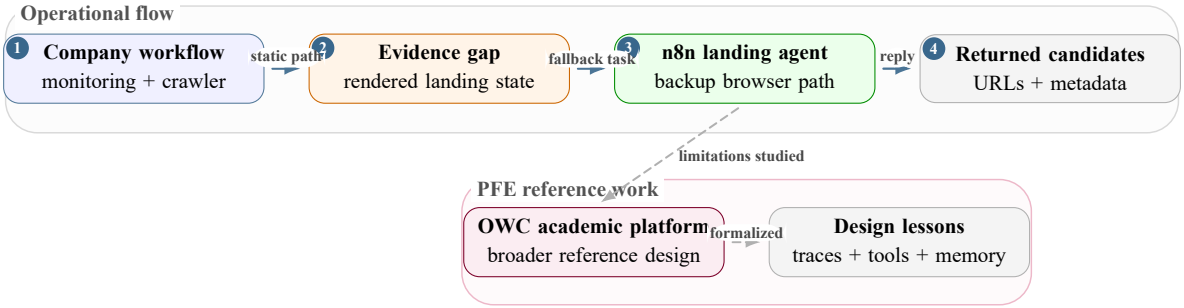


Figure 1.2: n8n landing-agent backup and OWC reference platform.

For that reason, the n8n landing agent was positioned as a backup to the company workflow, not as a replacement for it. When the primary flow could not find enough landing-page evidence, it could send a task with the target URL and execution metadata to the landing agent.

The backup agent then performed a focused browser task:

- open the rendered page;
- inspect visible match, channel, and schedule cues;
- follow candidate links when needed;
- return candidate URLs, metadata, screenshots, rejected links, and reasoning notes for review or downstream processing.

In this report, the n8n landing agent means the focused backup browser workflow used when the company workflow needs rendered landing-page reasoning.

OWC had a different role. It was the academic and engineering reference platform built during the internship to study the n8n limitations and propose a clearer architecture around:

- specialist agents and browser tools;
- traces, screenshots, and memory;
- model telemetry, persistence, and evaluation.

The deployed company-side decision stayed focused on a landing-agent backup. OWC describes the broader design that could support a more complete future platform. In this report, OWC means the reference platform built for the academic study, not the deployed company workflow itself.

1.4 Problem Statement

Illegal streaming websites are difficult to investigate because they combine unstable content, dynamic rendering, misleading navigation, popup traps, nested frames, and short-lived media URLs. The evidence path often has several roles:

- a landing page lists matches, events, schedules, or channel cards;
- a hosting page exposes play buttons, server controls, overlays, and iframe candidates;
- an embedded page or direct player exposes the final media behavior;
- a provider-enrichment stage connects collected stream evidence to host, network, country, or abuse-contact information.

If those roles are mixed together, the system becomes hard to debug. A failed result could mean several different things:

- the page was classified incorrectly;
- a landing candidate was missed;
- a hosting server was not activated;
- an iframe was not inspected;
- the final media request was not connected to provider evidence.

The project required an architecture that separates page roles, specialist agents, browser tools, screenshots, traces, model usage, provider enrichment, and final review outputs.

1.5 Internship Mission and Scope

My mission during the internship was to study, design, and validate an evidence collection workflow that uses browser-based AI reasoning to support dynamic evidence paths when the existing company workflow needs a backup. The work produced two related artifacts.

The first artifact was an n8n landing-agent workflow. It was used to validate the idea quickly in the company environment and to provide a deployable backup path for landing pages that required rendered-page reasoning. This workflow helped test whether an agent could inspect visible page state, follow candidate links, handle page-specific obstacles, and return useful match or hosting metadata.

The second artifact was Open Web Catcher (OWC), the complete platform designed during the internship as the reference architecture for the report. The internal name was chosen because the project catches evidence from the open web through browser-visible behavior rather than through a static HTML response.

In this report, OWC refers to the whole project:

- FastAPI backend and Next.js operator console;
- PostgreSQL persistence and MCP browser tooling;
- Puppeteer/Chrome runtime and specialist agents;
- model telemetry and evidence review outputs.

1.5.1 Mission Deliverables

The internship deliverables were:

- a validated n8n landing-agent workflow used as a deployable backup to the company workflow for dynamic landing-page targets;
- an OWC architecture separating backend, frontend, database, agents, browser tools, and evidence outputs;
- browser-tool profiles for classification, landing, hosting, and embedded-player investigation;
- support for screenshots, stream candidates, provider enrichment, channel metadata, tool traces, model telemetry, and cost visibility;
- an evaluation and reporting structure based on traceability and reviewability.

1.5.2 Scope Boundaries

The project did not replace the full company enforcement process. It did not automatically send takedown messages, and it did not claim to solve every anti-bot mechanism or every illegal streaming website. The practical scope was to produce an engineering foundation that could investigate dynamic pages, preserve evidence, expose failures clearly, and support future operational integration.

1.6 Main Contributions

The main contributions of the PFE work are:

- a clear analysis of why static page collection is insufficient for many dynamic streaming-evidence paths;
- a DSRM-based artifact design centered on Open Web Catcher;
- an n8n landing-agent workflow deployed as an operational backup to the company workflow for dynamic evidence cases;
- a multi-agent architecture that separates classification, landing discovery, hosting interaction, embedded-player inspection, provider enrichment, and review output generation;
- a browser-tool layer with profile-scoped inspection, navigation, interaction, screenshot, media extraction, memory, fingerprint, and proxy controls;

- an operator console and persistence model that make traces, screenshots, model usage, tool calls, costs, channel signals, and failure reasons reviewable.

1.7 Report Structure

The report is organized as follows:

- **Chapter 2** presents DSRM as the methodology and derives the project requirements.
- **Chapter 3** introduces streaming pages, browser automation, media manifests, agent workflows, provider caching, and evidence traceability.
- **Chapter 4** describes the OWC architecture: agents, browser tooling, channel detection, persistence, and reviewability.
- **Chapter 5** explains the implementation path from the n8n backup to OWC, including caching, observability, and the company deployment flow.
- **Chapter 6** presents evaluation and testing results.
- **Chapter 7** provides the general conclusion and perspectives.

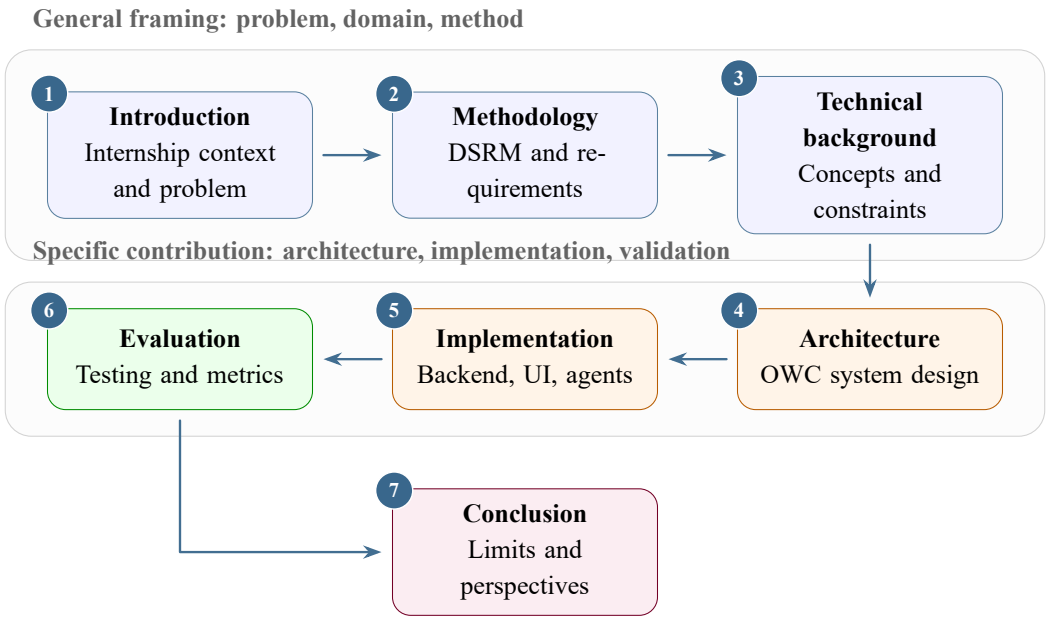


Figure 1.3: Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.

1.8 Conclusion

This chapter establishes the internship context, the operational problem, the project scope, and the report organization. The next chapter turns this context into a methodological frame and a set of requirements for the artifact.

CHAPTER 2

Methodology, Requirements Analysis, and Design Constraints

2.1 Introduction

The internship was organized around Design Science Research Methodology (DSRM). This choice fits the nature of the work. The project was not only about observing a phenomenon from the outside or training a predictive model. It was about designing, building, demonstrating, and evaluating an engineering artifact for a concrete organizational problem.

In this report, that artifact is Open Web Catcher (OWC), an evidence-collection platform built around:

- browser tooling and specialist agents;
- trace storage, screenshots, and stream evidence;
- provider enrichment and reviewable outputs.

DSRM provides the backbone for the chapter structure that follows:

- identify and motivate the problem;
- define the solution objectives;
- design and develop the artifact;
- demonstrate and evaluate the artifact;
- communicate the work through the PFE report and diagrams.

This phase logic follows the design-science view of information-systems artifacts proposed by Hevner et al. [9] and the DSRM process described by Peffers et al. [10].

This chapter explains the method used to organize the project. Design Science Research Methodology connects the operational problem to the artifact that was built, then helps translate the mission into functional requirements, non-functional requirements, design constraints, and an internship timeline.

2.2 DSRM as the Methodological Backbone

DSRM was selected because it gives equal importance to the organizational problem and to the engineered artifact. The operational problem was concrete: useful evidence often appeared only after browser rendering, interaction, iframe traversal, server switching, or media activation. The PFE work needed a methodology that could justify a new artifact, not only describe a work plan.

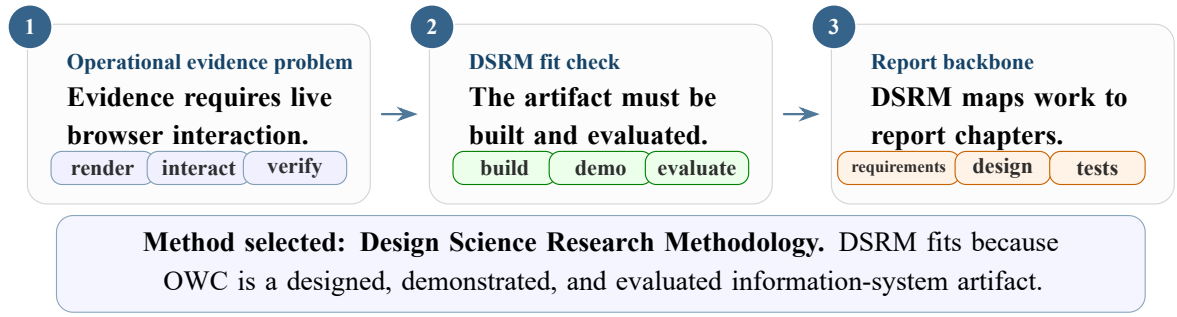


Figure 2.1: DSRM selection logic for an artifact-centered evidence-collection project.

2.2.1 Why DSRM Fits the Internship

The project has three properties that make DSRM suitable:

- **Artifact-centered**: the contribution is a platform and workflow design, not a pure analysis.
- **Use-based evaluation**: the artifact matters only if it can inspect real pages, produce traces, capture screenshots, and return evidence.
- **Organizational relevance**: Soft Stars needed a more adaptable way to handle streaming pages that became too dynamic for static extraction.

2.2.2 DSRM Phases Used in the Report

The six DSRM phases are not treated as abstract theory. Each one corresponds to work carried out during the internship. Figure 2.2 summarizes the adopted backbone.

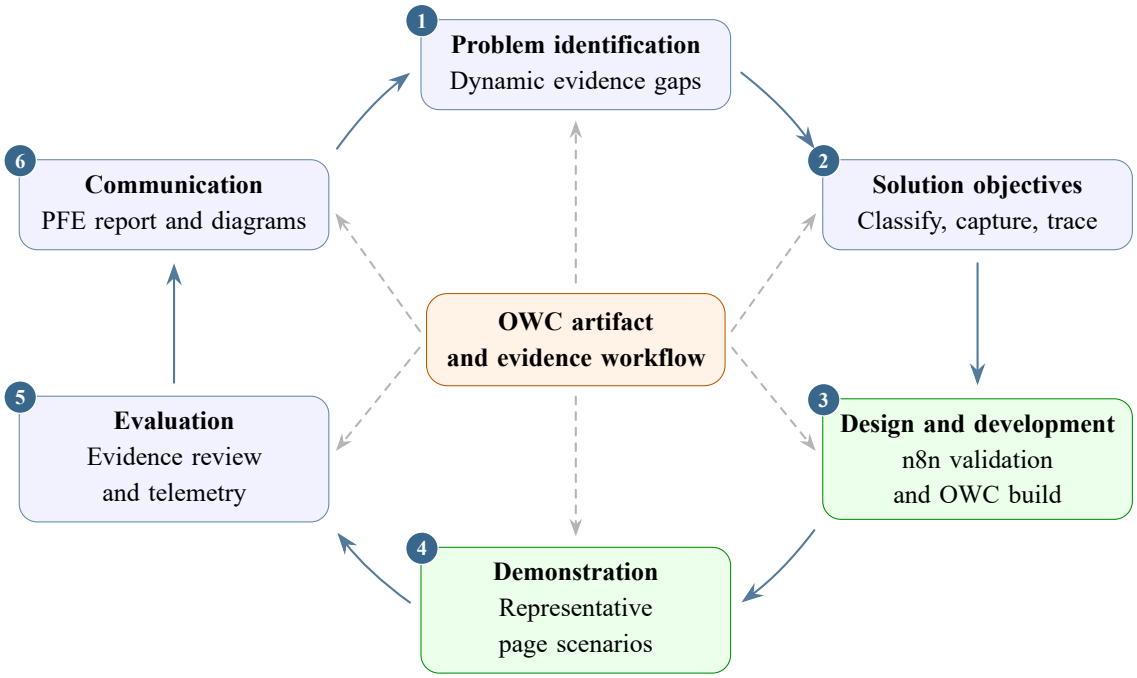


Figure 2.2: DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.

2.2.2.1 Problem identification and motivation

The first phase established the practical problem: suspicious streaming pages could be found, but collecting reviewable evidence was difficult. The difficulty appeared in several common cases:

- pages that exposed only lists of matches or channels;
- pages that required a server-button click before the player loaded;
- pages that hid the player inside an iframe;
- pages that generated short-lived manifest URLs only after playback.

A static HTML fetch can miss these states because it does not perform the same visual and network sequence as a browser user.

2.2.2.2 Objectives of the solution

The second phase translated the problem into objectives. The artifact needed to ingest a URL, classify the page role, follow landing-to-hosting and hosting-to-embedded handoffs, activate players when necessary, collect screenshots, detect stream evidence, preserve channel and provider metadata when available, and store enough trace data for later review. These objectives became the functional and non-functional requirements listed later in this chapter.

2.2.2.3 Design and development

The third phase produced the artifacts. The n8n workflow was used first because it allowed fast operational validation of an AI-assisted landing workflow and could act as a backup path for dynamic evidence cases. OWC then formalized the same ideas in a maintainable platform with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, profile-scoped sessions, model telemetry, and specialist agents.

2.2.2.4 Demonstration

The fourth phase demonstrated that the artifact could operate on representative page types: landing pages, hosting pages, embedded player pages, popup-heavy pages, tokenized manifest cases, and pages where channel evidence had to be verified from visible player or screenshot context. Demonstration was not limited to a happy path; it included runs where the system returned explicit failures such as no hosting pages, no streams, or skipped downstream enrichment.

2.2.2.5 Evaluation

The fifth phase evaluated the artifact through trace inspection, tool telemetry, screenshots, token and cost records, stream yield, provider enrichment, and dashboard snapshots. The aim was not to claim that every illegal streaming page can be solved. The aim was to verify whether the artifact made the extraction process more observable, repeatable, and reviewable than a one-shot static fetch or a manually inspected page.

2.2.2.6 Communication

The final phase communicates the artifact. This report presents the problem, methodology, technical background, architecture, implementation, evaluation, and limitations. The diagrams and tables make the artifact understandable as an engineering contribution rather than reproduce internal source files.

2.3 DSRM Mapping to the Internship Work

Table 2.1 maps each DSRM phase to the work performed during the internship and to the concrete outputs used in the rest of the report.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Analyze static-collection limits, dynamic page behavior, stream activation, iframe traversal, channel ambiguity, and the need for reviewable evidence.	Problem statement and motivation for OWC
Objectives of the solution	Define the extraction, traceability, screenshot, channel, provider, cost, and review requirements that the artifact must satisfy.	Functional and non-functional requirements
Design and development	Build the n8n validation workflow, then design OWC as a platform with backend, frontend, database, browser tools, agents, and observability.	n8n workflow and OWC architecture
Demonstration	Run the artifact on representative landing, hosting, embedded, popup, iframe, and tokenized-stream scenarios.	Demonstrated evidence paths and failure paths
Evaluation	Inspect traces, screenshots, stream evidence, provider enrichment, tool behavior, token usage, costs, and dashboard telemetry.	Evaluation metrics, results, and limitations
Communication	Produce the PFE report, diagrams, final PDF, and design explanation for academic and company review.	Written report and final synthesis

Table 2.1: DSRM phase mapping to project activities and outputs.

2.4 Project Evolution Timeline

The timeline follows the DSRM logic but remains concrete. From December to March, the work stayed close to feasibility: evidence needs, static-collection limits, n8n browser agents, Puppeteer experiments, and the first LangChain/LangGraph agent loops. During April and May, the focus moved to OWC itself: the FastAPI and Next.js platform, PostgreSQL observability, profiled MCP tools, evaluation screens, and final synthesis.

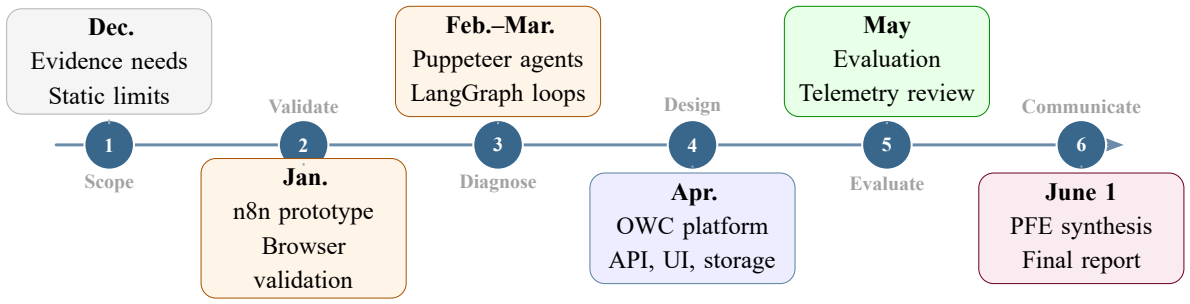


Figure 2.3: Internship timeline from December feasibility work to the final OWC synthesis on June 1.

2.5 Functional Requirements

The functional requirements describe what the artifact must do to support the evidence-collection workflow.

ID	Priority	Requirement	Evidence role
FR-1	**Critical**	URL intake and run lifecycle control	Creates a traceable run that can be started, cancelled, resumed, and reviewed.
FR-2	**Critical**	Rendered page-type classification	Decides whether the browser is on a landing, hosting, embedded, unsupported, or failed page.
FR-3	**Critical**	Orchestrator handoff by page type	Routes the run to the right specialist while preserving the evidence context.
FR-4	**High**	Landing discovery of matches, events, channels, and hosting candidates	Keeps listing-page and schedule-row context before the run moves downstream.
FR-5	**Critical**	Hosting controls, overlays, and play-button interaction	Handles the interaction needed to reach a player, server state, stream, or iframe.
FR-6	**Critical**	Embedded frame, player-state, and media inspection	Inspects iframe-local player state before judging whether final stream evidence exists.
FR-7	**High**	Screenshot capture at evidence and failure points	Gives human reviewers visual proof for accepted, partial, and failed runs.
FR-8	**Critical**	Manifest, media-URL, and embedded stream extraction	Collects the technical stream evidence that supports rights-protection review.
FR-9	**High**	Channel detection from page, player, OCR, and agent output	Links the extracted evidence to the channel or event being investigated.
FR-10	**Medium**	Provider enrichment and reviewable evidence bundle generation	Turns stream hosts and metadata into provider rows and review-ready evidence bundles.
FR-11	**High**	Console review of telemetry, tools, settings, and provider results	Lets operators inspect outcomes, costs, tool behavior, screenshots, and provider evidence.

Table 2.2: Functional requirements for the evidence-collection workflow.

2.6 Design Constraints and Requirement Traceability

The requirements were not independent checklist items. They came from a small set of constraints observed during the internship: pages were dynamic, players were nested, media URLs could be temporary, browser-agent runs consumed time and tokens, and the final output had to remain reviewable by a human analyst. Figure 2.4 shows how the operational problem, requirements, artifact response, and review outputs connect.

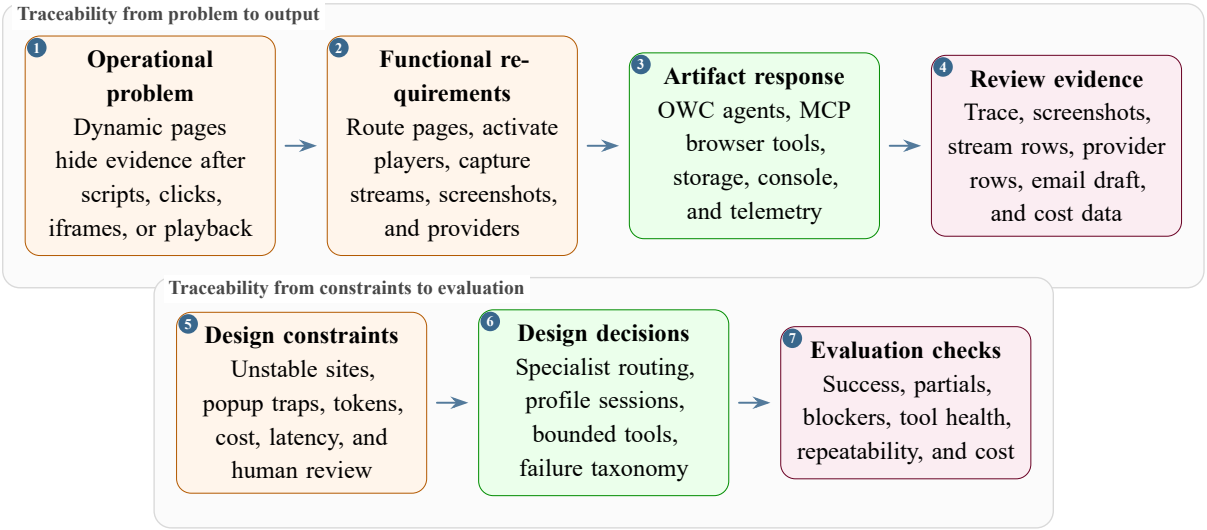


Figure 2.4: Requirement traceability from the operational problem to the implemented artifact and review outputs.

The second traceability view focuses on design pressure. Each constraint forced a specific engineering response rather than a generic automation script. Figure 2.5 summarizes these responses and explains why the artifact uses specialist agents, profile-scoped browser tooling, trace storage, telemetry, and a focused company deployment path.

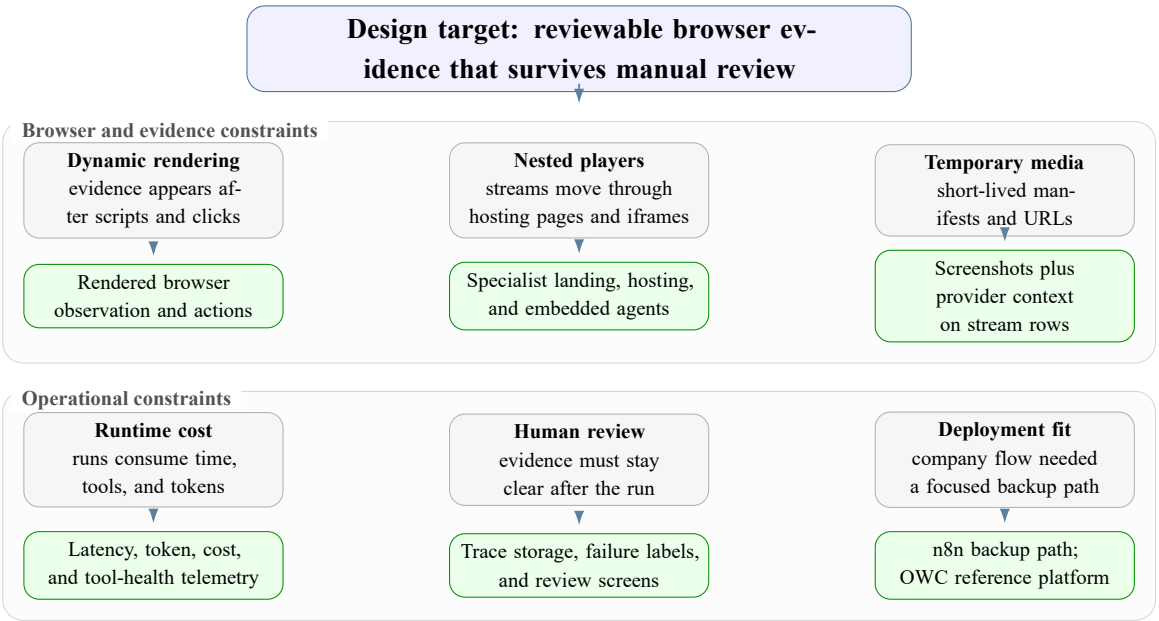


Figure 2.5: Main design constraints and the engineering responses selected for OWC.

2.7 Non-Functional Requirements

The non-functional requirements explain the qualities required for the artifact to be useful after a run finishes.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	OWC separates backend routes, frontend views, agent contracts, browser tools, runtime settings, and persistence records.
NFR-2	Observability	Runs persist runtime events, agent runs, LLM calls, tool calls, screenshots, streams, provider rows, and final statuses.
NFR-3	Traceability	Each result can be connected back to a URL, agent, tool call, screenshot, stream candidate, and final decision.
NFR-4	Reliability	Profile-scoped sessions, bounded retries, popup recovery, iframe recovery, media verification, and explicit failure states reduce silent errors.
NFR-5	Cost control	Token counters, cached input tracking, model settings, rate-limit awareness, and dashboard cost summaries make model usage visible.
NFR-6	Adaptability	Page-type specialists, browser profile controls, proxy configuration, fingerprint rotation, and memory hints support changing site behavior.
NFR-7	Deployment fit	The company deployment keeps n8n as a backup operational orchestrator and runs the landing agent through queue-driven container jobs.

Table 2.3: Non-functional requirements and implementation responses.

2.8 Conclusion

This DSRM framing defines the rest of the report. Chapter 3 introduces the technical concepts needed to understand the artifact. Chapter 4 describes the OWC architecture, including the agent routing, browser tool layer, fingerprint and proxy controls, and channel-detection path. Chapter 5 then explains how the design decisions were implemented.

CHAPTER 3

Technical Background

3.1 Introduction

Before presenting the system architecture and implementation, I first introduce the technical concepts that make the project understandable. I keep this chapter at a general level so the required web structures, streaming mechanisms, browser automation tools, browser containers, and agentic concepts are clear before the concrete architecture. The database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, workflow automation, LangChain, MCP servers, model providers, caching, and token usage.

This section establishes the vocabulary needed before the design chapters: streaming-site page roles, DOM structure, embedded players, media manifests, browser automation obstacles, screenshots, network logs, prompts, tool use, LangChain, MCP, provider behavior, caching, and token usage.

3.2 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.2.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside an iframe loaded from another domain.

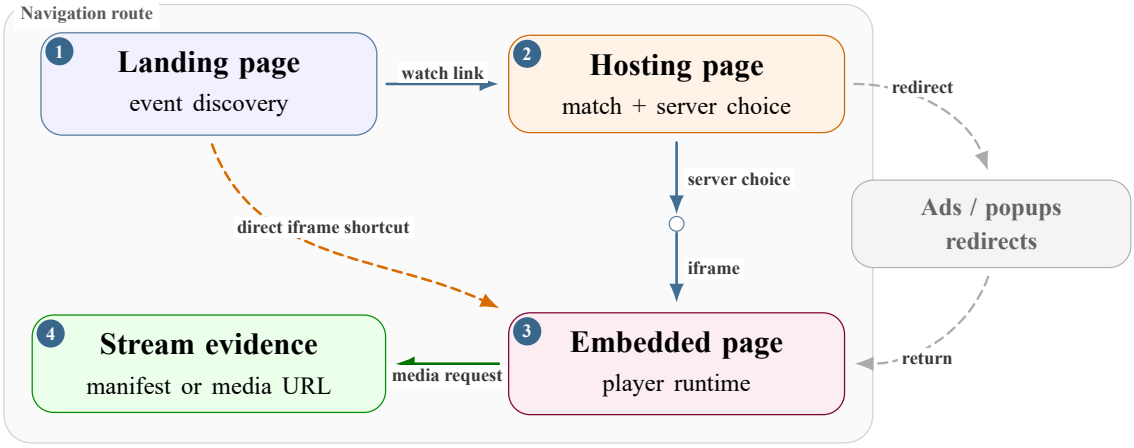


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

Figure 3.1 shows the navigation sequence as an abstract route. The next figure uses the same sequence from the browser's point of view: it highlights what each page role exposes to the automation pipeline and what output is expected from that role before the pipeline moves on.

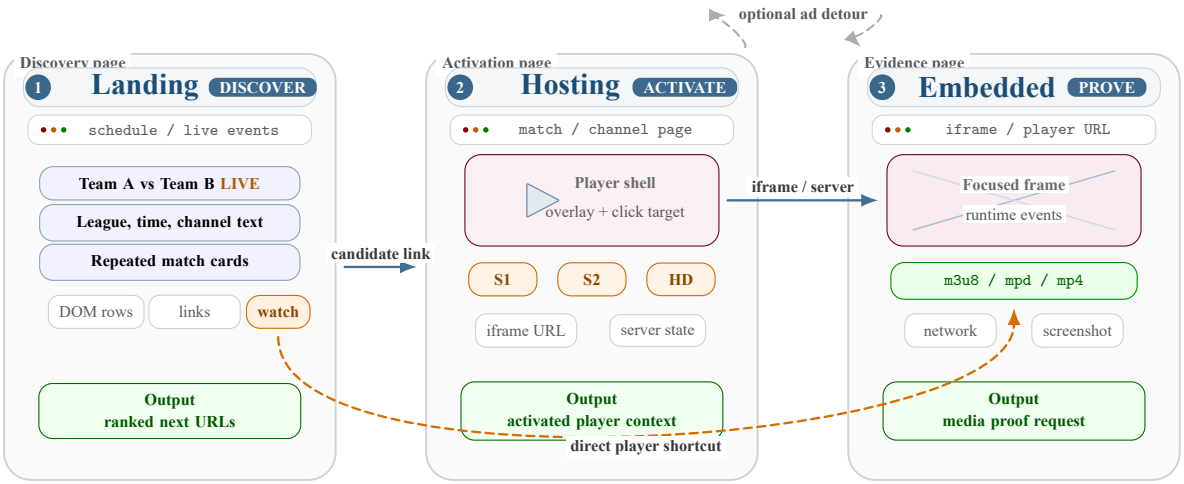


Figure 3.2: Browser-state view of landing, hosting, and embedded page roles.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still, this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.

3.2.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids**: repeated blocks with teams, time, league, and a watch link;

- **tables or lists**: one row per event, often with a direct action link;
- **tabbed schedules**: live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections**: match cards that appear only after scrolling or after an XHR request;
- **embedded JSON**: page data stored in `<script type=application/json>` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.2.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as `Server 1` or `HD`, and sometimes channel names or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an iframe or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;
- an iframe whose `src` points to a player page;
- player libraries such as Video.js, JWPlayer, Clappr, hls.js, or dash.js;
- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.2.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside iframes, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any `.m3u8` or `.mpd` URL, while the embedded player requests the manifest after user activation.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or iframe contains the player? Which runtime request proves that media was loaded?

Page type	Main role	Useful evidence
`landing_page`	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
`hosting_page`	Select server and activate player	server labels, iframe URLs, player state, page screenshot
`embedded_page`	Load the final media runtime	stream manifest, segment requests, player diagnostics
`other`	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: Streaming-site page taxonomy and evidence role.

3.3 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser's tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.3.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

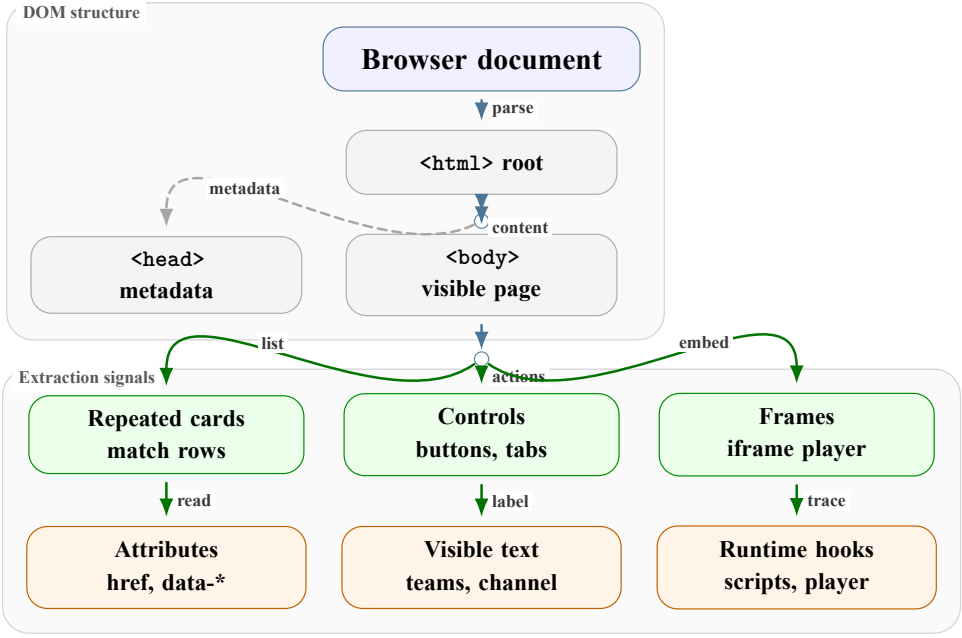


Figure 3.3: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.3.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
`<a href=…>`	landing and hosting pages	provides navigation targets
`data-*` attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
`<iframe src=…>`	hosting pages	points to embedded player context
`<video>` or `<source>`	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.4 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.4.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an iframe that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

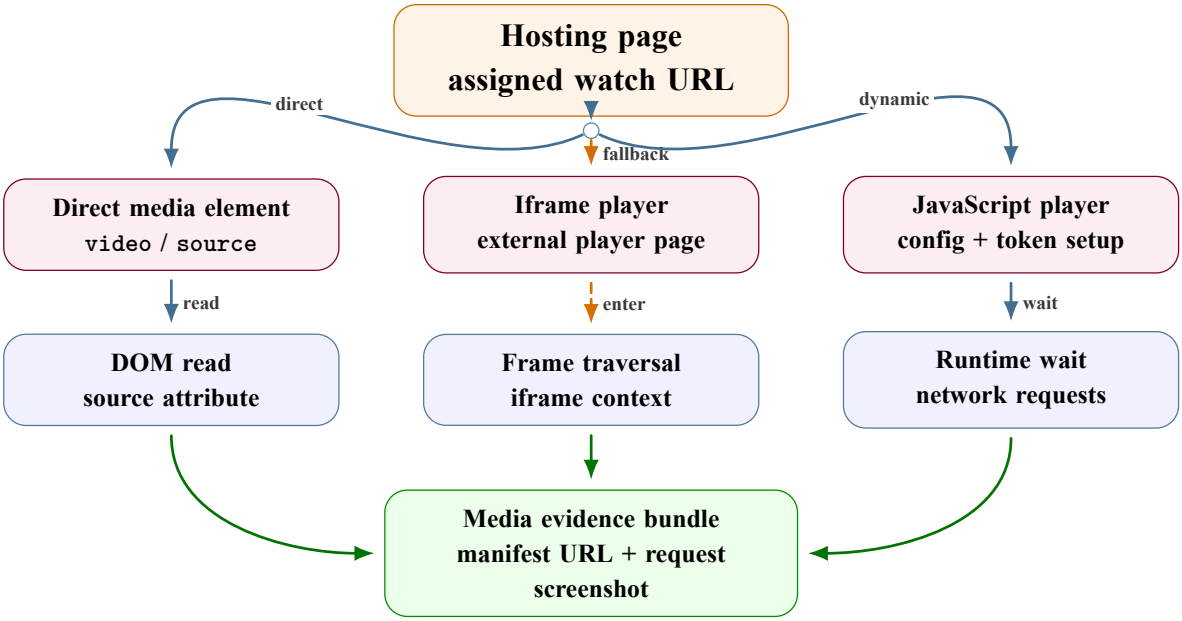


Figure 3.4: Main paths from a hosting page to media evidence.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.4.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a `.m3u8` playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML `.mpd` manifest that describes periods, representations, and segment addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [22, 23].

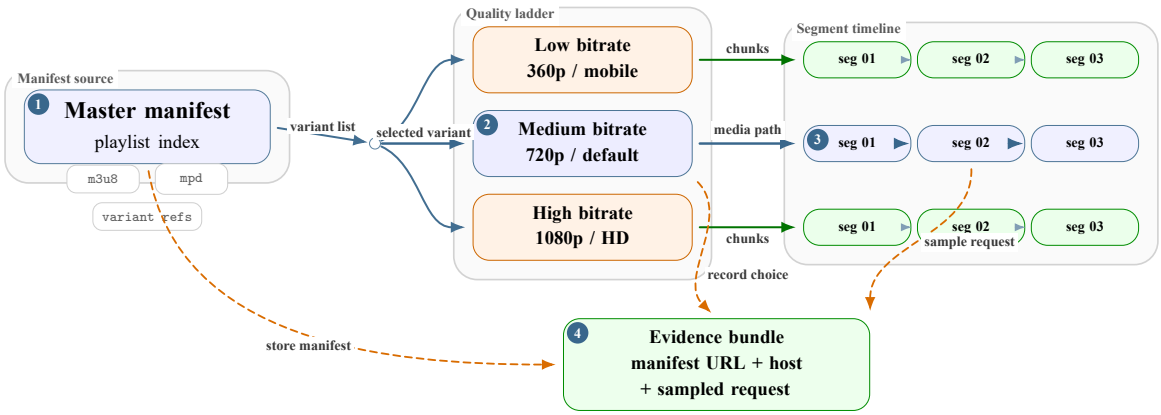


Figure 3.5: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

3.4.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=…` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection [21].

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. For that reason, the technical background includes both media protocols and web-provider behavior.

3.5 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Puppeteer-controlled Chromium sessions execute pages in a real browser and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity through the Chrome DevTools Protocol [7, 8].

3.5.1 Static collection versus rendered browser state

Static collection is efficient when the target evidence is present in the fetched HTML or in simple linked pages. Its limitation appears when the evidence is created only after a browser sequence:

- open a collapsed channel group;
- click a server tab;
- activate the player;
- wait for an iframe or player state;
- observe the network requests.

In those cases, the browser state is the evidence surface, not only the page source. This is why an evidence-collection system may add browser agents to static monitoring instead of treating the first page source as sufficient.

Rendered state as evidence.

Rendered state includes visible text, player overlays, frame trees, screenshots, media elements, and network requests that appear after interaction. It matters because the final stream or channel signal may not exist at the first page load. A screenshot, media-state record, or tool trace can explain why an agent believed the page exposed unauthorized streaming behavior.

A runtime evidence bundle is the combination of these observations after the page has executed. In this report, the word *evidence* therefore does not mean one DOM selector or one keyword. It means several browser-state views that agree enough for a reviewer to understand the result.

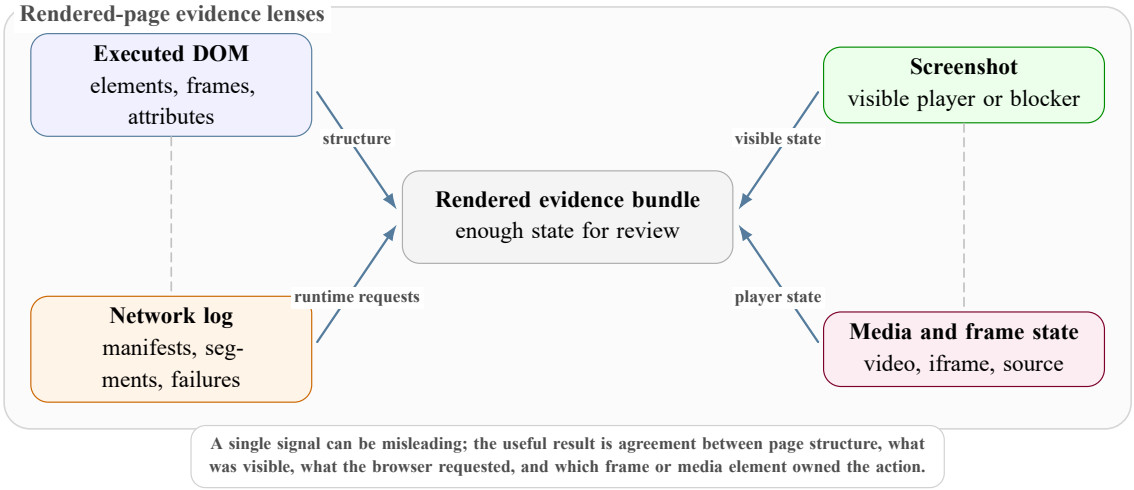


Figure 3.6: Rendered evidence as several browser-state views combined into one reviewable bundle.

3.5.2 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only "click a button" but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol (CDP);
- read browser-side state through page-evaluation calls.

Puppeteer acts as the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.5.3 Browser containers and session isolation

In this report, a browser container means the operational boundary around an automated browser session. It is not just the browser binary. It includes the browser engine, profile directory, cookies, local storage, network route, identity settings, extension policy, and the tool server that exposes controlled actions to the agent. Depending on deployment, this boundary may run inside a real infrastructure container, but the technical idea is session isolation: one controlled browser environment owns one coherent view of the target site.

This boundary matters for streaming evidence because media tokens, cookies, referers, and iframe state are often tied to the same browser profile. If a system opens one page with one identity and then collects media evidence from another unrelated session, the result can be misleading. A browser container keeps the automation, identity controls, and evidence logs close enough to explain what happened.

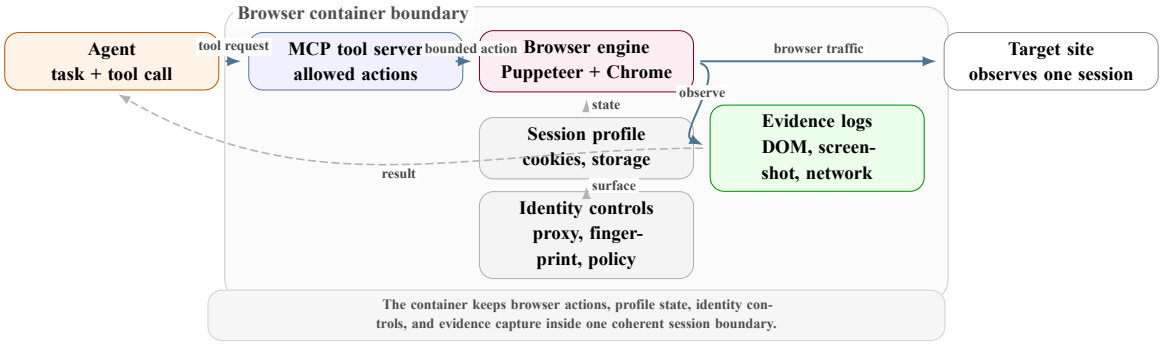


Figure 3.7: Browser-container isolation used for automated evidence collection.

3.5.4 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Browser-level obstacles and handling strategies.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [24]. However, they cannot replace browser reasoning. If a site serves ads and player assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.5.5 Browser identity, proxy rotation, and fingerprinting

Browser automation also has an identity problem. A target website does not only see the URL request. It can observe the source IP address, TLS and HTTP headers, user-agent string, client-hint headers, language, screen size, browser APIs, installed-like features, cookies, local storage, timing, and behavior. The combination of these signals is commonly called a browser fingerprint.

Proxy rotation changes the network route by sending traffic through a different proxy server. Its purpose is not only anonymity; in streaming extraction it can also help separate regional blocks, failed proxy candidates, and normal site failures. A reliable rotation strategy should validate candidate proxies, keep one proxy stable during a session, and rotate only when policy or failure requires it. Rotating the IP address on every click can make a browser session less coherent because cookies and tokenized media URLs may be tied to the previous route.

Fingerprint rotation changes the browser surface that the site sees. A coherent fingerprint means the user agent, platform, language, screen, and client hints agree with each other. If these values contradict each other, the automation looks less like a normal browser. For that reason, fingerprint rotation is usually applied at browser profile or session boundaries, not randomly inside one page interaction.

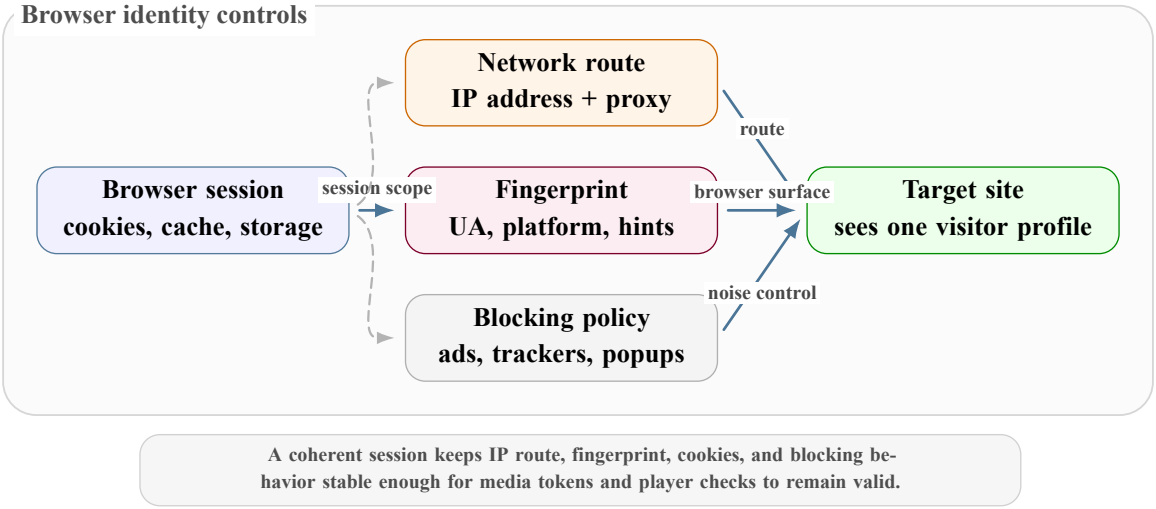


Figure 3.8: Browser identity controls used by automated evidence collection.

3.5.6 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.6 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.6.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing. In framework-based agent systems, this loop is usually represented through model messages, tool calls, observations, and structured outputs exposed by the runtime [13].

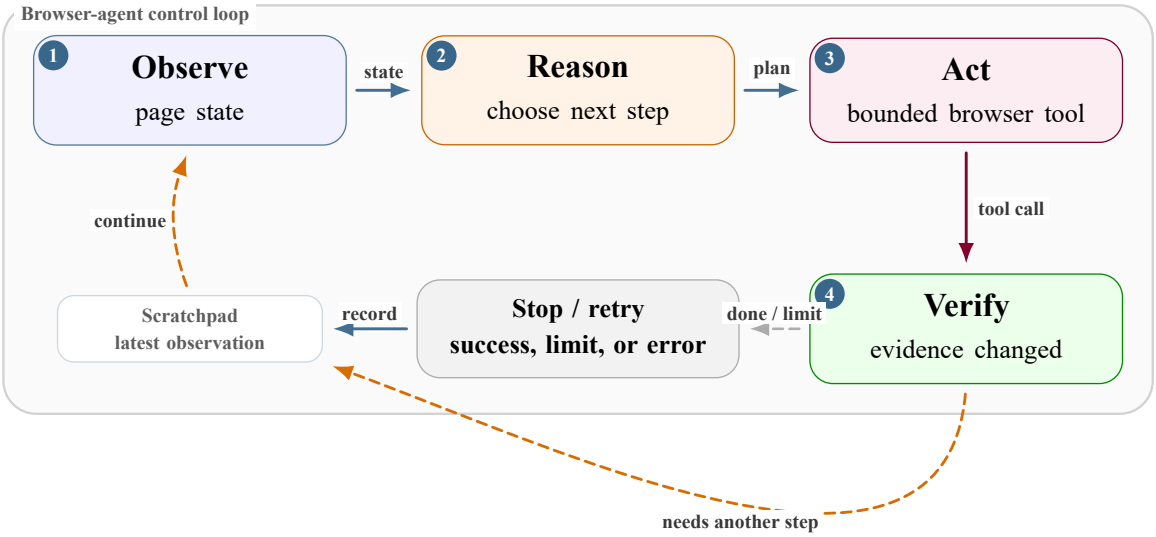


Figure 3.9: Observe, act, and verify loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.6.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of designing instructions, examples, constraints, context, and output requirements so that a language model behaves consistently for a task. It is not only about writing a clear question. In an agentic system, the prompt also defines how the model should use tools, what evidence it should trust, when it should stop, and how it should format the result.

Several prompt-engineering techniques are useful background for this project. They can be grouped into foundational techniques, reasoning techniques, and operational techniques:

Family	Technique	General idea	Relevance to browser agents
Foundational	**Zero-shot prompting**	Give the task instruction directly, without examples.	Useful for simple classification or summarization, but weak when page layouts and evidence rules vary.
Foundational	**Few-shot prompting**	Include a few input-output demonstrations that show the desired format and style.	Helps stabilize output shape when examples are representative, but can overfit the model to a small set of page patterns.
Foundational	**Role or persona prompting**	Assign a domain role, such as analyst, reviewer, or investigator.	Improves vocabulary and decision framing, but does not by itself guarantee correct browser actions.
Advanced reasoning	**Chain-of-thought prompting**	Ask the model to work through the problem step by step or provide a concise rationale.	Helps with multi-step interpretation, but can still guess if the prompt does not require tool evidence.
Advanced reasoning	**Tree-of-thoughts prompting**	Explore several possible solution paths, compare them, and select or backtrack from the best path.	Useful for ambiguous diagnosis, but expensive when every branch would require browser actions.
Advanced reasoning	**Step-back prompting**	First ask a broader abstraction question before solving the specific task.	Helps when the model needs to identify the page role or general strategy before choosing an action.
Operational	**Prompt chaining**	Split a large objective into ordered prompts, where one output becomes the next input.	Useful for staged workflows such as classify, extract, enrich, and draft.
Operational	**Generated-knowledge prompting**	Ask the model to generate background facts or assumptions before writing the final answer.	Risky for live websites unless generated knowledge is treated as a hypothesis and rechecked with tools.
Operational	**Structured-output enforcement**	Require a strict output format such as JSON, tables, or tagged sections.	Essential when downstream code must parse agent results reliably.
Operational	**ReAct prompting**	Combine reasoning with actions and observations: observe, decide, use a tool, verify, and repeat.	Best fit for browser evidence collection because the next step depends on the latest page state.

Table 3.4: Prompt-engineering technique families relevant to agentic evidence collection.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and can also increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

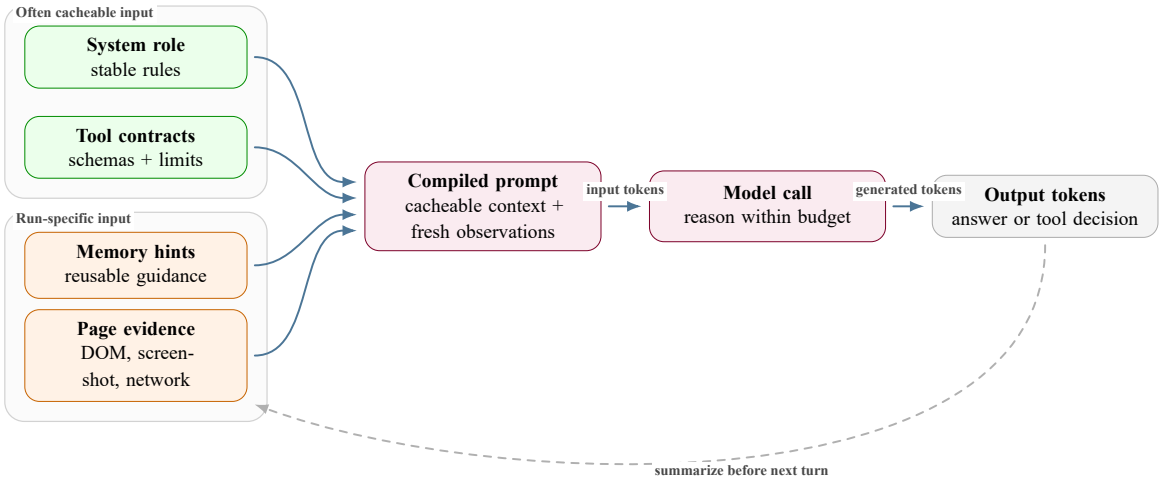


Figure 3.10: Prompt and token-budget composition for an evidence-driven browser agent.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, pricing, caching, and rate limits [2, 3, 1]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

For an agentic browser workload, caching has two different meanings. The first is **provider-side prompt caching**: a model provider can charge repeated input tokens differently when a stable prefix is reused. This matters because browser agents send the same policy, role contract, output schema, and tool-use rules on many calls. If those sections remain in a consistent order, the provider has a better opportunity to count them as cached input instead of fresh input [3]. The price difference is operationally important because a batch run may call the model thousands of times, and input tokens are not only the user's short instruction: they also include the system prompt, tool schemas, memory hints, browser observations, and trace context.

The second meaning is **tool-output caching**. A browser tool may return a compact DOM summary, a frame tree, media state, candidate links, or network evidence. Once the runtime sends that result back to the model, it becomes part of the next model input. Repeatedly inserting the same large observation can therefore be more expensive than the browser call itself. Caching

a safe repeated observation avoids both repeated browser work and repeated token growth. It must be conservative: observations that depend on a changed page state, a click, a navigation, or a media action should be invalidated instead of reused.

This is why cache-friendly prompt assembly is part of the architecture, not only a cost optimization. A naive tool-using LangChain setup can rebuild the prompt, memory block, tool schema order, and message history differently on each turn. That is valid from an agent-control point of view, but it weakens provider cache hits because the repeated prefix is no longer stable. In contrast, a runtime that separates a stable prefix from a dynamic suffix gives caching a predictable boundary while keeping the current URL, working state, and memory hints fresh.

3.6.3 Workflow automation tools

Workflow automation tools let developers connect HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow. They are useful for fast validation because a workflow can be assembled and adjusted before a full application is built.

In this report, workflow automation is treated as an orchestration concept. It helps to:

- sequence evidence-collection steps;
- pass data between browser, model, and processing nodes;
- call model providers in a visible workflow;
- make experimental extraction logic easier to inspect.

The specific workflow tool used during implementation is introduced in Chapter 5.

3.6.4 Agent frameworks, graph workflows, and tool servers

Modern LLM applications often need more than a single prompt-response exchange. They need a way to pass messages to a model, expose external tools, keep state between steps, branch to different tasks, and produce a result that another program can check. This is the general problem space where frameworks such as LangChain, LangGraph, and MCP become useful.

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions [13].

LangGraph is related but more focused on stateful control flow. Instead of treating an agent workflow as one long script, a LangGraph-style design represents the workflow as named nodes, shared state, and conditional edges [14]. A node can read the current state, perform one bounded operation, update the state, and let an edge decide the next step. This is useful for browser agents because extraction has branches: one page may reveal another target, a tool action may fail, or the workflow may need to stop when enough evidence has been collected.

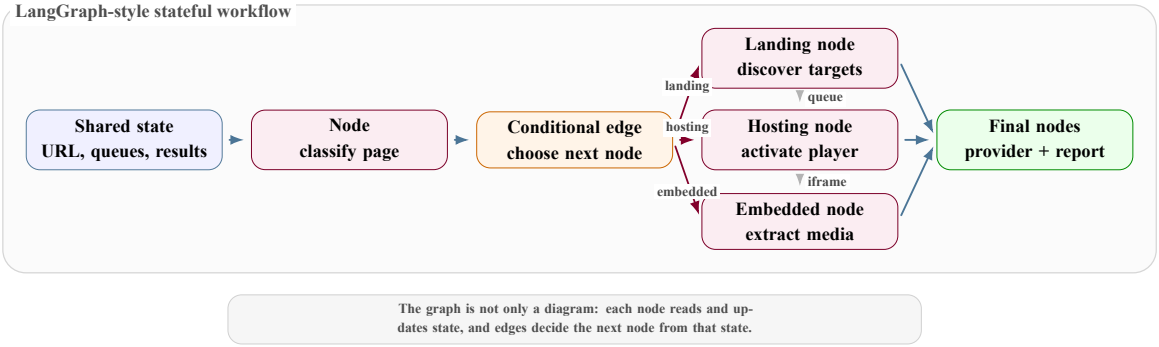


Figure 3.11: LangGraph workflow concept: named nodes, shared state, and conditional routing.

The conceptual separation is simple: LangChain is mainly about model and tool interfaces, while LangGraph is mainly about workflow state and routing. The application domain still matters because the framework does not know by itself what counts as good evidence, which routes are allowed, or what output is acceptable.

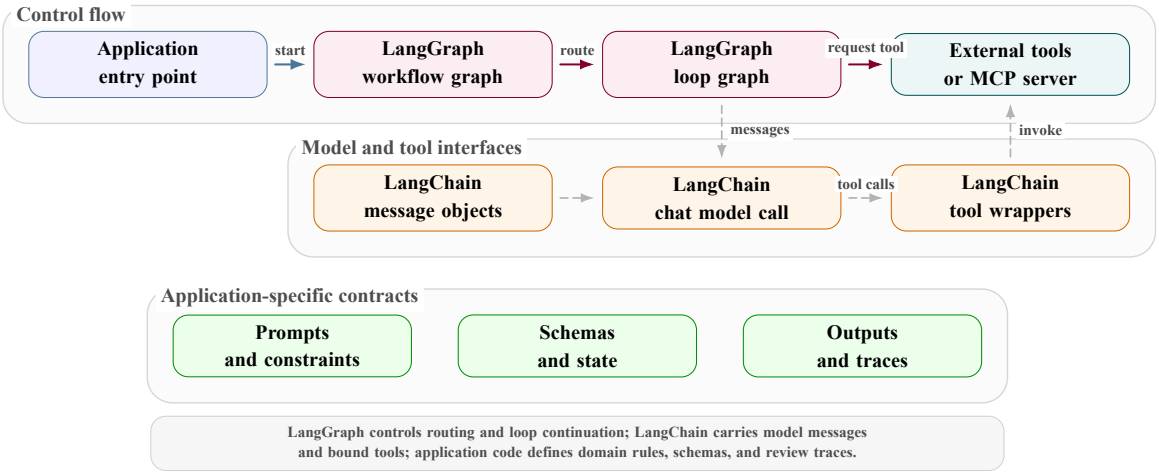


Figure 3.12: Separation between LangGraph control flow, LangChain interfaces, tools, and application contracts.

A typical tool-using model loop follows the same pattern regardless of the exact framework. The system gives the model instructions and context. The model either returns a final answer or asks for a tool call. The runtime invokes the tool, converts the result into a message, and sends that observation back to the model. The loop continues until the runtime reaches a final answer, a budget, a timeout, or another stop condition.

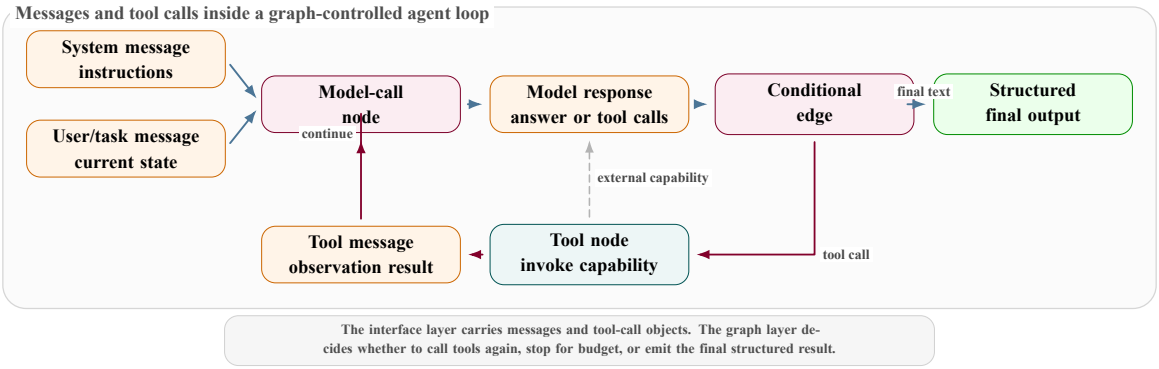


Figure 3.13: Message and tool-call cycle controlled by a stateful agent loop.

Concept	General responsibility	Typical question it answers
Model interface	Send system, user, model-response, and tool-result messages to the selected LLM.	What information does the model see at this turn?
Tool interface	Describe callable functions with names, arguments, limits, and structured results.	What external action or lookup can the model request?
State graph	Represent the workflow as nodes, shared state, and conditional edges.	Which step should run next?
Domain contract	Define the task-specific rules, evidence standard, output shape, and stopping criteria.	What counts as a valid result for this problem?
Trace and review layer	Record decisions, tool calls, observations, errors, and final outputs.	How can a person inspect what happened later?

Table 3.5: General responsibilities in a tool-using LLM workflow.

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape [15].

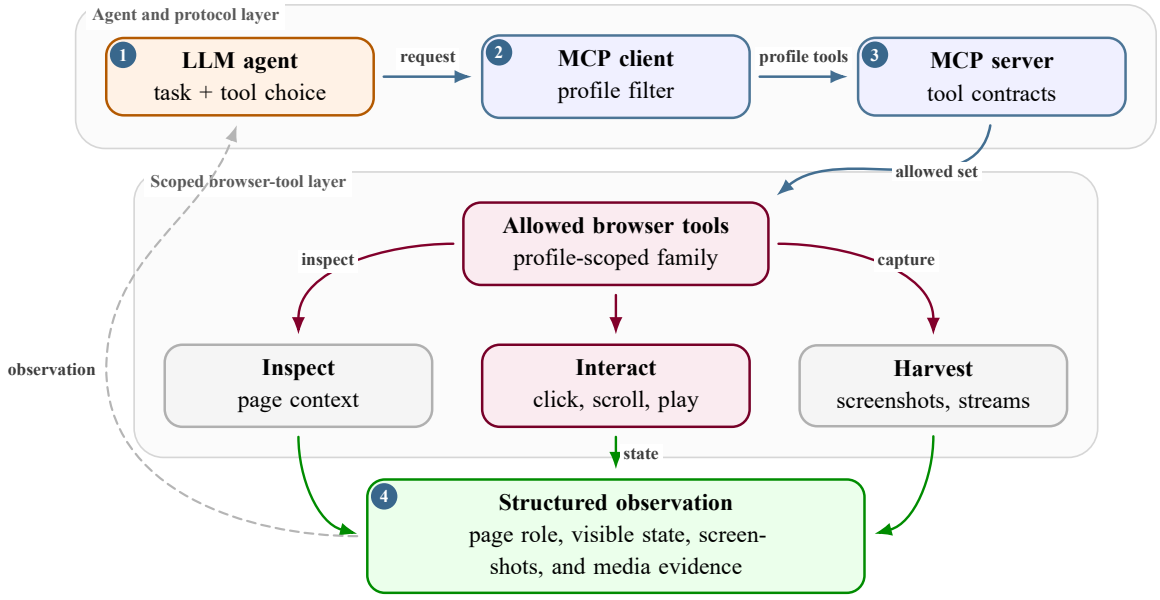


Figure 3.14: MCP bridge connecting an LLM agent to scoped browser tools.

A tool contract is the small boundary between model reasoning and executable browser code. It names the tool, limits the accepted arguments, describes the bounded action, and returns a structured result instead of free text. This is why the later implementation can expose many browser capabilities without asking the model to invent browser-control code.

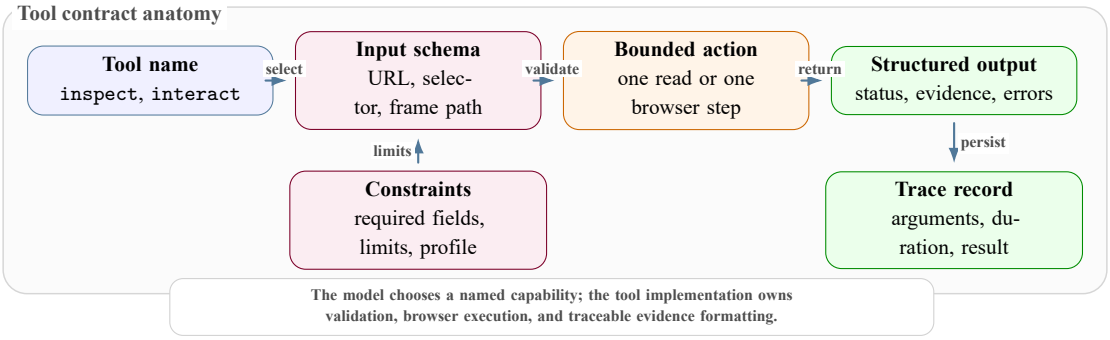


Figure 3.15: Tool contract structure connecting an agent decision to executable browser code.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.7 Conclusion

With this background in place, the report can now move from general concepts to the system design. This chapter explained:

- the page flow from landing pages to hosting pages and embedded players;
- the DOM structures used to inspect those pages;

- the streaming protocols and player embeds that carry final evidence;
- the browser obstacles caused by ads, cookies, popups, iframes, and provider checks;
- browser identity concepts such as proxy rotation, fingerprint rotation, and ad-blocking policy;
- the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, workflow automation, browser containers, LangChain, LangGraph, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

CHAPTER 4

System Architecture

4.1 Introduction

I present the architecture of Open Web Catcher (OWC) as the evidence-collection platform developed during the internship. The chapter starts with the user-facing system and runtime components, then narrows into the multi-agent pipeline, agent handoffs, browser tooling, and persisted review data.

Here, OWC refers to the reference platform I built during the internship: backend, console, browser tools, specialist agents, runtime telemetry, persistence, and review outputs. Chapter 5 then maps these architectural decisions to the implementation.

The system is a staged evidence pipeline, not a single unrestricted browser agent. FastAPI starts and records runs. The orchestrator owns the route, handoffs, queues, aggregation, provider analysis, and email drafting. Four browser-facing specialists perform bounded work: classification, landing-page discovery, hosting-page extraction, and embedded-player extraction. Each specialist uses profile-scoped MCP browser tools, returns typed output, and leaves events that explain success or failure.

4.2 System Purpose and Architecture Reading Path

The platform is an evidence-collection workbench for illegal streaming websites. Its roles are separated on purpose:

- the operator launches investigations and reviews evidence;
- the engineer or administrator maintains models, prompts, pricing, datasets, browser settings, and MCP tools;
- external services supply target sites, model inference, screenshot storage, and provider data.

The useful architectural view is not a generic use-case picture. It is the path from an operator action to the records that make the action reviewable later.

I present the architecture in four layers:

1. **Product layer**: the operator console launches runs, monitors progress, and reviews evidence.
2. **Runtime layer**: FastAPI validates runtime readiness, schedules jobs, resolves settings, and persists trace data.
3. **Agent layer**: the orchestrator routes work through specialized agents instead of asking one prompt to do every task.

4. **Evidence layer**: streams, screenshots, provider rows, emails, events, model calls, and tool calls are normalized for later review.

Console responsibility	Operator intent	Persisted or observable response
Launch investigation	Submit one URL, a single-agent test, or a dataset batch.	Create a traceable run, validate runtime readiness, and schedule execution.
Monitor execution	Follow agent transitions, tool calls, screenshots, events, and failures.	Stream active state and persist each important runtime event.
Review evidence	Inspect streams, screenshots, provider analysis, metrics, and takedown drafts.	Assemble a reviewable evidence bundle with final status and supporting trace data.
Configure system	Tune models, prompts, MCP tools, browser settings, pricing, and datasets.	Persist configuration and expose health or preflight feedback before runs start.

Table 4.1: Operator-facing responsibilities and the runtime records they create.

4.3 Global Runtime Architecture

The runtime separates the operator console, API, agent execution, browser automation, persistence, and external enrichment services. The Next.js console does not access the database directly; it calls FastAPI endpoints. FastAPI owns background-job execution, active trace assembly, settings resolution, and database access. Browser work is delegated to profile-scoped MCP tooling so the agents receive only the tools allowed for their page type.

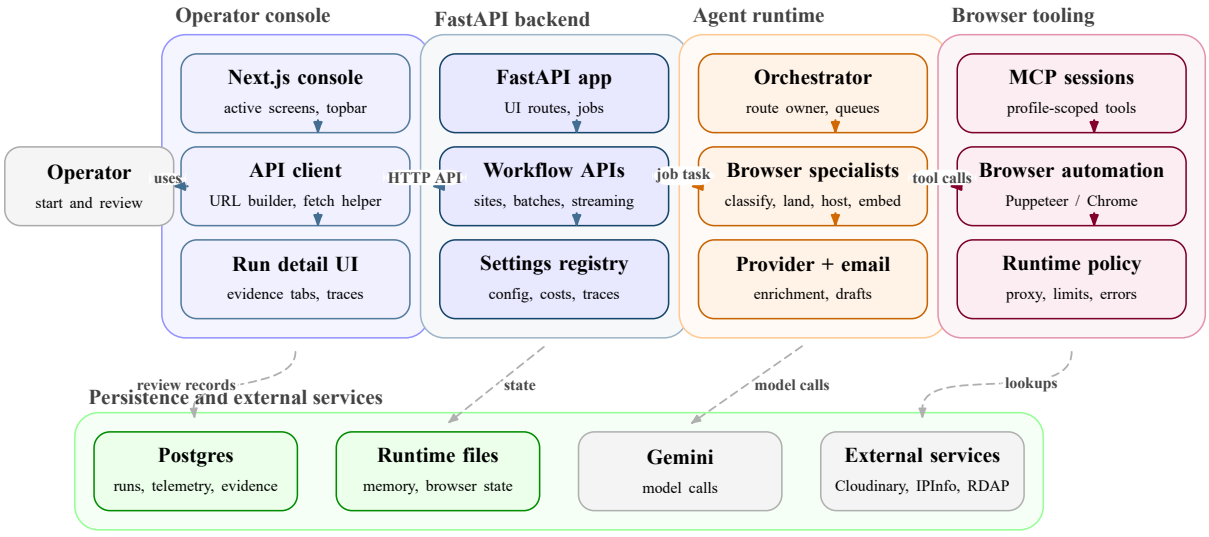


Figure 4.1: OWC runtime component boundaries from operator action to persisted evidence.

Runtime area	Main responsibility	Why it is separate
Operator console	Launch workflow runs, inspect run detail, manage datasets, providers, tools, browser settings, and model configuration.	The UI stays thin; it consumes API payloads instead of owning persistence.
FastAPI backend	Validate preflight state, create jobs, run background work, assemble active and persisted traces, and expose review payloads.	Runtime control and database access remain testable from backend routes.
Agent runtime	Execute the orchestrator graph and the specialist model/tool loops.	Routing policy remains explicit instead of hidden inside one prompt.
MCP browser tooling	Provide profile-scoped page, frame, screenshot, interaction, and network tools.	Browser dependencies and page failures are isolated from the API process.
Persistence and services	Store normalized run records and call Gemini, Cloudinary, IPInfo, RDAP, and local runtime files.	The final answer and the failure path can be reviewed after execution.

Table 4.2: Global runtime boundaries used by OWC.

The frontend route map is part of the architecture because the console is the operator's entry point into the runtime. The active screens are the dashboard, live pipeline, results, run detail, provider lookup, and settings pages; the `/agents` path is kept only as a redirect to `/live`. The route map starts with the shared console shell, then moves to the active screens, and finally shows the FastAPI route groups consumed by each screen.

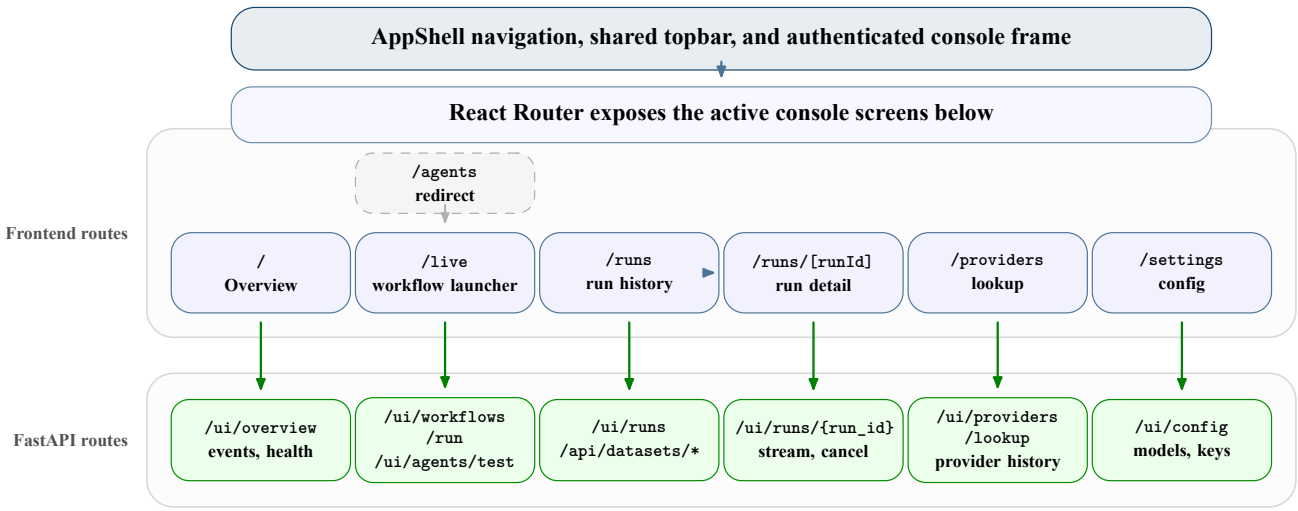


Figure 4.2: Frontend console routes and API route groups.

This separation is practical rather than cosmetic. Browser automation, LLM reasoning, dashboard rendering, and persistence fail in different ways. Keeping their boundaries visible makes failures easier to classify: a browser-tool timeout, a model retry, a missing stream, a skipped provider lookup, and a dashboard payload issue are different problems.

4.4 End-to-End Agent Pipeline

At large scale, the platform behaves as a controlled multi-agent pipeline, not as one unrestricted browser agent. The operator gives the system a seed URL. The API starts a run, creates an active trace, and calls the orchestrator. The orchestrator checks memory as soft guidance, calls classification, routes to the appropriate specialist path, collects evidence, and only then runs provider analysis and email generation.

The complete pipeline view is shown in Figure 4.3. It starts with the operator and API readiness check, then follows the orchestrator as it initializes pipeline state, reads memory as soft guidance, calls classification, and either records an unsupported page or prepares bounded handoffs for specialist work. The same activity diagram continues through the specialist-extraction loop to the reviewable result. Landing, hosting, and embedded work can add more hosting or embedded targets, but every turn returns typed JSON to the orchestrator. The orchestrator then normalizes evidence, aggregates matches, streams, screenshots, and failures, and decides whether downstream provider and email stages are allowed to run.

Across this pipeline, there are four important rules:

- **Readiness gates execution**: browser, tool, or API setup problems return blocking reasons before the agent graph starts.
- **Agents do not call each other directly**: every specialist returns typed output to the orchestrator.
- **The orchestrator owns queues**: landing can add hosting or embedded targets, hosting can add more hosting or embedded targets, and embedded returns final media evidence.
- **Downstream stages require streams**: provider analysis and email generation are skipped when no stream URL exists.

The stage-specific sequence diagrams later in the chapter then zoom into timing inside the orchestrator and each browser-facing specialist.

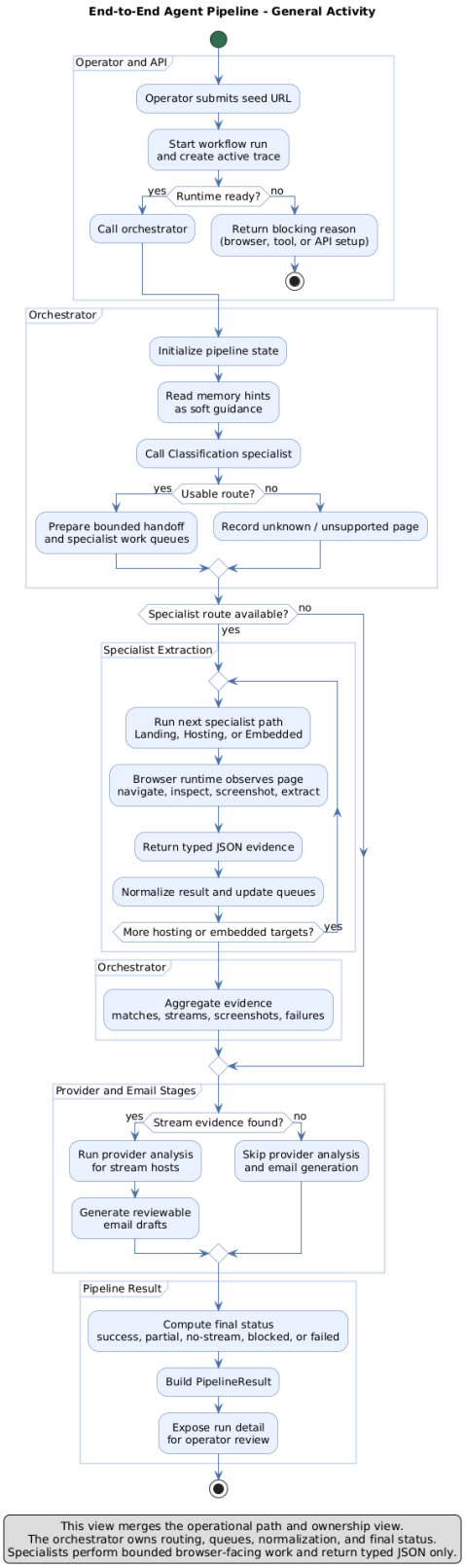


Figure 4.3: End-to-end agent pipeline as one activity diagram, from operator launch and runtime readiness through orchestrator routing, specialist extraction, provider analysis, email drafting, and the final `PipelineResult`.

4.5 Agent Communication and Orchestrator Handoffs

The orchestrator is the routing authority. It builds the LangGraph state machine, checks memory as soft guidance, calls classification, prepares handoffs, executes landing, hosting, and embedded specialists as needed, aggregates structured outputs, then runs provider analysis and email generation only when stream evidence exists. This keeps failures interpretable: a classification timeout, missing hosting candidate, hosting-player failure, embedded-player failure, and provider skip are different operational states.

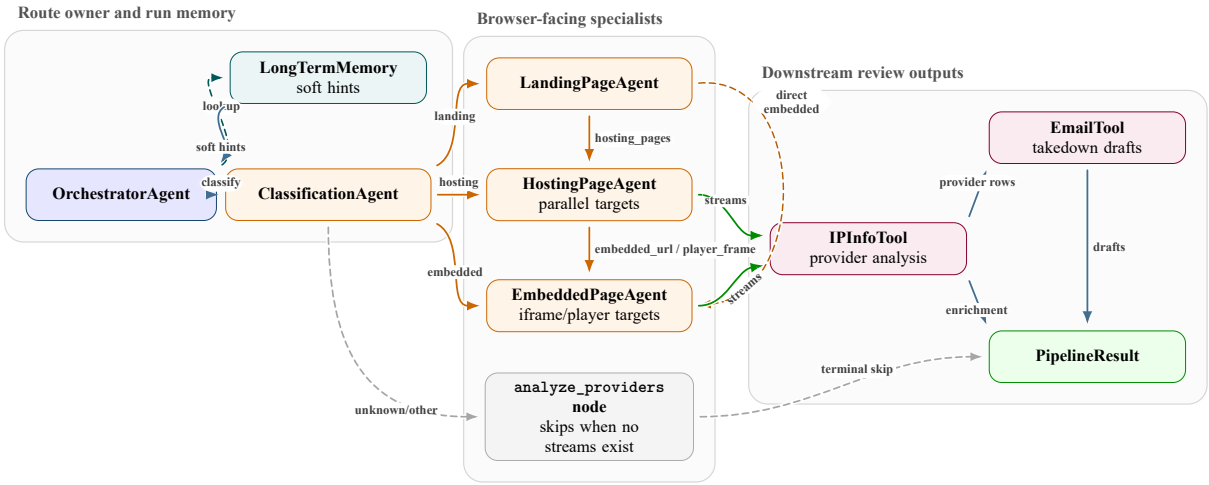


Figure 4.4: Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.

The handoff is the main communication contract between agents. It contains the root URL, target URL, page type, classification reasoning, route source, memory hints, navigation policy, required evidence, and known pattern context. A specialist receives this context inside its task brief, performs only its bounded job, and returns a schema that the orchestrator can normalize and route.

In this design, communication is visible at three levels:

- route decisions are graph edges, so the next stage is explicit;
- handoff text gives each specialist the local context it needs;
- the observer records agent starts, model calls, tool calls, screenshots, status changes, costs, and final rollups.

That is why the dashboard can explain what happened instead of only showing a final status. The concrete JSON fields used by the current schemas, orchestrator, and prompts are summarized later in Table 4.7.

Prompt compilation is also part of the communication contract. Each browser-facing specialist receives the same basic ingredients but a different stage contract:

- the base policy and agent-specific instructions;
- the URL task brief and runtime context;
- long-memory site hints and short-memory working state.

Stage	Main input	Responsibility	Main output
Orchestrator	Seed URL, settings, observer, and memory hints.	Build the LangGraph route, prepare handoffs, aggregate results, compute final status, and call downstream tools.	`PipelineResult`, trace events, provider rows, and email drafts.
Classification	URL and profile-scoped browser context tools.	Decide page type without extracting streams or provider data.	Classification result with page type, confidence, and reasoning.
Landing	Landing URL plus orchestrator handoff.	Discover watch or hosting candidates while preserving match, iframe, channel, and player hints.	`ExtractionResult` with match records and hosting or embedded candidates.
Hosting	Assigned hosting URL plus evidence checklist and navigation policy.	Operate server/source controls, activate playback, collect network evidence, and return explicit embedded handoffs when needed.	Screenshots, server rows, stream URLs, diagnostics, or embedded player URLs.
Embedded	Verified player or iframe URL plus handoff context.	Stay inside the assigned player context, recover from drift, and harvest final stream evidence.	Stream URLs, screenshots, frame diagnostics, and player state.
Provider and email tools	Stream URLs, provider rows, screenshots, and extractions.	Enrich stream hosts and draft reviewable takedown notices.	`ProviderInfo` records and `TakedownEmail` drafts.

Table 4.3: Agent and tool responsibility map.

Short memory is run-local. It tracks recent navigation, tool calls, selectors, candidates, streams, server records, screenshots, and the next working objective. Long memory is cross-run site memory. It stores summarized playbooks so a future run can receive hints without trusting them blindly. In both cases, live browser evidence still decides the route.

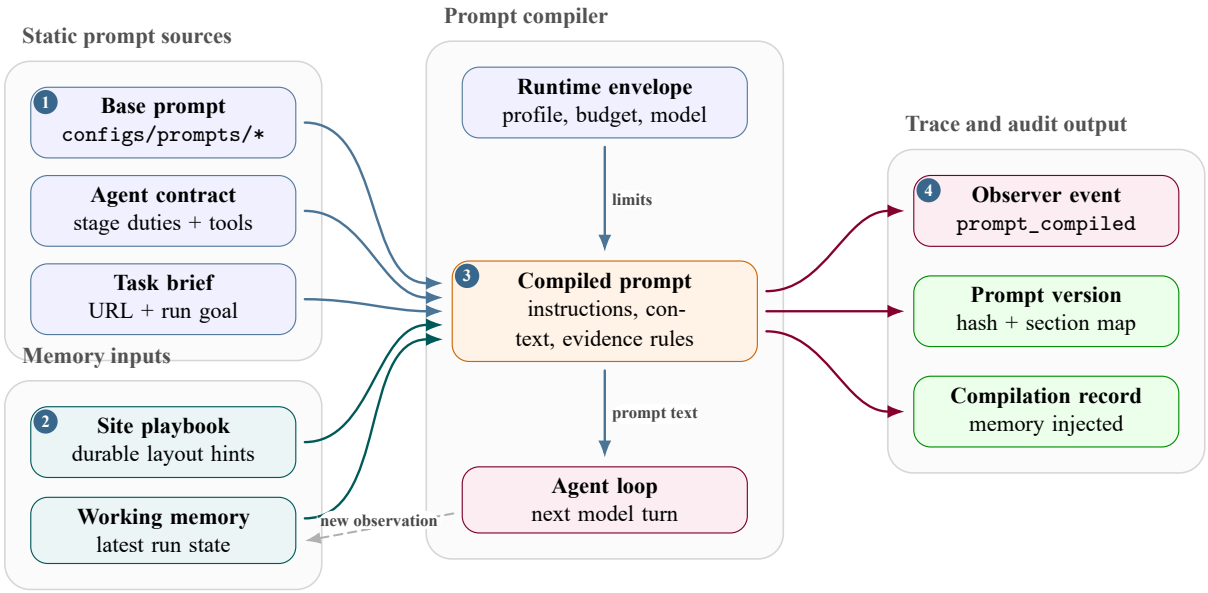


Figure 4.5: Prompt compilation and memory inputs for each browser-facing specialist.

That compilation step explains what each model call receives. The shared loop below focuses on what happens after the prompt is ready: browser inspection, tool use, observation recording, and normalized output.

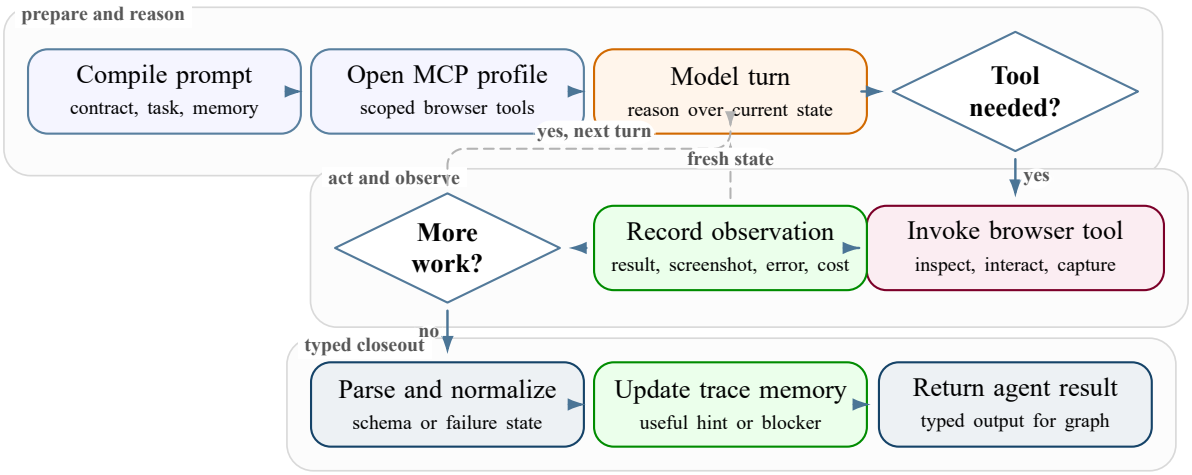


Figure 4.6: Shared browser-agent loop for prompt compilation, tool use, verification, and structured output.

The browser-facing specialists then reuse the same inner loop. Each one compiles its prompt, opens the correct MCP browser profile, lets the model request tools, records every model and tool turn, controls context growth, and normalizes the final answer into the stage schema.

4.6 Agent-by-Agent Design

4.6.1 Orchestrator Agent

The orchestrator is the control plane for one run. It owns state, queues, handoffs, aggregation, provider and email execution, final status, and trace emission, but it does not inspect pages directly.

Figure 4.7: Orchestrator activity from run setup through classification, specialist routing, aggregation, provider/email handling, and final `PipelineResult`.

The activity view is intentionally procedural. The next card summarizes the orchestrator as a component: its inputs, decisions, outputs, and failure signals.

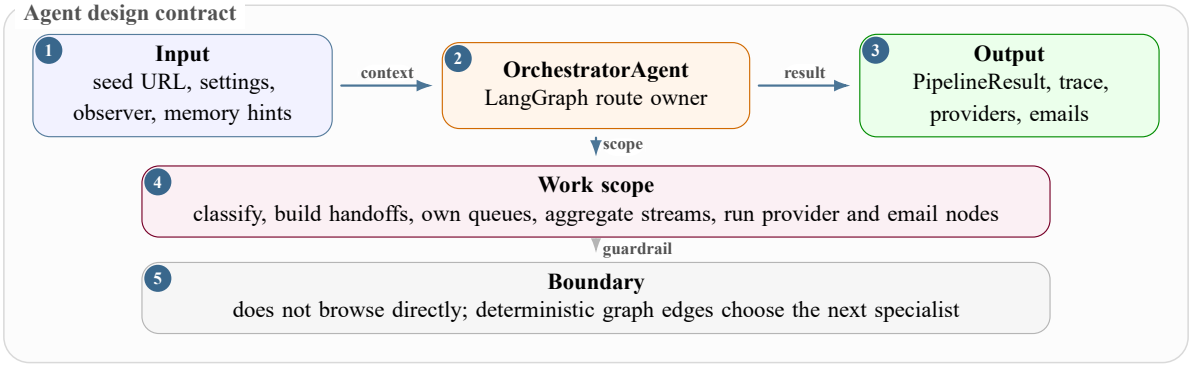


Figure 4.8: Orchestrator-agent contract map.

The sequence view closes the orchestrator subsection by showing the real-time handoff pattern: route preparation, specialist execution, trace events, and the final route decision.

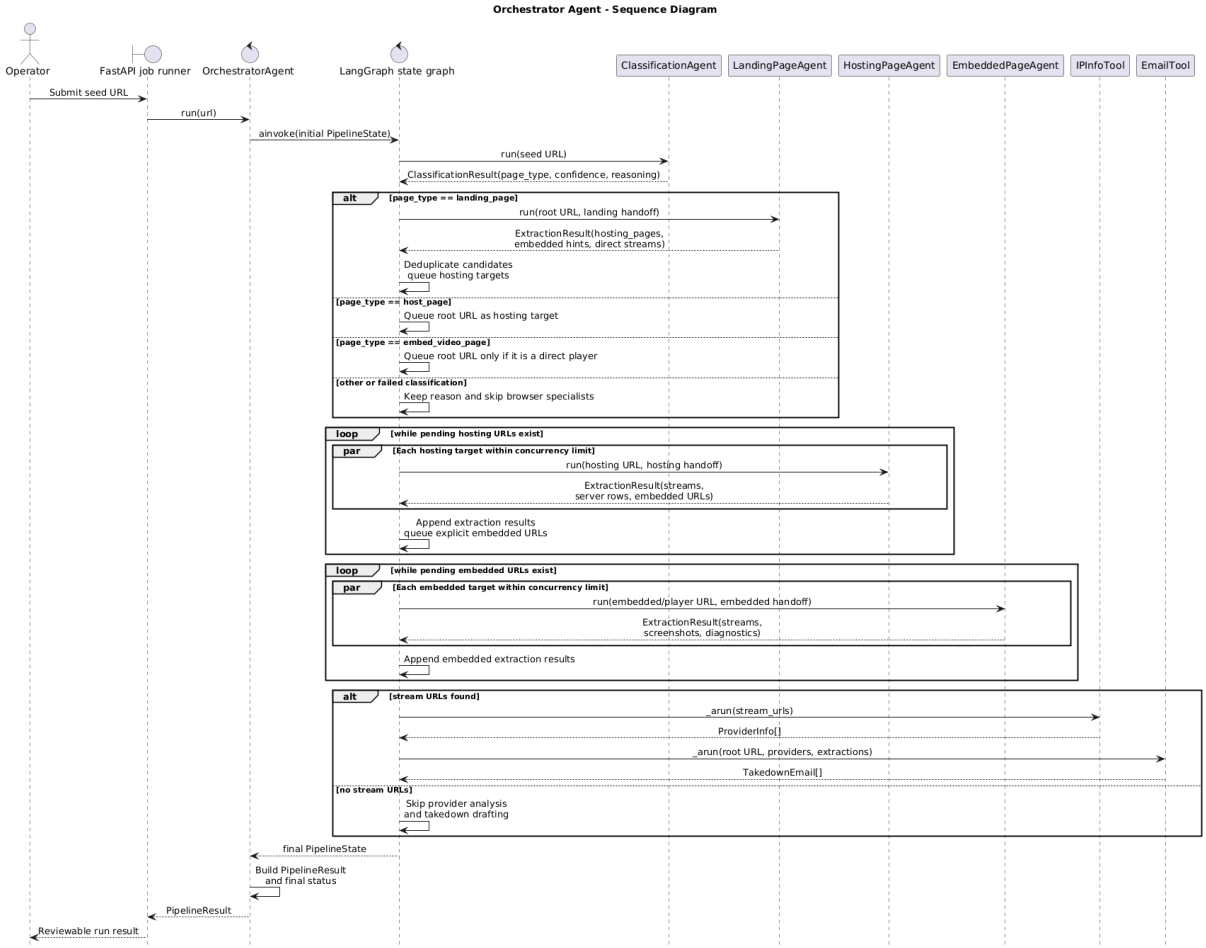


Figure 4.9: Sequence diagram for orchestrator handoff, trace recording, and route updates.

4.6.2 Classification Agent

The classification agent answers one question: what type of page is this URL? It distinguishes landing, hosting, embedded, unknown, and irrelevant pages before extraction begins.

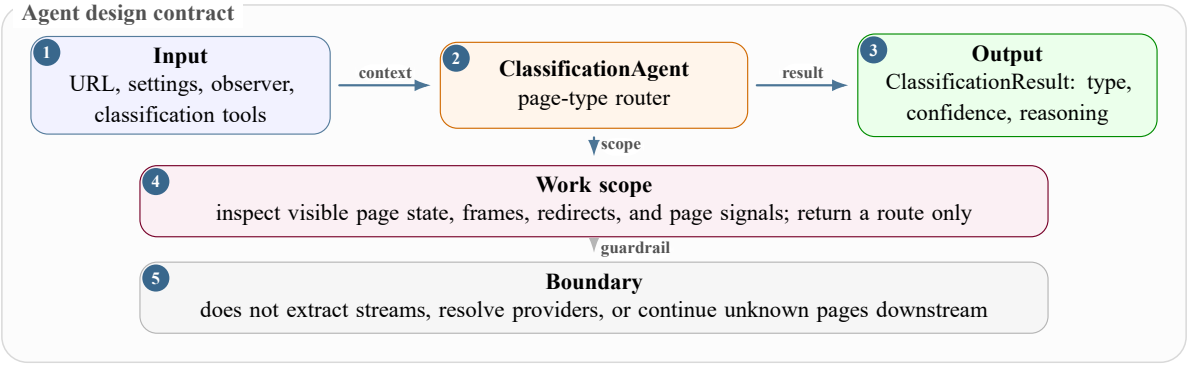


Figure 4.10: Classification-agent contract map.

A landing result sends the run to the landing agent. A hosting result queues the root URL for the hosting agent. An embedded result queues the root URL for the embedded agent only when the page looks like a direct player. The sequence view shows this decision as a bounded classification exchange that returns only a typed route result, not stream evidence.

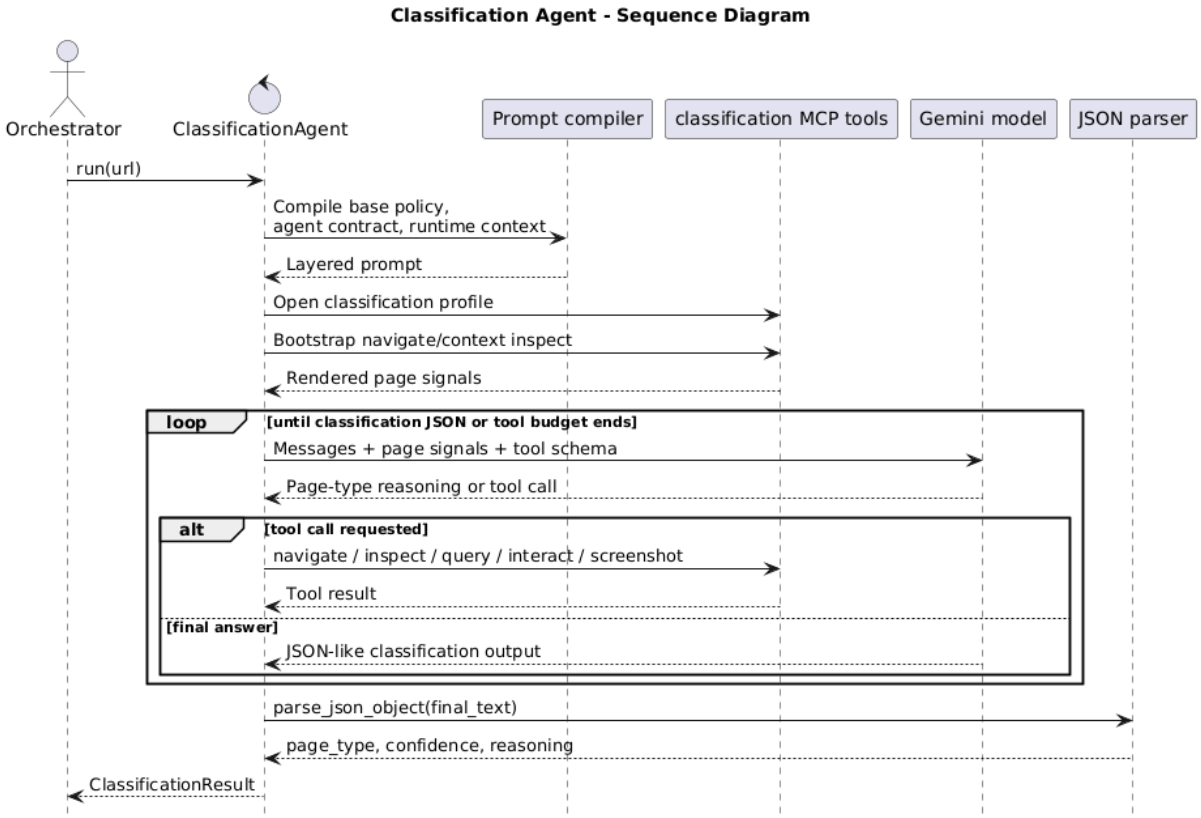


Figure 4.11: Classification-agent sequence: inspect page state and return a routing decision.

4.6.3 Landing-Page Agent

The landing-page agent turns broad site pages into bounded downstream targets. It discovers useful watch, hosting, embedded, or direct-stream candidates from homepages, schedule pages, directory pages, and match listings.

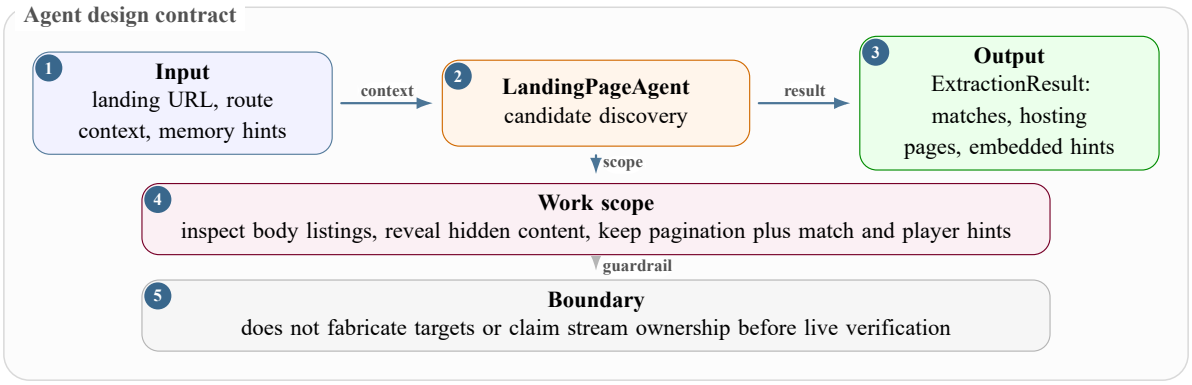


Figure 4.12: Landing-page-agent contract map.

The orchestrator deduplicates landing outputs: watch or server pages enter the hosting queue, direct player URLs enter the embedded queue, and provider-like media URLs remain stream evidence. The sequence view focuses on candidate discovery and queue updates, not playback.

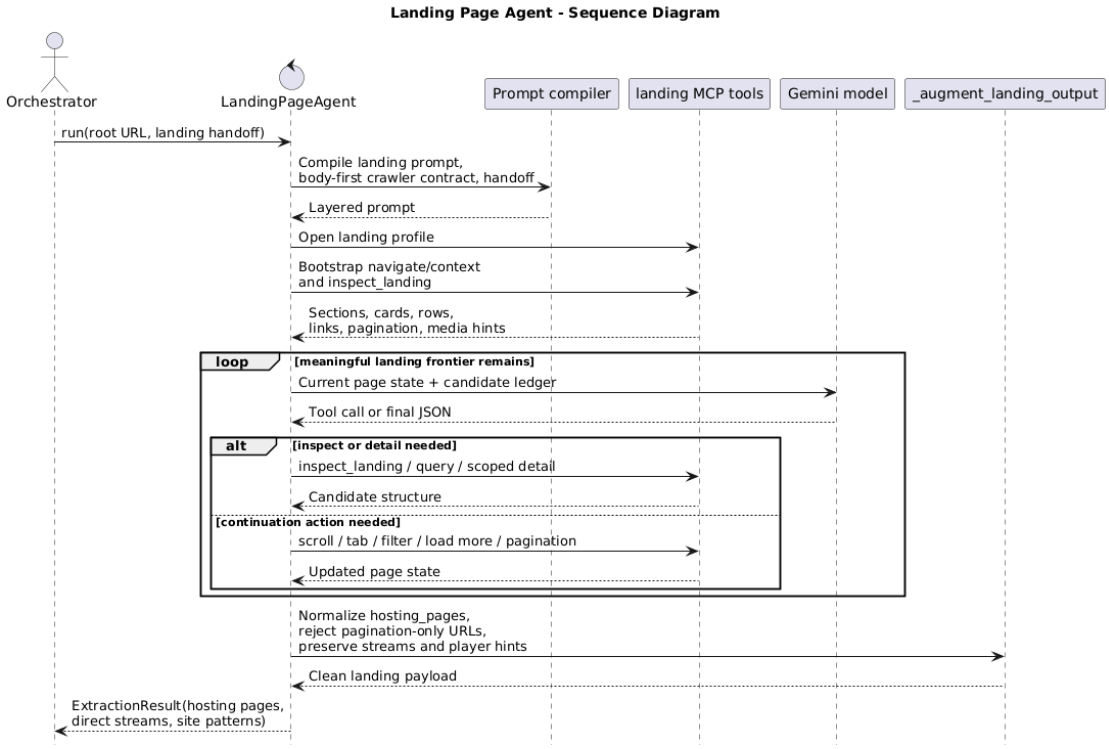


Figure 4.13: Landing-agent sequence: convert listing-page evidence into downstream queues.

4.6.4 Hosting-Page Agent

The hosting-page agent owns watch pages and server/source selection pages. It activates players, operates server controls, handles overlays, captures screenshots, and returns streams or explicit embedded-player handoffs.

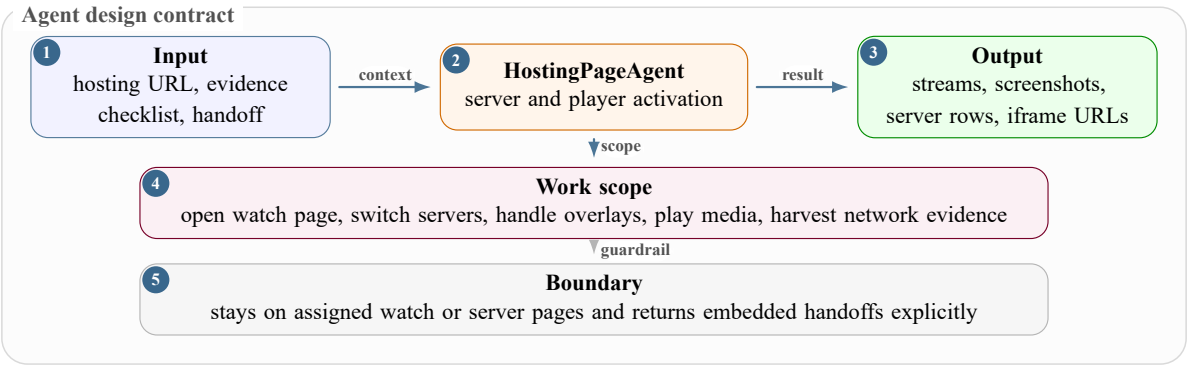


Figure 4.14: Hosting-page-agent contract map.

Hosting returns either stream evidence, an embedded handoff, or diagnostics. The sequence view separates server selection, player activation, evidence capture, and the final stream-or-handoff result.

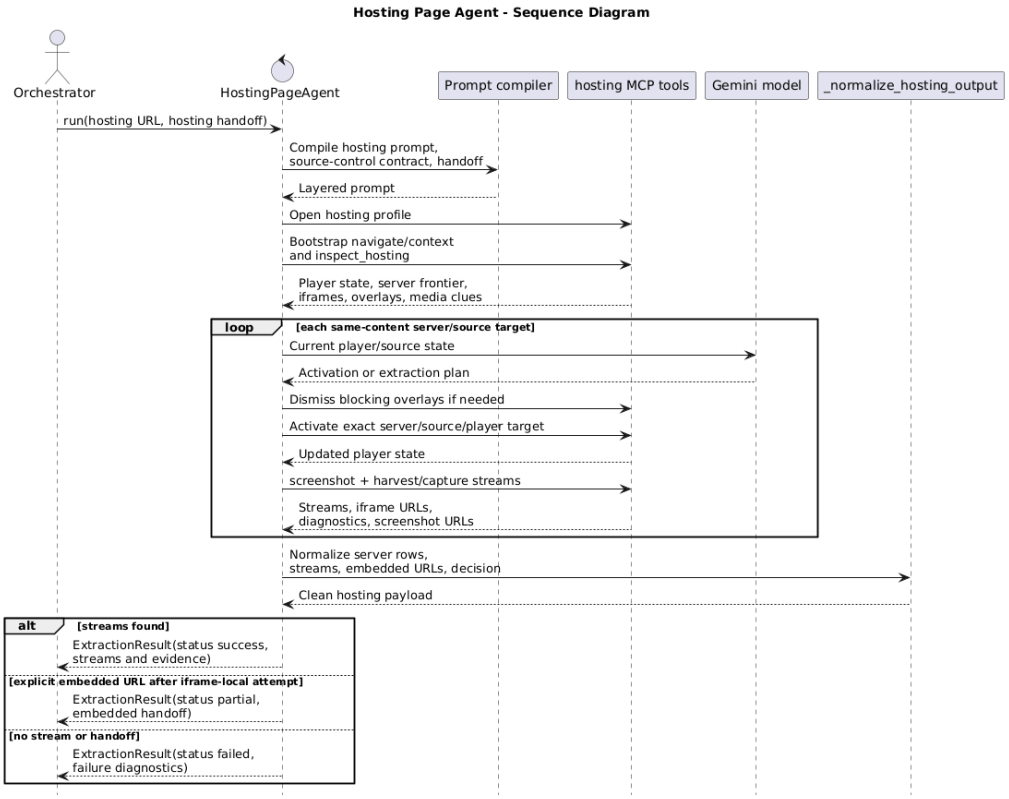


Figure 4.15: Hosting-agent sequence: activate watch pages and return stream or iframe evidence.

4.6.5 Embedded-Page Agent

The embedded-page agent owns direct player or iframe URLs. It stays inside the assigned player context, inspects frames, activates iframe-local controls, recovers from popup or redirect drift, and collects final media evidence.

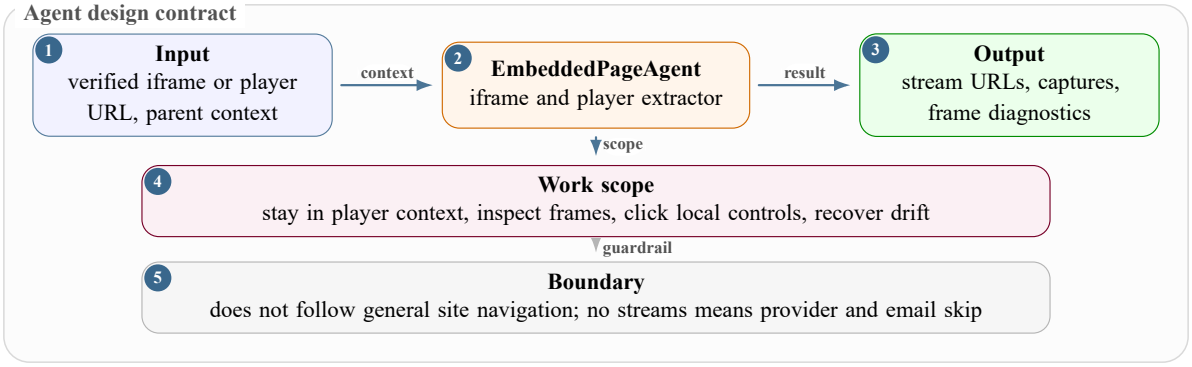


Figure 4.16: Embedded-page-agent contract map.

The component card gives the embedded agent's scope. The sequence diagram then shows the narrower runtime path: stay inside the assigned player context, inspect frames, activate playback, capture evidence, and return either final media evidence or a blocker reason.

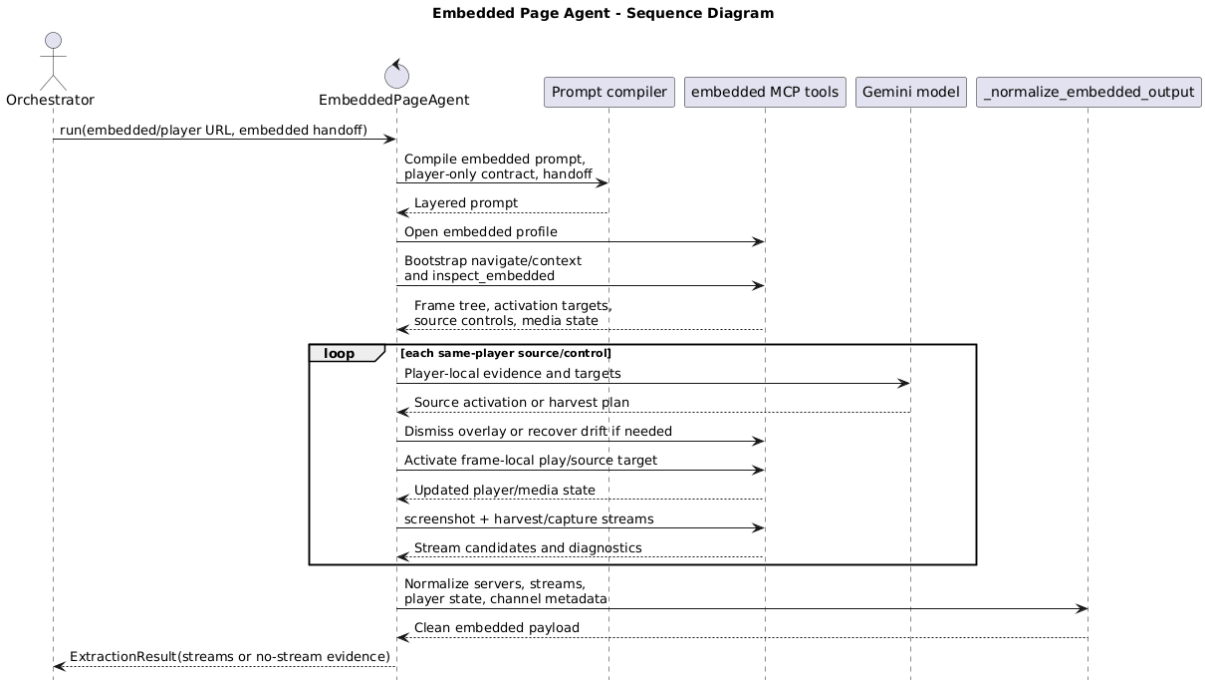


Figure 4.17: Embedded-agent sequence: stay in the player context and harvest final media evidence.

4.6.6 Channel Detection Across Agents

Channel detection is an agent responsibility, not a standalone browser tool. The tools expose page text, DOM structure, frame state, screenshots, server labels, and media requests. The agents decide whether those signals are strong enough to become evidence.

This distinction matters because a channel name printed on a landing page is only a hint. A channel label near an activated player or inside the embedded player context is stronger evidence.

The responsibility is shared across the route:

- classification preserves obvious channel text from the first page observation;
- landing keeps channel and match hints attached to candidate hosting URLs;
- hosting checks whether the selected server or player still matches the same channel;
- embedded confirms, refines, or rejects the signal from iframe-local text, player overlays, and screenshots.

The orchestrator then normalizes these observations as candidates, final channel fields, and confidence notes in the pipeline result.

Agent stage	Channel evidence it can read	How the signal is treated
Classification	Page title, visible headings, URL text, and broad page context.	Preserved as an initial hint, not accepted as final proof.
Landing	Match cards, league labels, event rows, revealed tabs, and candidate hosting links.	Attached to each downstream candidate so the route keeps its sports context.
Hosting	Server labels, player container text, active source state, screenshots, and network-adjacent player metadata.	Used to verify or override landing hints after the watch page is activated.
Embedded	Iframe-local DOM, player overlays, frame title, screenshot text, and final media evidence context.	Treated as the strongest page-level confirmation before provider analysis.

Table 4.4: Channel detection responsibilities across the specialist agents.

4.6.7 Provider Analysis and Email Generation

Provider analysis and email generation are downstream evidence-packaging stages. They are not page-navigation specialists. Provider analysis reads the collected stream URLs, resolves host or IP ownership, and returns provider information and abuse contact data. Email generation groups provider rows and stream evidence into reviewable takedown drafts. These drafts are not sent automatically; they are artifacts for human review.

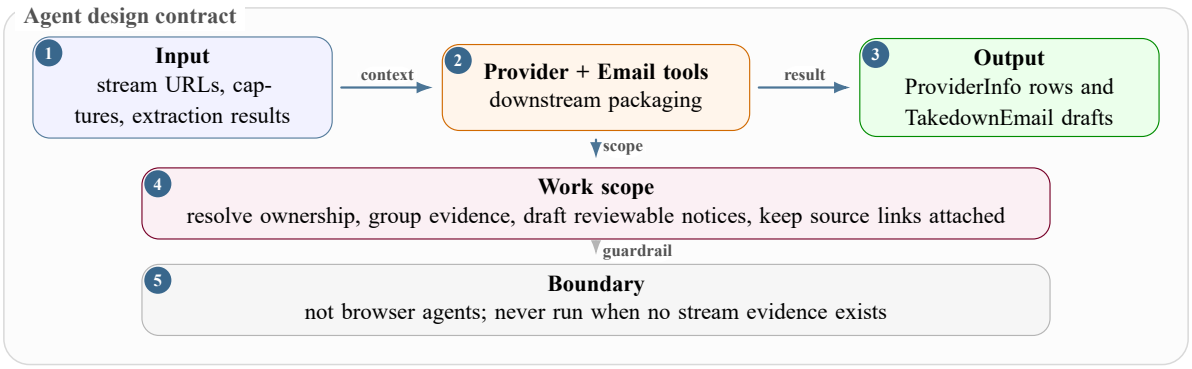


Figure 4.18: Provider-analysis and email-generation contract map.

The two downstream tool sequences make this packaging boundary explicit. Provider analysis first deduplicates stream hosts and resolves ownership evidence. Email generation then groups provider rows, stream evidence, screenshots, and channel context into review-only drafts.

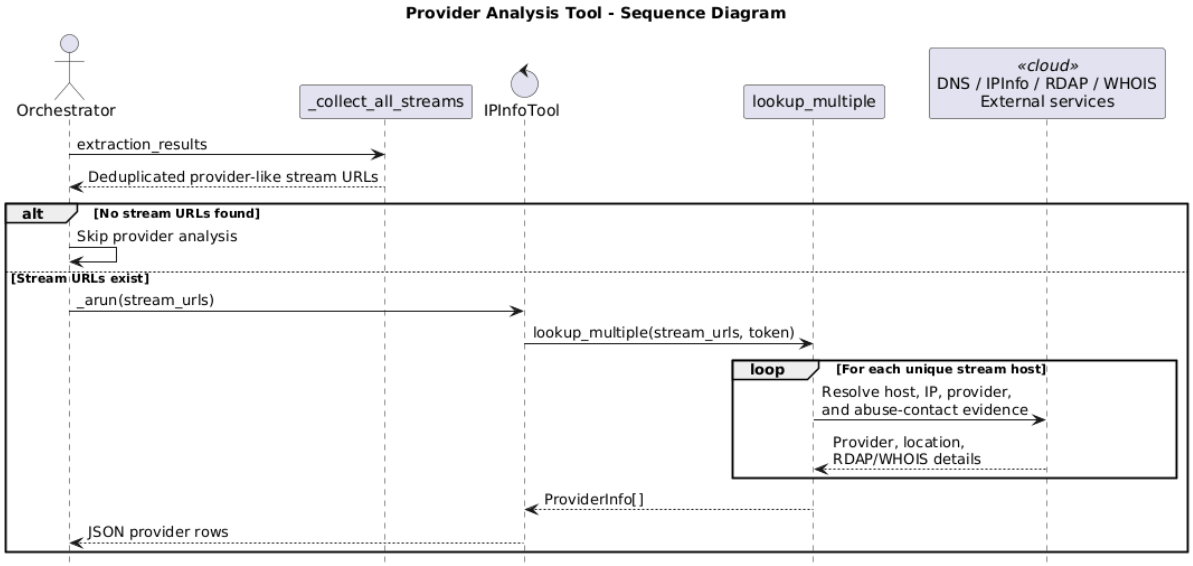


Figure 4.19: Provider-analysis tool sequence: collect stream hosts, resolve provider details, and return reviewable provider rows.

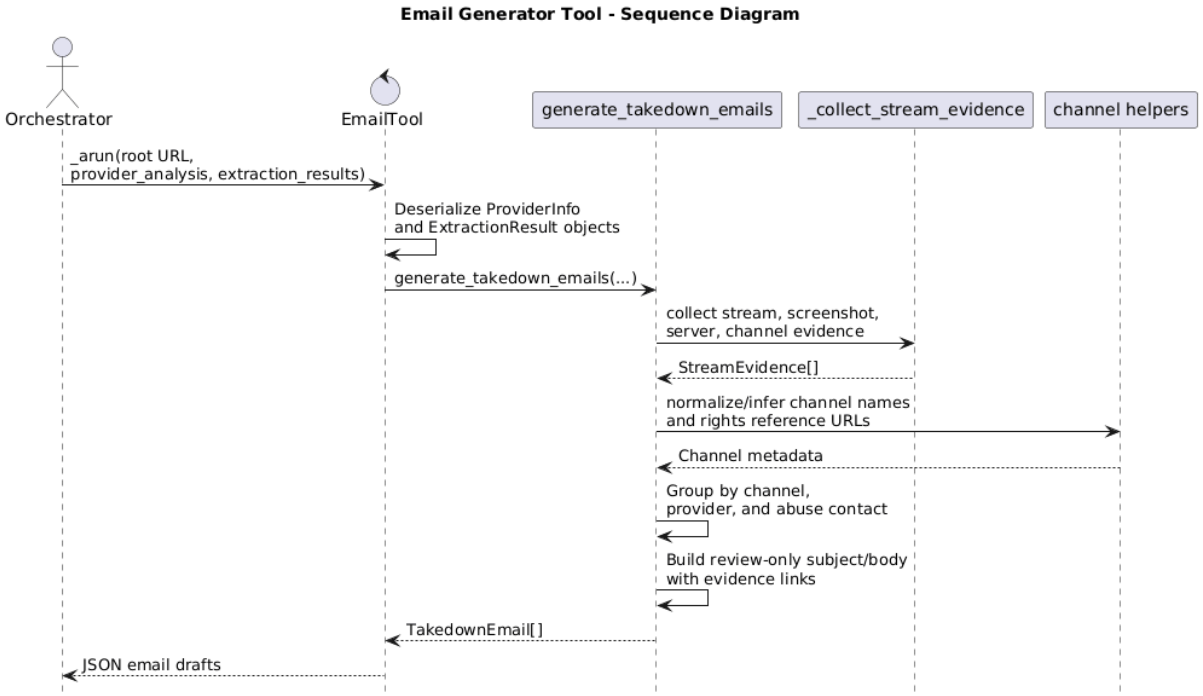


Figure 4.20: Email-generator tool sequence: group provider and stream evidence into review-only takedown drafts.

4.7 Browser Tooling and Runtime Controls

After the agent responsibilities are clear, the browser-tool layer explains how the agents observe and change real web pages. OWC does not ask an agent to reason from raw HTML alone. Illegal streaming pages are often:

- rendered by JavaScript;
- split across nested frames;
- guarded by overlays, popups, and server tabs;
- dependent on tokenized URLs or delayed media requests.

For that reason, every browser-facing specialist receives a profile-scoped MCP tool surface backed by Puppeteer and Chrome.

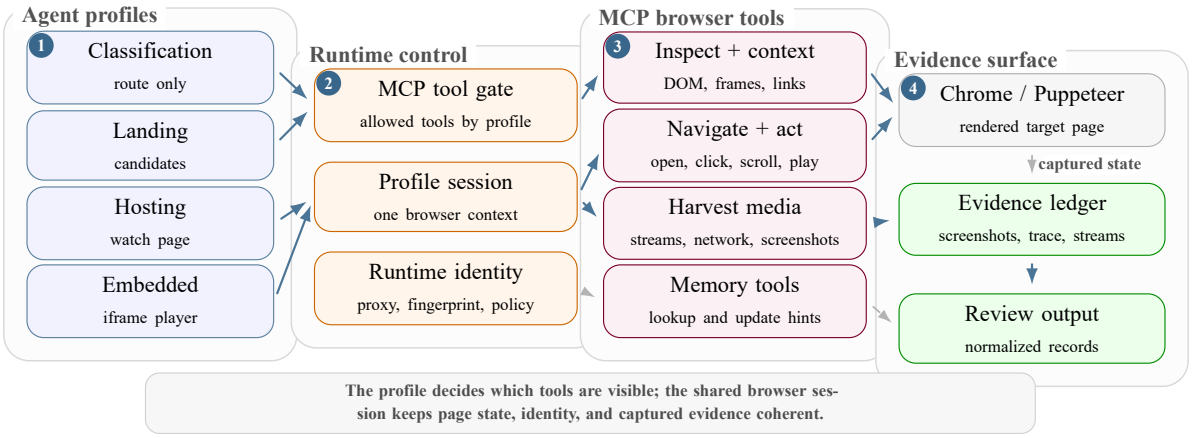


Figure 4.21: Profile-scoped MCP browser tooling and runtime controls used by OWC agents.

4.7.1 Tool Families

The Puppeteer tool layer is divided into tool families so that each agent can ask for the smallest useful observation or action. This keeps context growth under control: the model receives a compact view of the rendered page, then requests a narrower element, frame, media, or screenshot operation when it needs proof.

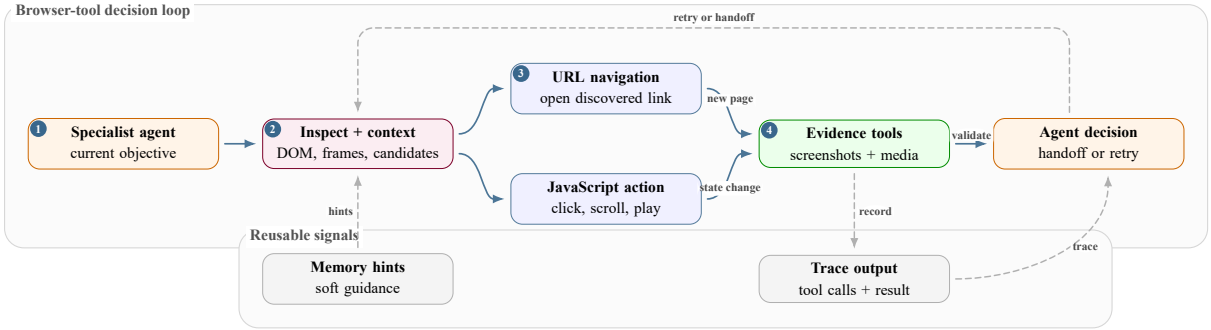


Figure 4.22: Browser-tool families used to move from rendered context to interaction, evidence, and memory hints.

4.7.1.1 Inspection and context retrieval

Inspection tools are the first read layer. They inspect the rendered Document Object Model (DOM), not only the first server response. Broad inspectors summarize visible text, grouped anchors, buttons, candidate cards, server controls, iframe trees, player containers, media clues, and screenshot links.

Page-type inspectors make that broad idea more precise:

- the landing inspector looks for match cards and hosting candidates;
- the hosting inspector looks for server/source controls and player state;
- the embedded inspector focuses on iframe-local player evidence.

Context tools are narrower. When an agent sees a possible target in a broad observation, it can request element details, a selector-specific subtree, a frame tree, candidate element references,

or media state. This avoids repeatedly sending a large DOM summary back to the model. The architectural rule is simple: broad context is used to choose where to look, and scoped context is used before acting.

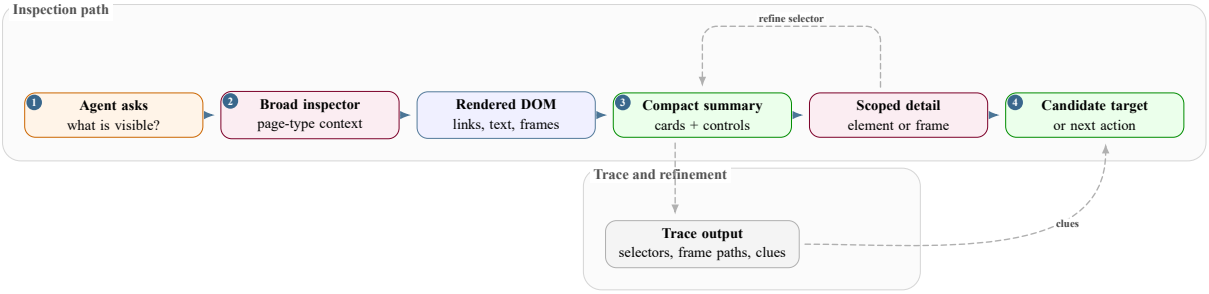


Figure 4.23: Inspection and context-retrieval flow from broad page observation to scoped targets.

4.7.1.2 Navigation and interaction

Navigation has two different meanings in OWC:

- **URL navigation**: open a real link or candidate URL, wait for the document to settle, then inspect the new page.
- **In-page interaction**: stay in the same browser context and click tabs, server buttons, scroll triggers, inputs, selectors, or playback controls.

Both paths must be followed by a new observation. A new document or a changed DOM state can invalidate the previous context.

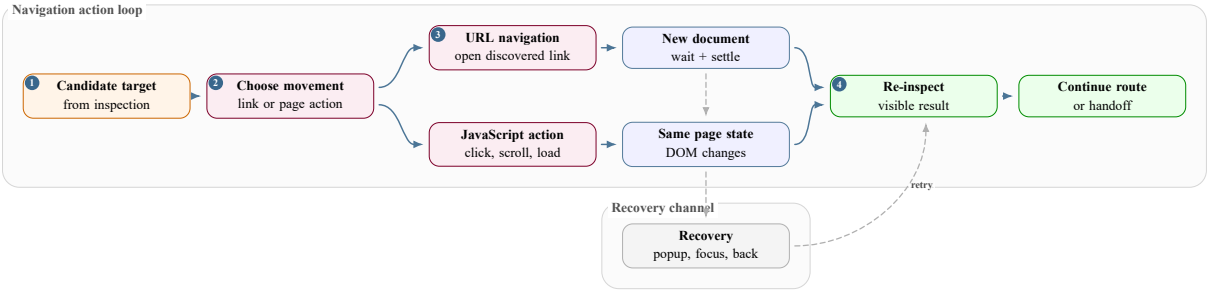


Figure 4.24: Navigation flow split between direct URL navigation and JavaScript-driven page interaction.

4.7.1.3 Evidence and media extraction

Evidence tools convert browser state into reviewable artifacts:

- screenshots show what the page looked like after navigation or interaction;
- media-state tools confirm player presence, playback state, and active frame changes;
- stream-harvesting tools collect `m3u8`, `mpd`, `mp4`, or related candidates from network and browser signals.

The agent still decides whether the candidate is acceptable evidence, but the tools provide the structured facts that make the decision auditable.

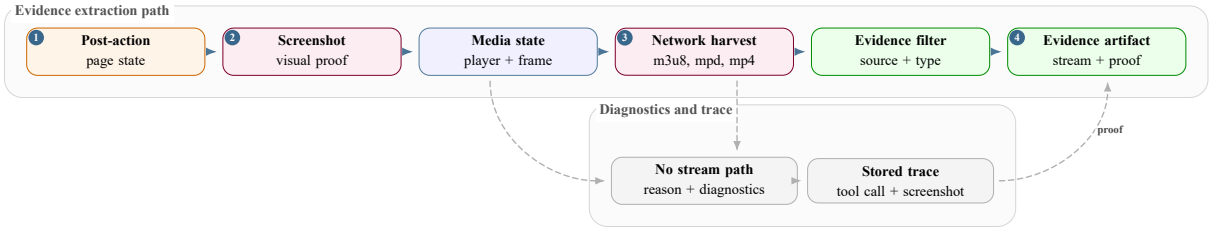


Figure 4.25: Evidence and media-extraction flow from post-action page state to reviewable artifacts.

4.7.1.4 Memory and site hints

Memory tools read and update reusable site hints such as known server tabs, useful selectors, common iframe patterns, and previous failure causes. They do not replace live browser evidence. A memory hint can tell an agent where to start, but the agent must still inspect the current page, act through Puppeteer if needed, and validate the result with screenshots, player state, or stream evidence.

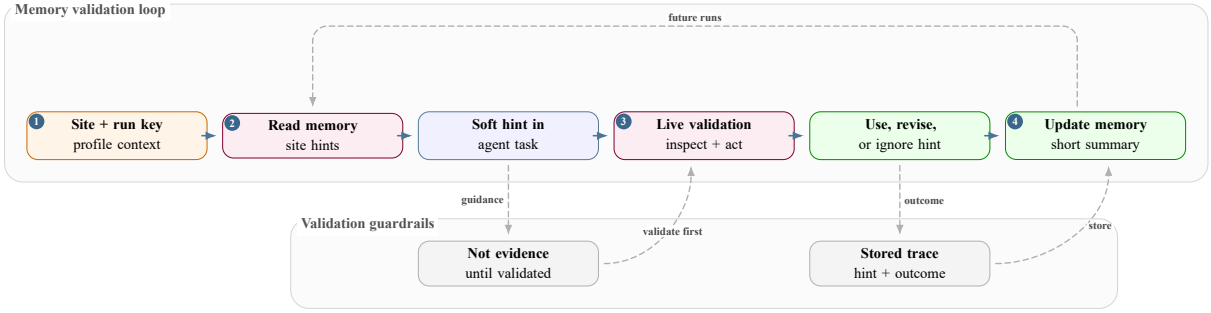


Figure 4.26: Memory-tool flow showing hints as guidance that must be validated live.

4.7.2 Fingerprint Handling and Proxy Rotation

OWC treats browser identity as a runtime setting rather than as prompt text. Browser profiles keep cookies, cache, and session state scoped to the current run or agent profile. Fingerprint handling controls properties such as browser identity, navigator surface, viewport-related behavior, and profile fallback so that repeated browser sessions do not expose an obviously inconsistent automation profile.

Proxy rotation is handled at runtime as a validated sticky strategy:

- candidate proxies are collected from configured remote or custom sources;
- each candidate is validated against a test endpoint;
- the selected proxy stays with the current browser profile while healthy;
- rotation happens after failure, not after every click.

Changing network identity mid-run can break cookies, session-bound player URLs, and tokenized media links. If no proxy candidate works, the runtime can fall back to direct browsing instead of incorrectly reporting the target site as unreachable.

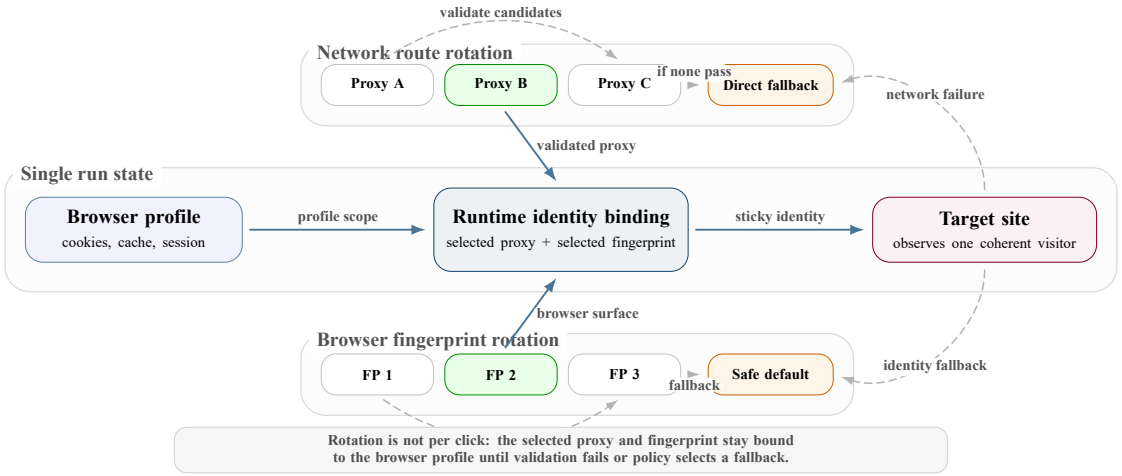


Figure 4.27: Proxy and fingerprint rotation as one coherent runtime identity.

Runtime concern	Architectural rule	Reviewable output
Tool scope	Each specialist receives only the tools needed for its page role.	Tool calls and profile identifiers attached to the run trace.
Browser profile	State is scoped by browser/profile so cookies, cache, and session-bound pages stay coherent during a run.	Profile, runtime settings, and browser events.
Fingerprint handling	Browser identity is hardened and rotated according to runtime policy, not improvised by the agent.	Runtime configuration and browser diagnostics.
Proxy rotation	Proxies are validated and kept sticky until failure; direct fallback avoids false site-down conclusions.	Proxy strategy, failures, and final navigation status.
Context retrieval	Agents request broad DOM summaries first, then scoped element, frame, or media details before acting.	Inspector output, selectors, frame paths, and observation events.

Table 4.5: Browser runtime controls that support reliable and reviewable evidence collection.

4.8 Data, Trace, and Reviewability View

The reviewability view is stored as normalized database records, not only as one large final JSON object. The central class is `PipelineRunRecord`. It stores:

- stable run identity and root URL;
- final status and high-level classification fields;
- aggregate evidence counts;
- token totals and cost totals.

Background jobs can exist before a pipeline row is finalized. Once the run is persisted, agent, telemetry, prompt, memory, evidence, provider, screenshot, and email records attach back to

pipeline_runs. Table 4.6 summarizes those records as review families rather than as one monolithic JSON blob.

Database table family	Representative tables	Architectural reason
Run identity and job state	background_jobs, pipeline_runs, run_snapshots, legacy runs.	Keeps the durable run handle, root URL, status, counts, duration, token totals, cost totals, and fallback snapshot.
Agent timeline	agent_runs, agent_outputs, runtime_events.	Shows the actor, target URL, model/tool turns, status, and event sequence behind the final decision.
Model and prompt telemetry	llm_calls, run_model_usage, prompt_versions, prompt_compilations, pricing_configs.	Keeps provider, model, prompt version, cache mode, token split, context-window use, and estimated cost visible.
Browser and tool evidence	tool_calls, tool_playground_calls, run_screenshots.	Connects browser actions to arguments, status, errors, timing, screenshots, target URLs, and event sequence.
Extracted evidence and review output	run_streams, provider_analyses, provider_lookup_checks, takedown_emails.	Separates streams from provider enrichment and draft notices so emails trace back to stream URLs, screenshots, server labels, and contacts.
Human review, evaluation, and memory	run_decisions, run_tasks, dataset_sites, dataset_batches, dataset_site_runs, memory_entries, memory_hints_used.	Supports manual follow-up, batch evaluation, and reusable hints without replacing live browser evidence.

Table 4.6: Database table families used to turn an agent run into reviewable records.

This is the database shape behind the run-detail console. The dashboard can show the final answer and the failure path because agent runs, model calls, browser tool calls, runtime events, prompt compilations, memory hints, streams, screenshots, providers, and email drafts are queryable as separate rows.

PipelineResult is the aggregate runtime object returned by the orchestrator. It groups classification, matches, extraction results, streams, screenshots, provider analysis, emails, and final status. The surrounding PipelineRun and RunMetrics records carry execution state, model usage, tool usage, and cost totals. In the Python runtime this object graph is convenient,

but the report also needs to show why the final result is split into database tables that answer different review questions.

The table structure is intentionally normalized because different reviewers ask different questions:

- "What stream was found?" points to `run_streams`.
- Hosting-stop questions point to `agent_runs`, `runtime_events`, and `tool_calls`.
- "What did this run cost?" points to `run_model_usage` and `llm_calls`.

This separation lets the dashboard show product evidence and operational traceability without treating one JSON blob as the only source of truth.

The contract between agents is intentionally simple. The model can inspect the page, use tools, and reason through a difficult browser state, but the value that crosses a stage boundary must be structured. This prevents the next agent from depending on vague prose such as "the second server looked good".

Instead, the orchestrator passes a bounded `HandoffContext` containing:

- the target URL, page role, and route source;
- known match or channel context;
- navigation policy and required evidence checklist;
- memory hints that still need browser verification.

The following summary is a chain of contracts, not a chat between agents:

- classification produces a small routing object;
- landing turns a listing page into candidate watch targets;
- hosting captures streams or hands an explicit player URL to embedded;
- embedded works inside that player context and returns final media, screenshot, or blocker evidence;
- provider and email tools run only after stream rows exist.

This is why the final `PipelineResult` can point back to the screenshots and stream hosts that justify the review output.

A useful way to read the contract is to ask what the next stage is allowed to trust. It can trust typed fields such as `page_type`, `hosting_pages[]`, `embedded_urls_for_processing[]`, and `all_stream_urls[]`.

It should not trust:

- the previous model's private chain of thought;
- temporary browser impressions;
- a long natural-language summary that cannot be checked later.

This is why the handoff object carries both the evidence already collected and the checklist that still has to be proved.

This contract also makes failed runs easier to review:

- if landing found schedule rows but no valid hosting target, the run stops with that specific reason;
- if hosting found an iframe but no stream URL, embedded receives the iframe as a new bounded task;
- if provider and email steps run, they start from persisted stream and screenshot rows, not from a vague claim that a stream existed.

The same structure is what makes the dashboard understandable to a reviewer. Instead of one large log blob, the run can be read as a sequence: route decision, landing candidates, hosting evidence, embedded-player evidence, provider identification, and draft email. Each step has an input object, an output object, and a reason for either continuing or stopping.

Stage	JSON input it receives	JSON output it must return	Why the boundary matters
Classification	Seed URL, runtime settings, observer, and memory hints.	`ClassificationResult` with `url`, `page_type`, `confidence`, `reasoning`, and `agent_type`.	Converts the first rendered browser state into a route decision instead of letting later agents guess the page role.
Landing	`HandoffContext` with root URL, target URL, upstream page type, classification reasoning, and memory hints.	Landing JSON with hosting pages, direct stream URLs, site patterns, and rejected URLs, normalized into `MatchInfo[]` and `ExtractionResult`.	Preserves match rows, schedule context, hosting links, and rejected candidates before the workflow moves to player pages.
Hosting	`HandoffContext` with assigned watch URL, match/channel hints, server hints, navigation policy, and required evidence checklist.	Hosting JSON with `decision`, server rows, stream URL rows, embedded-player handoff URLs, and channel metadata.	Separates server/player interaction from landing discovery and creates explicit embedded handoffs when the player is inside an iframe.
Embedded	`HandoffContext` with embedded/player URL, source hosting status, navigation policy, and evidence checklist.	Embedded JSON with `page_classification`, `servers[]`, `all_stream_urls[]`, `failed_servers`, `session_summary`, and channel fields.	Keeps iframe/player inspection focused; it can return stream evidence or a clear no-stream/blocker explanation without restarting discovery.
Provider and email	Final stream rows, screenshots, extraction results, and the original infringing URL.	`ProviderInfo[]` and `TakedownEmail[]`, then the final `PipelineResult`.	Turns browser evidence into review material while keeping the email a draft for human validation.

Table 4.7: Agent JSON contract summary used by the orchestrator.

Architectural element	Boundary	Reviewable output
Orchestrator state	Current URL, classification, pending hosting and embedded queues, extraction results, provider rows, emails, and final error state.	Route decisions and final `PipelineResult`.
Handoff context	Root URL, target URL, page type, route source, evidence checklist, navigation policy, and memory hints.	Bounded specialist task context.
Runtime observer	Agent lifecycle, model calls, tool calls, screenshots, status changes, cancellation, and cost counters.	Events, rollups, traces, and metrics.
Persistence layer	Background jobs, run records, agent runs, evidence rows, screenshots, streams, provider rows, and email drafts.	The data behind `GET /ui/runs/{run_id}`.

Table 4.8: Runtime data boundaries and persisted review outputs.

4.9 Conclusion

The architecture can be read as a staged, reviewable evidence pipeline:

- the operator starts and reviews runs through the console;
- FastAPI controls execution, persistence, settings, and run-detail APIs;
- the orchestrator uses LangGraph to route between agents and preserve handoff state [13, 14];
- specialist agents use LangChain/LangGraph loops and profile-scoped MCP browser tools for rendered DOM context, frame inspection, screenshots, media state, and network evidence;
- browser runtime controls handle profile state, fingerprint behavior, proxy selection, popup recovery, iframe recovery, and screenshot capture;
- provider and email tools run only after streams exist;
- storage preserves both the final result and the failure path.

The next chapter maps these architecture boundaries to the concrete implementation.

CHAPTER 5

Implementation

5.1 Introduction

The implementation progressed in three major parts:

- **n8n prototype agents**: fast exploration of the four browser responsibilities: classification, landing, hosting, and embedded-player extraction.
- **Open Web Catcher**: the reference platform built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability.
- **Deployment direction**: the company kept its existing operational workflow and deployed only the n8n landing agent as a focused backup cloud job.

This split matters because the systems did not have the same purpose. The n8n phase tested four page-role agents: classification, landing, hosting, and embedded. OWC was built after those lessons, with clearer engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation. The deployment work then narrowed the cloud target to the part the company wanted to operationalize during the internship: the landing agent.

This chapter turns the architecture into concrete implementation work. It first explains the n8n prototype used to validate the browser-agent idea, then presents the OWC platform across FastAPI, Next.js, MCP browser containers, Puppeteer, specialist agents, prompts, model telemetry, memory, PostgreSQL observability, and the focused deployment path chosen for the company environment.

5.2 Part 1: n8n Workflow Prototype

5.2.1 Purpose of the n8n phase

The n8n implementation acted as the project playground [16]. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

This prototype was built on n8n's AI-agent surface, which is documented as part of n8n's LangChain node family and exposes a Tools Agent pattern for connecting chat models to tool nodes [17, 18]. In other words, the prototype was not a separate AI paradigm from the later OWC architecture. It used a managed LangChain/LangGraph-family agent node inside a visual

workflow: LangChain-style tool binding was handled by n8n's AI nodes, graph-style routing was expressed by the n8n canvas, and OWC later made those same control ideas explicit in code with LangChain tool wrappers and LangGraph state graphs [14].

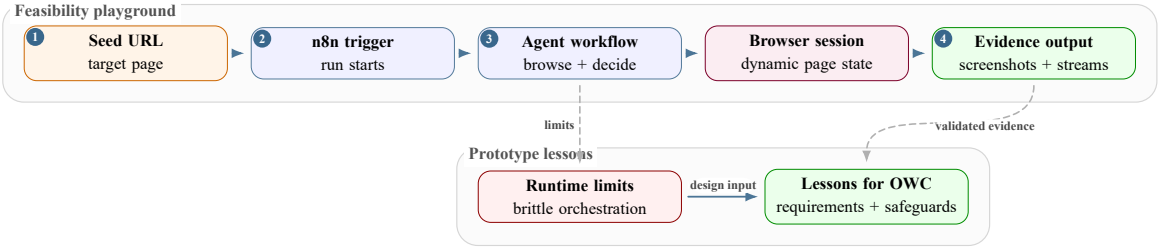


Figure 5.1: Role of the n8n workflow as a feasibility playground for browser-agent evidence collection.

The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

5.2.2 n8n prototype-agent screenshots

The n8n prototype connected the same responsibilities that later became specialist agents: a coordinator selected the next page role, a classification branch labelled the page, a landing branch discovered candidate hosting links, a hosting branch activated server buttons and watched the network, and an embedded branch inspected iframe players. The workflow also cleaned outputs and stored intermediate results so each handoff could be reviewed. Figure 5.2 keeps the full canvas as one landscape view because it shows this prototype handoff order most clearly.

This was useful as a fast implementation test, but it also showed why OWC had to become code-first: browser tools, trace storage, prompt contracts, and evaluation needed stable versioned layers instead of being spread across a visual automation.

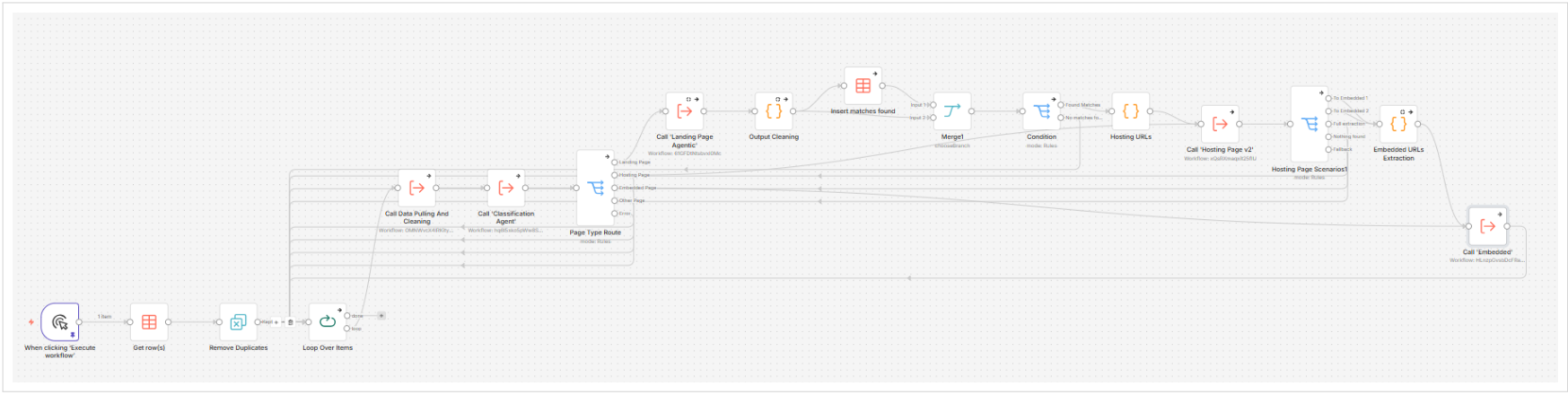
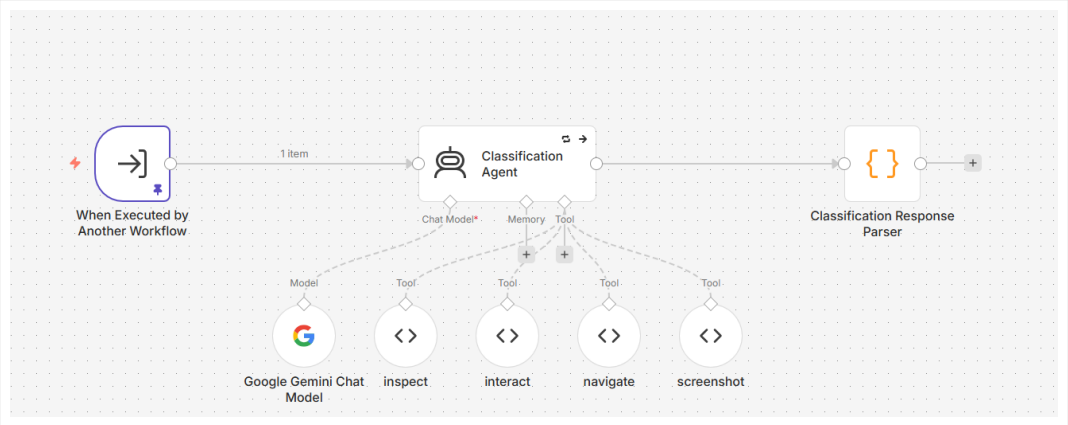


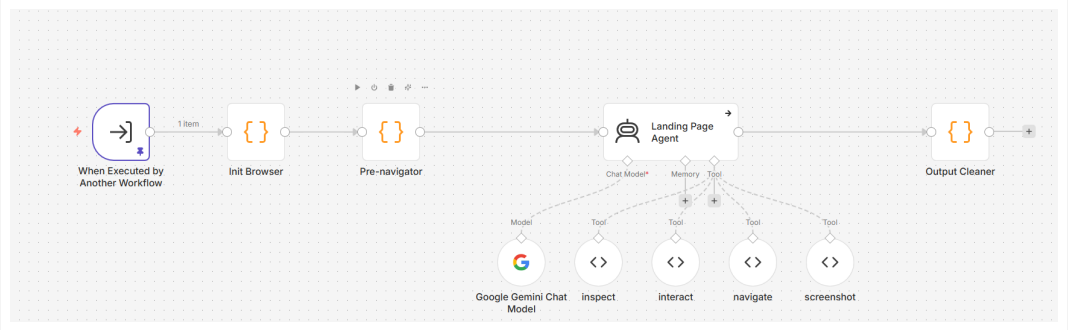
Figure 5.2: Full n8n prototype workflow.

This screenshot is kept as one wide canvas because it records the real n8n workflow shape. Read it from left to right: the first branch classifies the page, the middle steps preserve landing-page candidates, and the right side hands hosting and embedded targets to browser-capable agent calls.

The full workflow view gives the overall sequence, but it is too dense to explain each agent responsibility on its own. The next figure separates the first two prototype responsibilities: classification and landing-page exploration. The landing branch is the one later deployed as the operational backup; the other three remained prototype work that informed OWC.



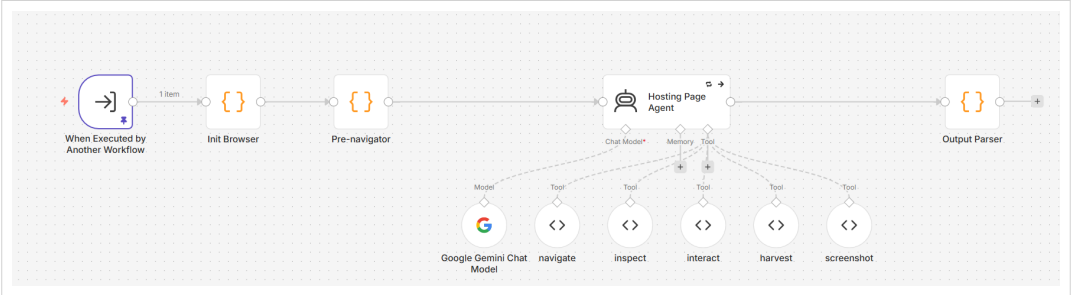
Classification workflow



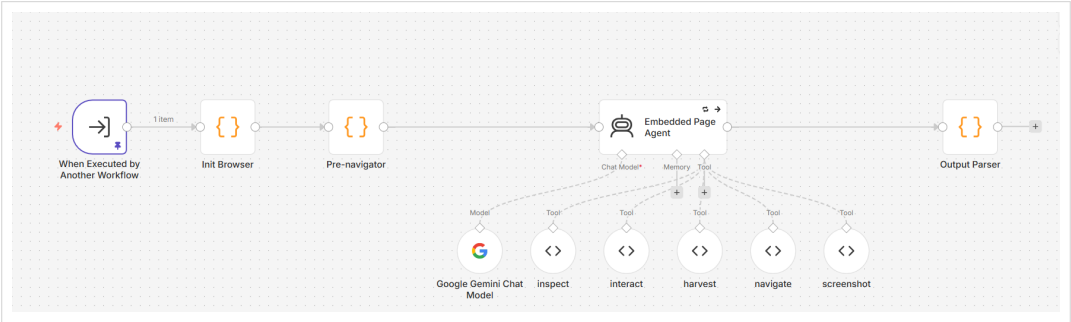
Landing-page workflow

Figure 5.3: n8n classification and landing workflow screenshots.

The prototype then moved from discovery into browser work. The hosting and embedded screenshots cover four checks: player activation, server handling, iframe inspection, and stream harvesting.



Hosting-page workflow



Embedded-player workflow

Figure 5.4: n8n hosting and embedded-player workflow screenshots.

The useful output of these screenshots is easier to read as a role map than as another table. Figure 5.5 shows how the four prototype branches became the implementation contracts used later by OWC.

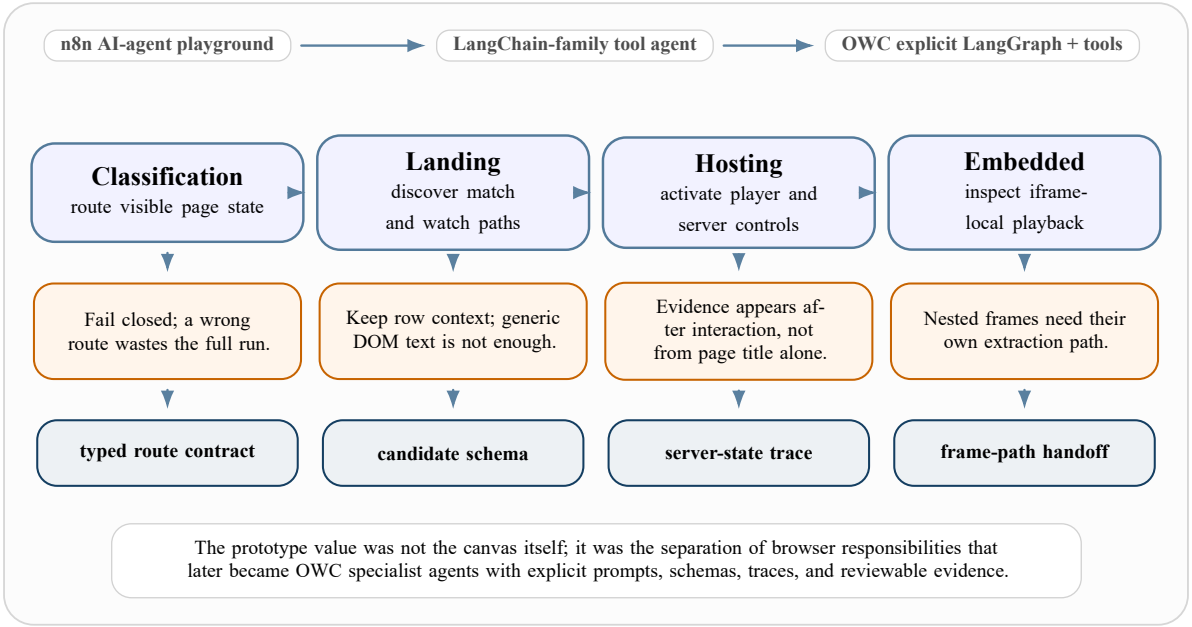


Figure 5.5: n8n prototype-agent role map and the implementation lessons carried into OWC.

5.2.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Single-chain routing	Each candidate moved through explicit handoffs between n8n subworkflows.	Large pages became slow to inspect because every branch waited on browser-tool work before the next handoff could be reviewed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Versioning and comparison were harder than in a normal Python and TypeScript repository.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A `puppeteer.connect` workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.1: n8n prototype limitations and engineering consequences.

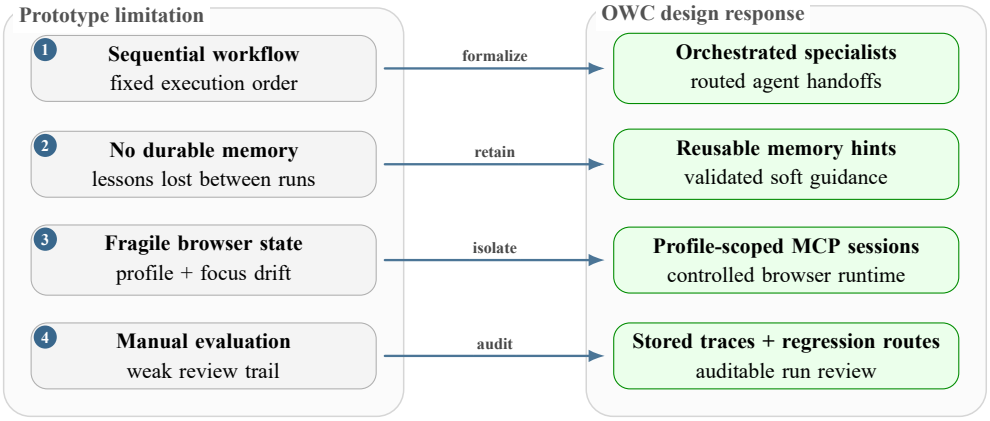


Figure 5.6: How n8n limitations shaped the Open Web Catcher implementation.

5.3 Part 2: Open Web Catcher Platform

OWC keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

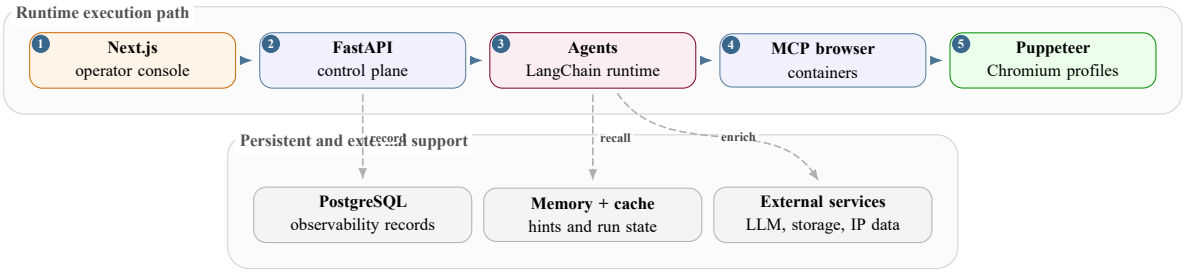


Figure 5.7: Five implementation areas of the Open Web Catcher platform.

5.3.1 FastAPI Backend

The FastAPI backend is the control plane of OWC [11]. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

5.3.2 Next.js Operator Console

Although it presents dashboard views, the Next.js frontend is the OWC operator console for launching runs, following execution, reviewing evidence, and changing runtime configuration [12].

The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found,

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
MCP tool diagnostics	Expose profile-specific tool status and logged tool calls.	Separates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.2: Backend API responsibilities in OWC.

and where the model or browser runtime failed. The screenshots in this section therefore show the implemented review surfaces, not mockups: the dashboard, run detail, browser evidence view, provider enrichment, email draft, tool-call payload inspector, and runtime settings.

Frontend surface	What it shows or controls	Main backend dependency
Dashboard	Overview, cost, token, provider, tool, and agent telemetry.	Dashboard aggregation endpoints and persisted observability tables.
Live Pipeline	Workflow and single-agent launch with runtime preflight.	Workflow launch, health, and browser/MCP readiness endpoints.
View Results	Website dataset, batch execution, per-site status, and run history.	Dataset, batch, and run repositories.
Provider Results	Stream URL enrichment, IP resolution, provider distribution, country distribution, and manual lookup.	Provider-analysis records and lookup services.
Settings	Gemini model assignments, cache/runtime controls, browser settings, display preferences, API key status, notifications, and MCP tool enablement.	Configuration endpoints and runtime settings persistence.

Table 5.3: Current Next.js operator-console surfaces and backend dependencies.

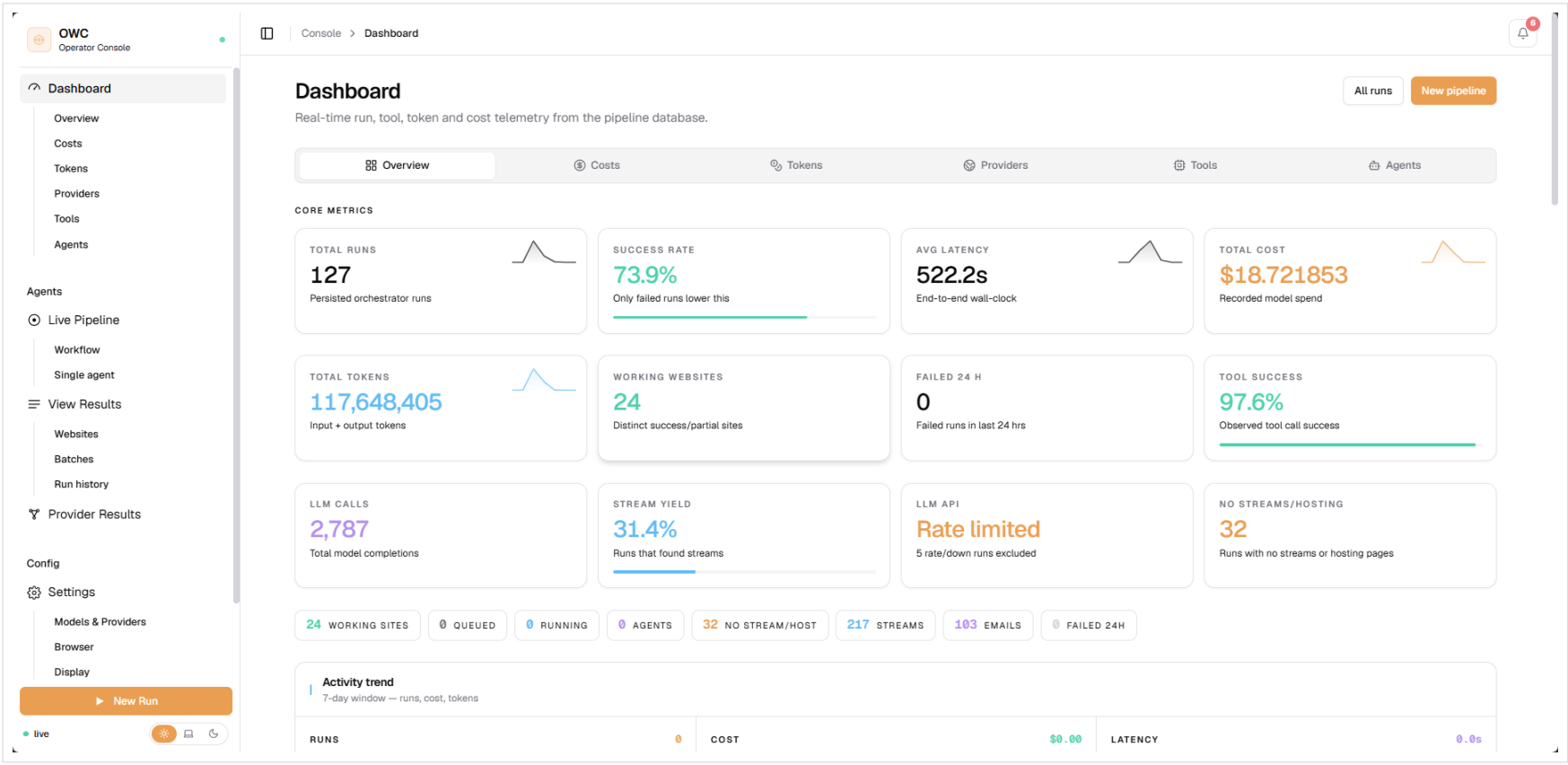


Figure 5.8: OWC dashboard overview telemetry screen.

The dashboard overview makes completed executions inspectable by summarizing run status, success rate, working websites, cost, token, latency, stream yield, tool, and agent telemetry.

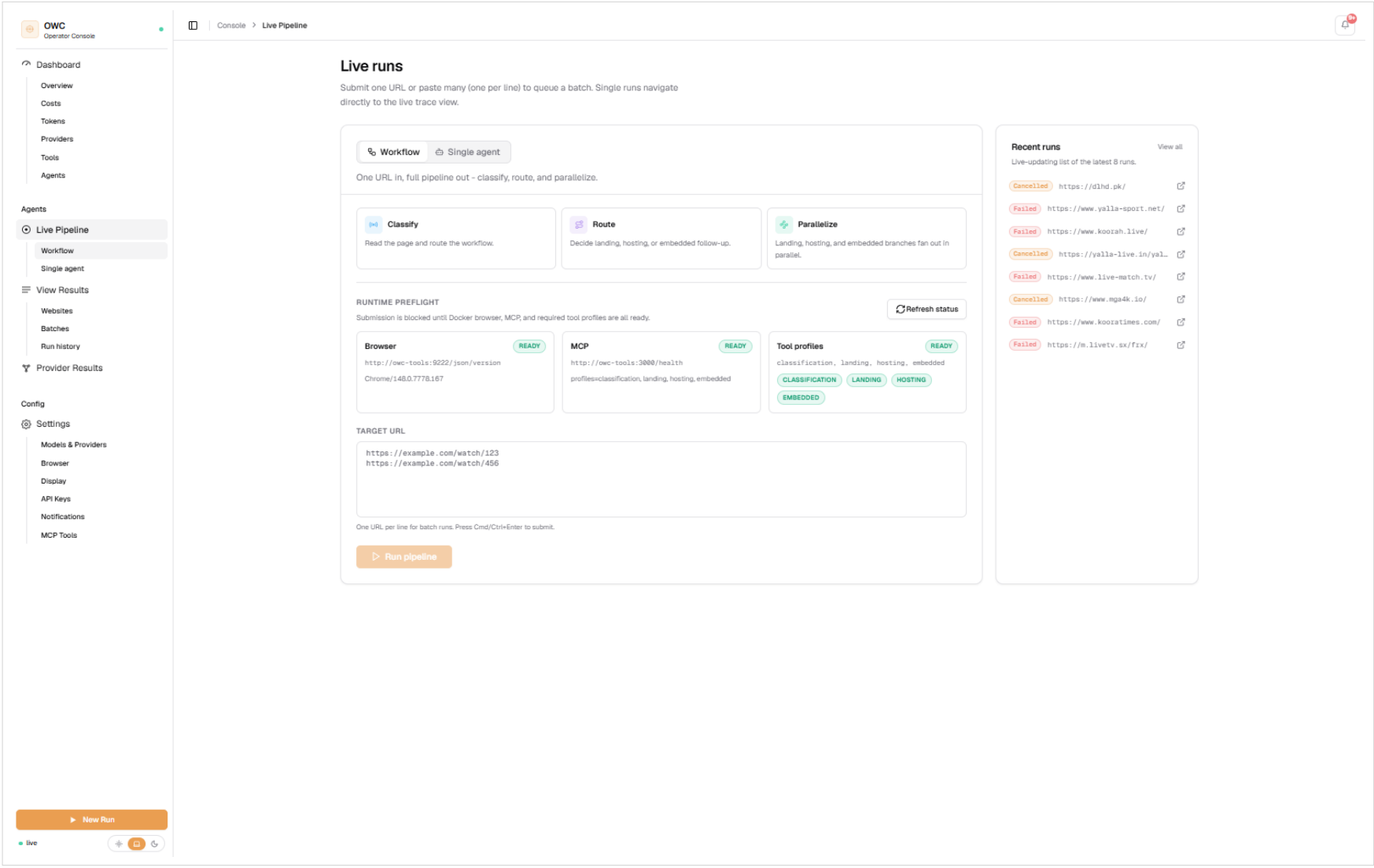


Figure 5.9: OWC live workflow launcher screen.

The live launcher exposes workflow and single-agent starts with runtime preflight checks, so operators can see why a run is ready or blocked before execution.

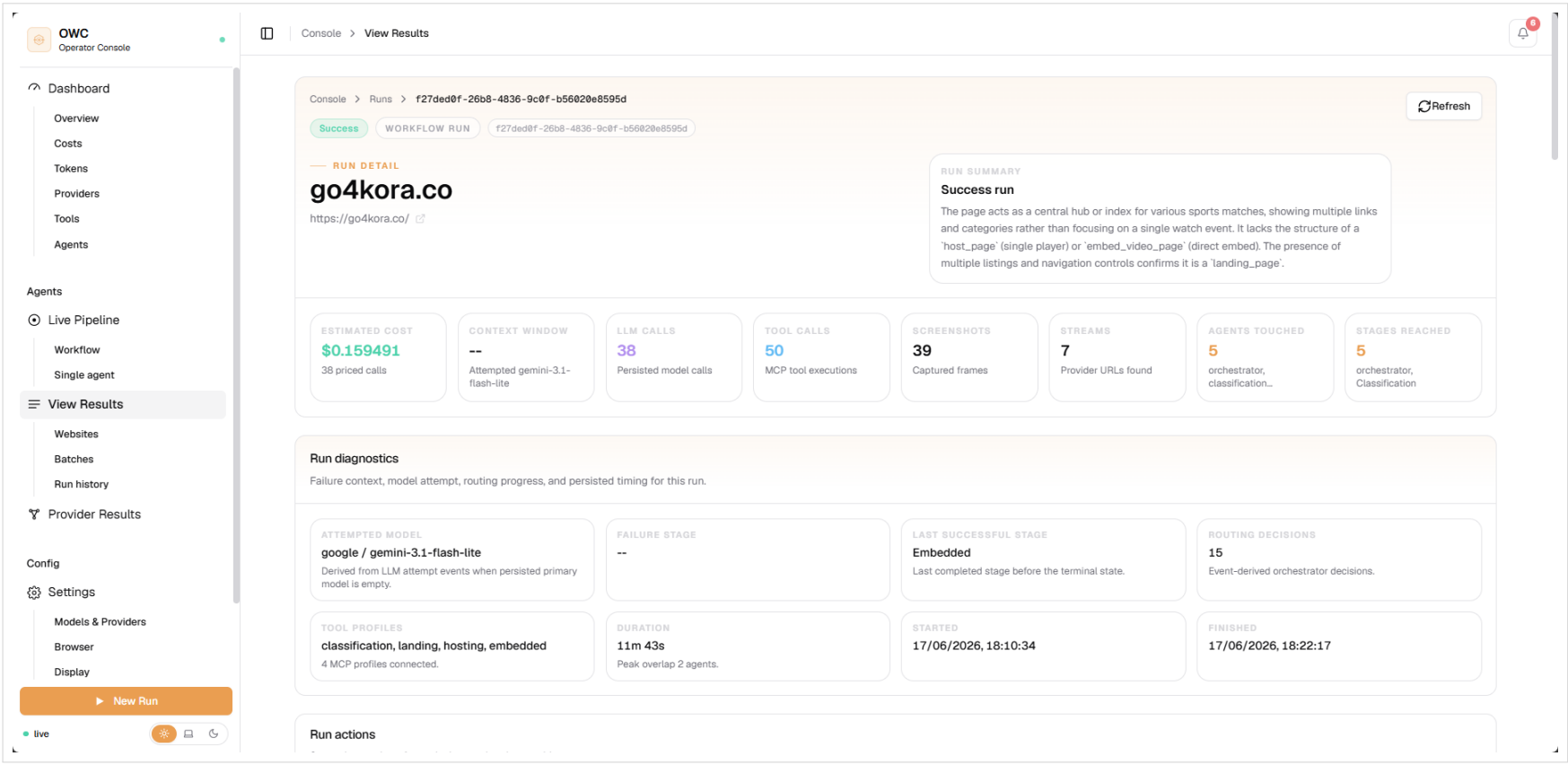


Figure 5.10: OWC run detail review screen.

The run detail page turns one completed execution into a reviewable record: status, root URL, summary, estimated cost, LLM and tool calls, screenshots, streams, agents touched, and stage progression.

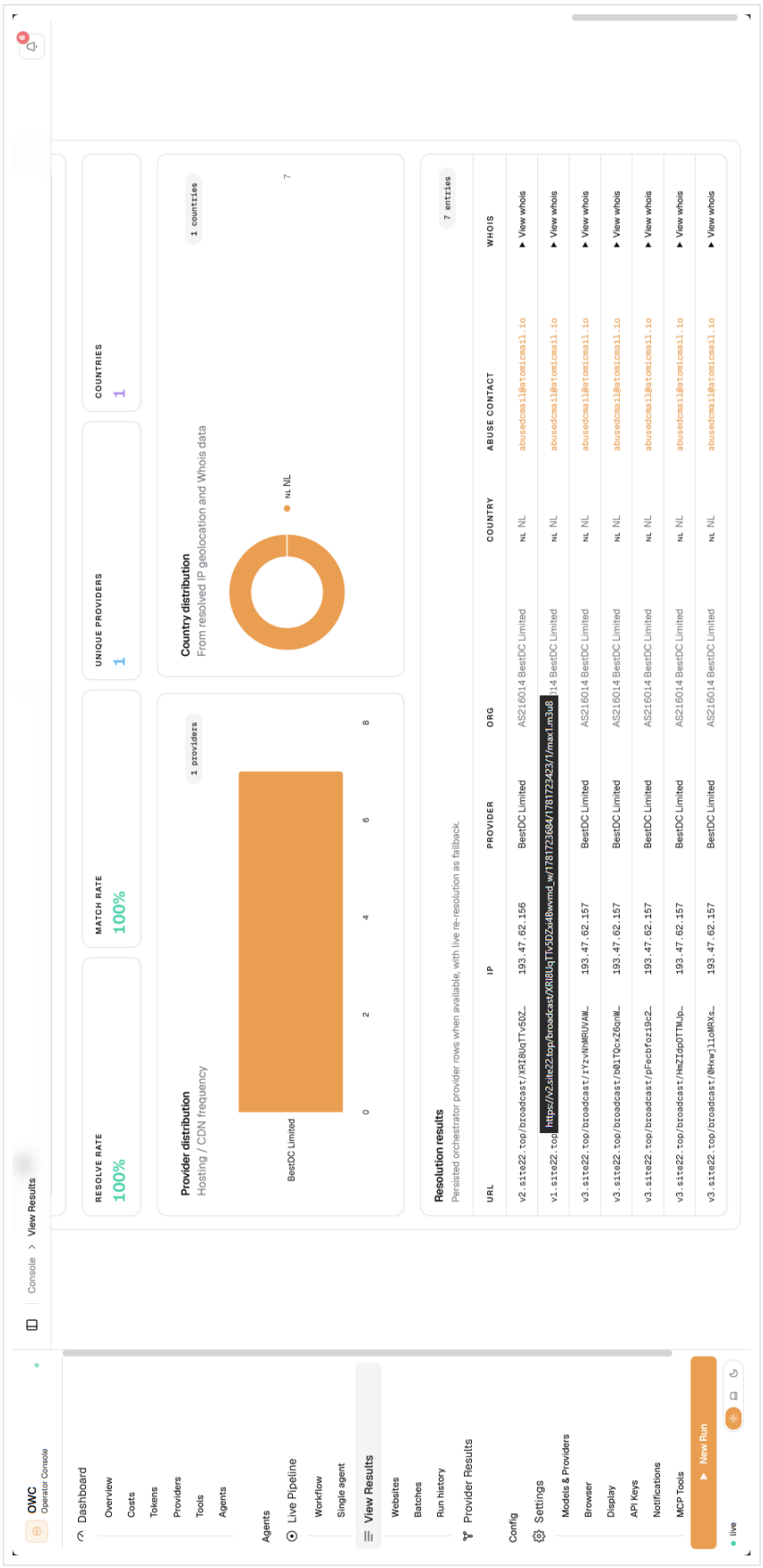


Figure 5.11: OWC provider enrichment review screen.

The provider enrichment view connects recovered stream URLs to provider and country evidence, making the result bundle easier to review after extraction and before a takedown draft is approved.

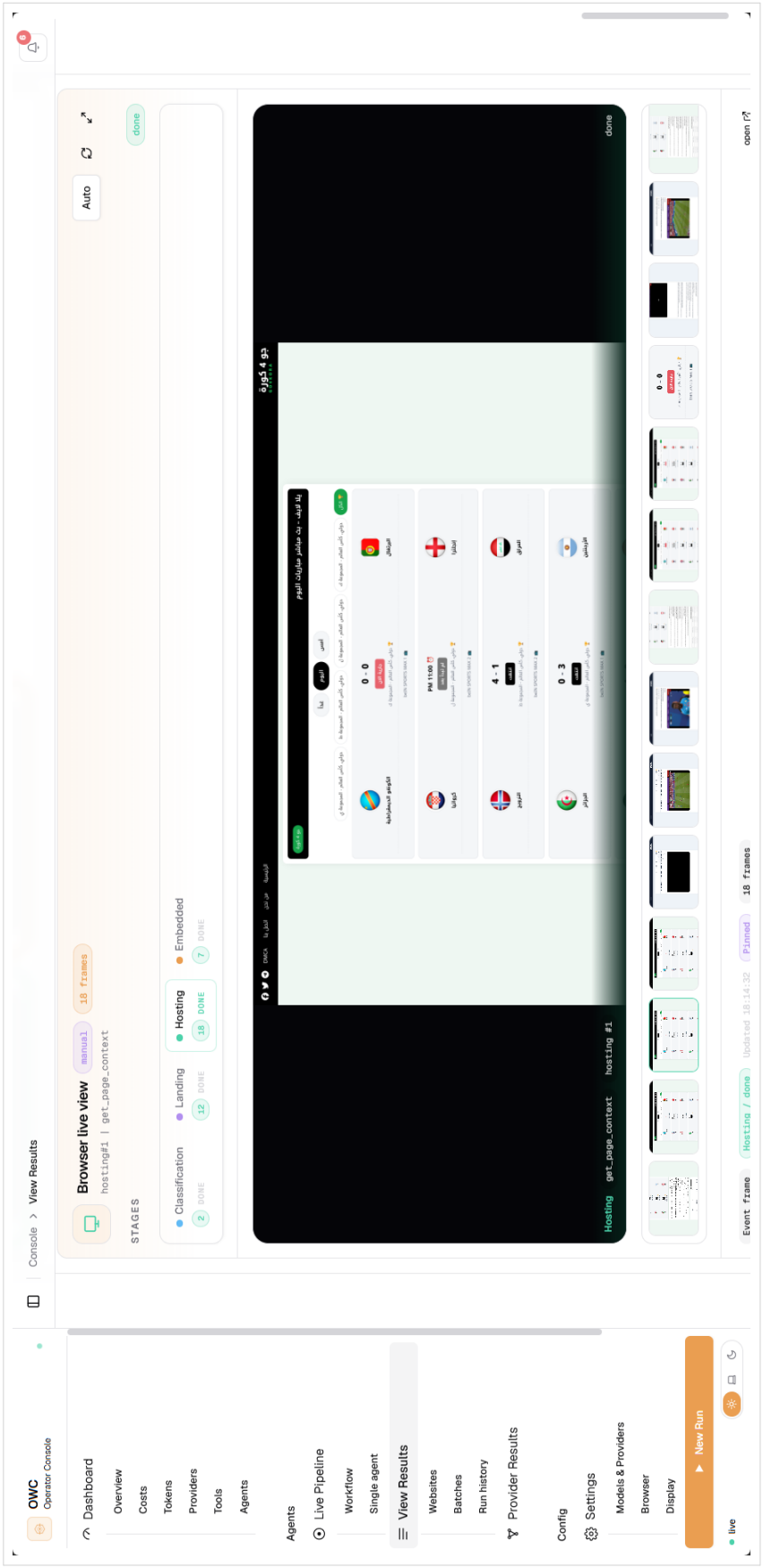


Figure 5.12: OWC browser live-view evidence screen.

The browser live view links agent stage progress to rendered page evidence. It shows the active stage, captured frame thumbnails, and the page state that produced the evidence being reviewed.

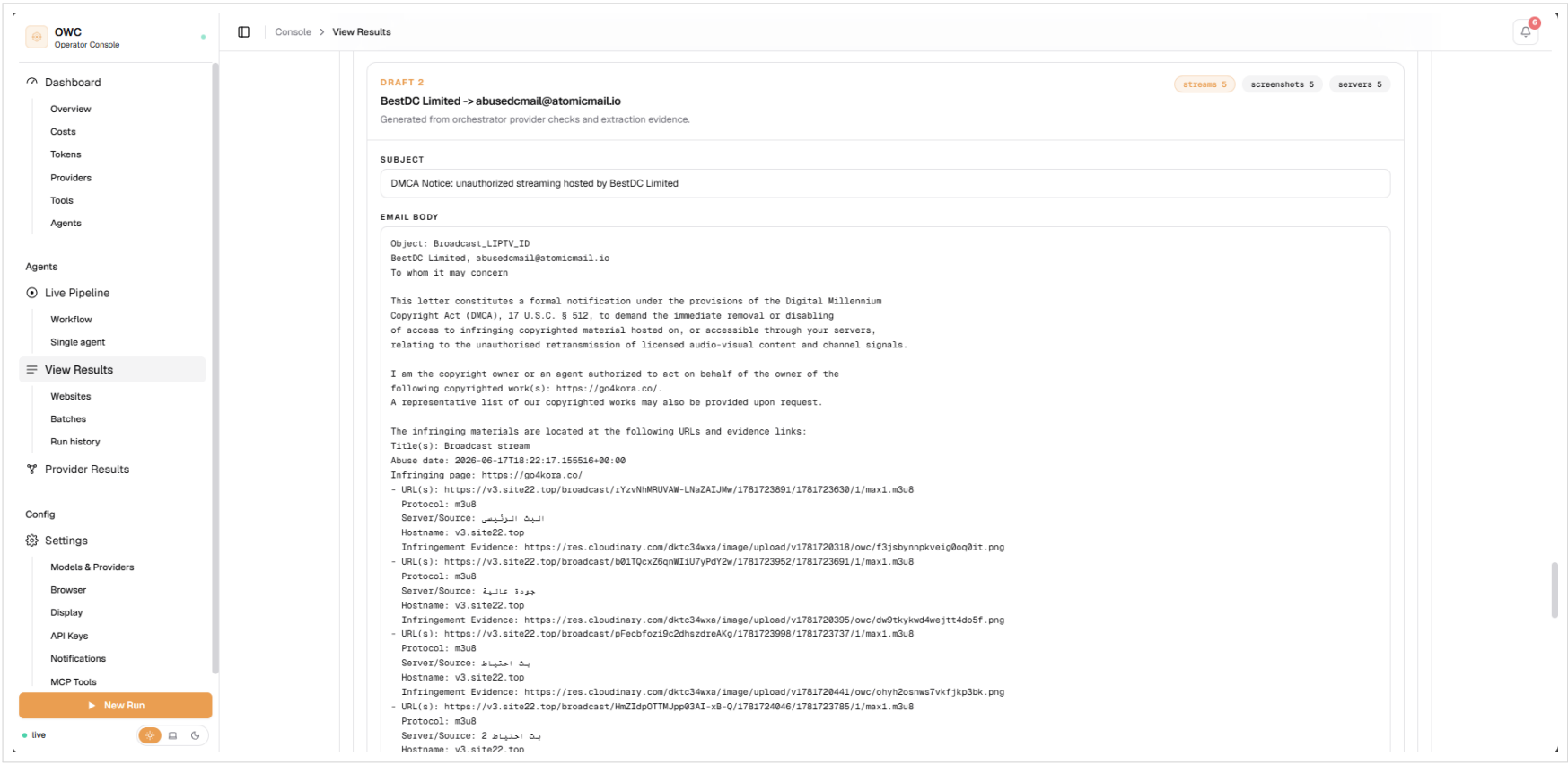


Figure 5.13: OWC generated takedown-email draft screen.

The email draft screen keeps the generated notice reviewable by exposing the provider contact, subject, stream URLs, protocol evidence, server labels, hostnames, and screenshot links before any human action is taken.

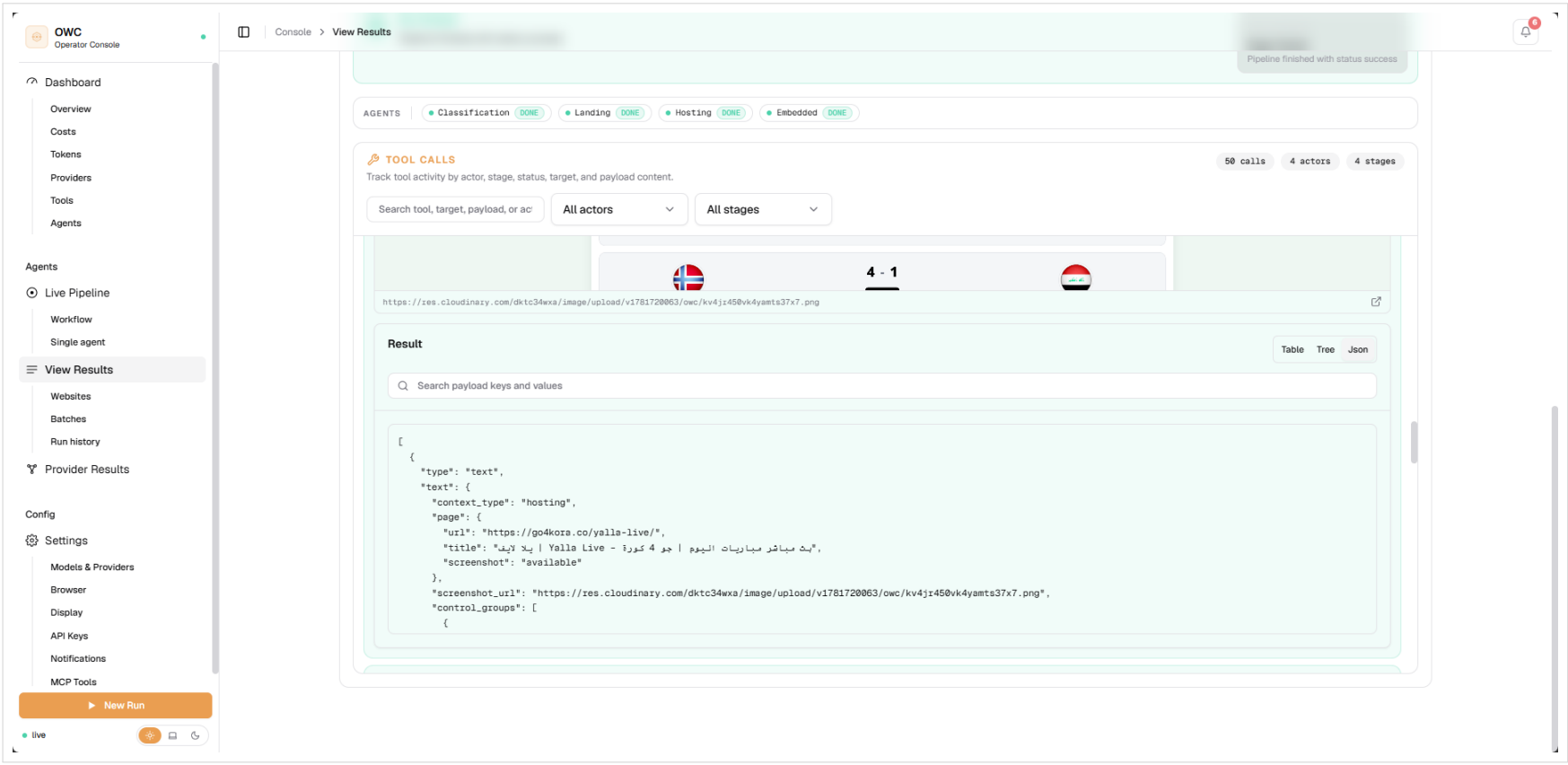


Figure 5.14: OWC tool-call JSON payload inspection screen.

The tool-call payload inspector keeps low-level browser observations auditable. The operator can inspect the structured result returned by a tool call instead of relying only on a final model summary.

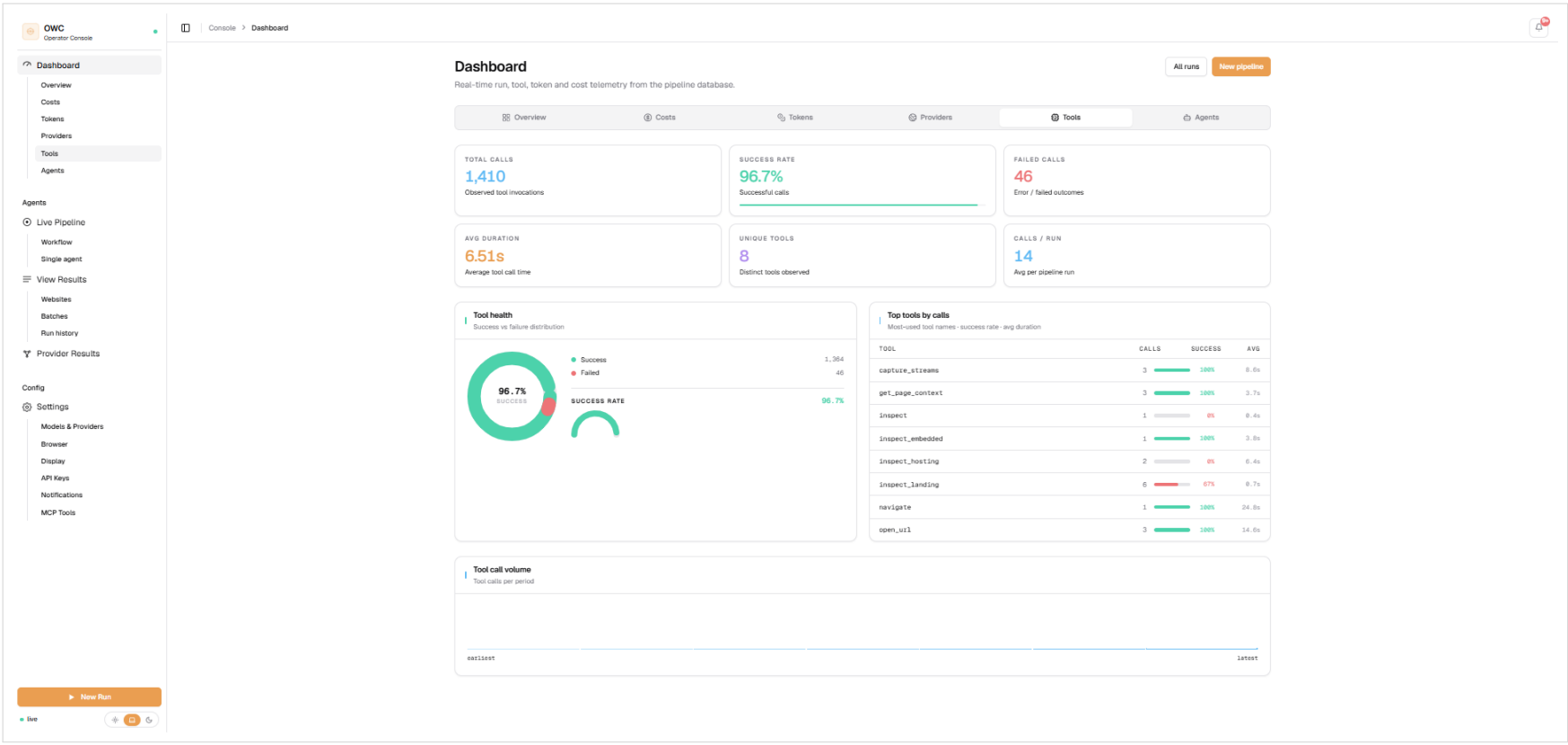


Figure 5.15: OWC tool telemetry screen.

The tool telemetry page separates browser-tool diagnostics from prompt behavior, helping explain failures caused by unavailable MCP tools or profile configuration.

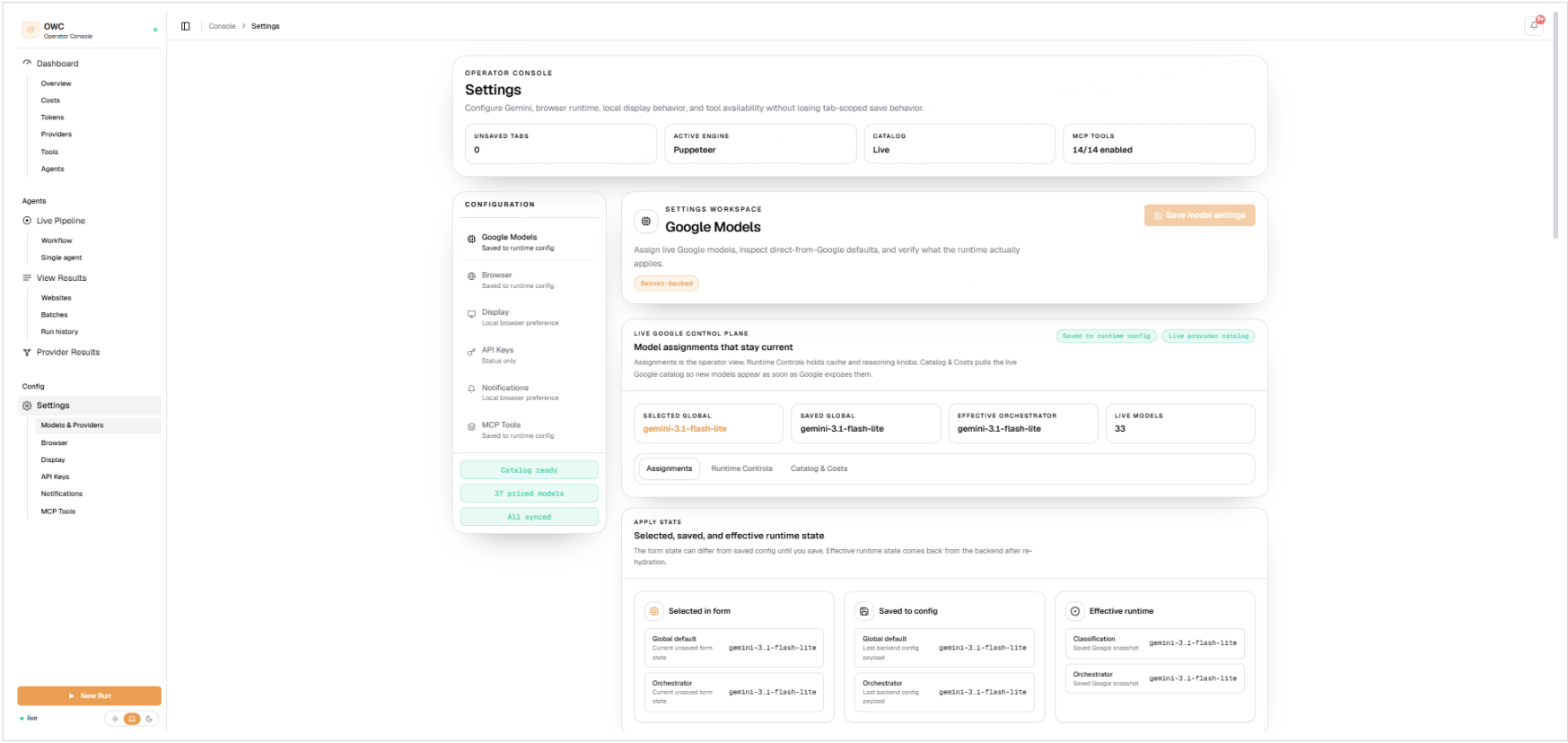


Figure 5.16: OWC model and runtime settings screen.

The settings view exposes the user-facing controls for model assignments, runtime state, cache behavior, browser options, notifications, and MCP tool enablement.

5.3.3 MCP Browser Containers and Puppeteer Tools

The browser part of OWC is deliberately Puppeteer-based. The n8n prototype had already proved that a real Chrome session could open pages, dismiss popups, inspect iframes, activate players, and capture stream requests. OWC kept that proven engine, then moved the messy browser work out of n8n code nodes and into dedicated MCP browser services. The model no longer receives a vague instruction such as "use the browser"; it sees a small set of named tools with typed arguments and structured results [15].

The implementation decision was practical rather than decorative:

- **Chrome DevTools Protocol access** was needed for network capture, request/response diagnostics, failed-request evidence, and browser session control; Puppeteer exposes that path directly [7, 8].
- **Session continuity** mattered more than launching a fresh page for every tool call. The Python MCP client pins one SSE session for the whole agent profile, so `navigate`, `inspect_*`, `interact`, and `harvest` share the same browser state.
- **Profile filtering** prevents tool drift. Classification, landing, hosting, and embedded agents connect to different MCP profiles, and each profile exposes only the tools required by that role.
- **Isolated browser mode** creates a temporary Chrome profile per MCP session when enabled. That keeps cookies, local storage, proxy metadata, fingerprint state, and popup history scoped to one run instead of leaking across unrelated sites.

5.3.3.1 Actual MCP tool shelf

OWC kept Puppeteer as the main browser engine and built a named tool shelf around it. The registry is deliberately explicit; the agent does not ask for a vague browser action, it calls a tool with a real name and a bounded contract.

The implemented tool surface is grouped like this in `tools/puppeteer/tool-registry.js` and mirrored by the profile requirements in `src/tools/mcp_client.py`:

Tool group	Tool names and contract
Navigation tools	Open pages and recover from route drift with `navigate`, `open_url`, `go_back`, `wait_for_page_state`, `scroll_page`, and `scroll_to_element`.
Inspection tools	Read rendered-page evidence with `inspect`, `inspect_landing`, `inspect_hosting`, `inspect_embedded`, `get_page_context`, `query_elements`, `get_element_detail`, `get_frame_tree`, and `get_media_state`.
Action tools	Perform browser moves with `interact`, `click_element`, `click_css`, `click_text`, `click_xpath`, `click_checkbox`, `click_radio`, `type_into`, `select_option`, `play_media`, `swipe_region`, and `click_coordinates`.
Evidence tools	Preserve run evidence with `screenshot`, `capture_streams`, and `harvest`.
Memory tools	Reuse site-specific hints without treating old knowledge as proof with `memory_lookup` and `memory_update`.

The smaller OWC-specific tools are the ones that changed the behavior most. The page-role inspectors keep the model from reading raw DOM noise. `interact` turns player clicks into verifiable browser-state changes. `harvest` watches the network long enough to catch media requests that do not exist in the first HTML response. The figure opens those three tools because they are the implementation details that Chapter 4 only names at architectural level.

The order is intentional. `inspect_*` appears before `interact` because interaction normally consumes the element references, frame paths, and candidate groups returned by inspection. The practical sequence is therefore: stabilize the page, inspect it, interact with one chosen target, then harvest network evidence.

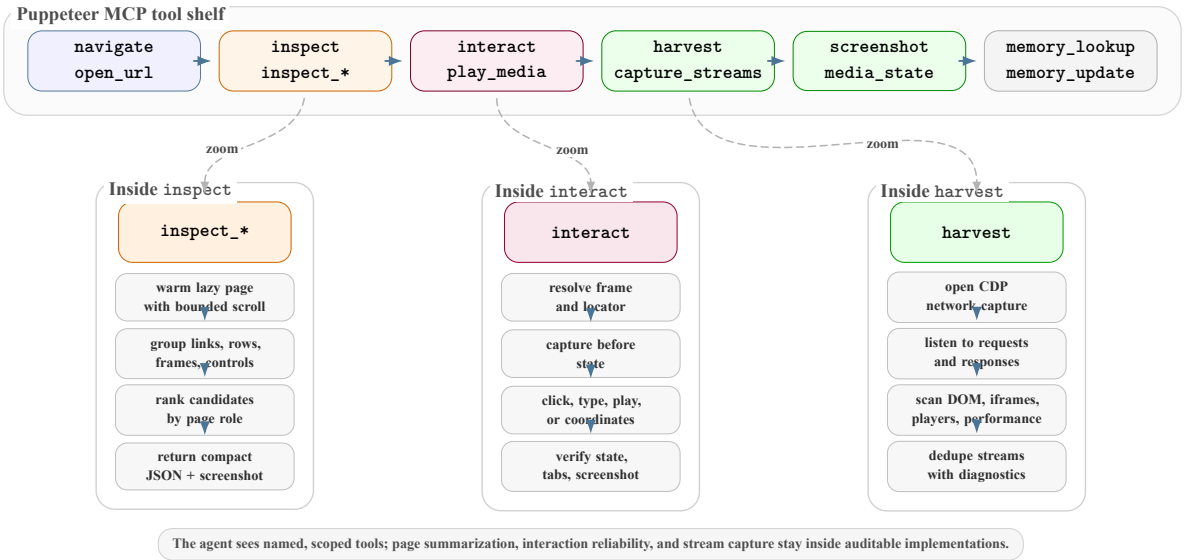


Figure 5.17: Puppeteer MCP toolbox with implementation zooms into `inspect`, `interact`, and `harvest`.

5.3.3.2 Navigation tools: entering and stabilizing pages

The navigation tools are deliberately plain. Their job is to put the browser on the right rendered page before the model starts reasoning from page evidence.

- `navigate` is the main entry tool. It opens the URL, waits with the configured page-load strategy, records redirects and HTTP status when available, and returns the final URL instead of assuming the input URL survived unchanged.
- `open_url` is the lighter direct-open path used when the agent already knows the browser should leave the current route.
- `go_back` exists because popup recovery and wrong-route recovery happen often on streaming sites. It lets the agent return to the previous page without guessing the original address.
- `wait_for_page_state` gives scripts and player shells time to settle after a click, server switch, or iframe adoption.
- `scroll_page` and `scroll_to_element` are used to reveal lazy sections, schedule rows, hidden server lists, and controls that are not present in the first viewport.

In practice, these tools are simple on purpose. They create a clean state for the more interesting tools. A page that redirected through a challenge, opened a popup, or never finished loading is marked as such before the agent spends tokens on detailed inspection.

5.3.3.3 Inspection tools: compressed page-role reads

The architecture chapter only needs the idea of inspection. The implementation has to be more concrete: OWC's inspectors scroll enough to warm up lazy content, reset the viewport to the top, group candidate evidence, and return compact fields that the prompts can cite directly. They are closer to page summaries than to DOM dumps.

The important design choice is that inspection is role-aware. The same page can contain navigation links, article links, ad frames, player frames, schedule rows, server buttons, and stream-looking strings. A single unstructured DOM dump makes the LLM sort all of that inside the prompt. The inspector scripts do the first pass in code: they group repeated sections, classify candidate actions, keep representative selectors and XPath handles, and attach screenshots or frame metadata when useful.

Each inspector has a different job:

- `inspect` is the general read used by classification. It returns classification hints, grouped links and actions, top candidates, player evidence, frame overview, pagination, blockers, and lazy-load statistics. The point is to decide the page role without filling the prompt with raw HTML.
- `inspect_landing` is tuned for listings. It keeps grouped sections, match groups, navigation groups, candidate ledgers, reveal actions, collapsed sections, pagination, and popup evidence together. This is what keeps a schedule row or match card attached to the correct candidate URL.

- `inspect_hosting` is built around source selection. It returns server frontier entries, event-server routes, blocker candidates, playback groups, iframe groups, player evidence, top server controls, and activation targets. The hosting prompt can then choose a real server or play target instead of guessing.
- `inspect_embedded` narrows the same idea to player pages and iframes. It returns frame-focus groups, source controls, player targets, blocker candidates, popups, and media evidence so the embedded agent stays inside the assigned player context.

This is why the inspect tools are part of prompt engineering, not just browser plumbing. They give the model small, named facts such as `server_frontier`, `activation_candidates`, or `frame_focus_groups`; the prompt can then ask for an exact action and verify the result.

The compression is also intentional. The inspectors estimate payload size, keep representative rows, and prefer bounded fields such as `candidate_ledger`, `top_match_candidates`, and `top_playback_targets`. That keeps the browser evidence readable for a human reviewer and cheap enough for repeated agent turns.

5.3.3.4 `interact`: reliable browser interaction

`interact` exists because a normal click helper was too weak for this domain. Streaming pages often put the real control behind overlays, inside iframes, or on canvas-like player areas that do not have stable selectors. The tool accepts several action modes–click, play, type, select, checkbox, radio, or coordinates–and can target an element reference, CSS selector, XPath, visible text, frame path, or frame URL fragment.

The tool does a small amount of browser work before it claims success:

- it resolves the frame first, usually `root` or an iframe path such as `root.0`;
- it tries the inspected element reference, then falls back to selector, XPath, or text when the original handle has gone stale;
- it records a before-state with the URL, title, text sample, video count, iframe count, scroll position, and target snapshot;
- it performs the action, watches for popup tabs, waits for the page to settle, and returns the after-state with verification fields such as `verified`, `verification_reason`, and `opened_targets`.

The mouse-movement detail is small but intentional. If the model supplies coordinates, OWC does not jump straight to the point. It checks the element under the point, moves through an intermediate location, and then clicks with several mouse steps. This helps on players that react to hover or pointer movement before accepting a click, and it gives the trace a clearer record of what was under the pointer before and after the action.

The tool also treats popups as part of the result instead of hiding them. It tracks new tabs, blocked popup attempts, adopted player pages, selected targets, and the active page URL. That is why a hosting trace can later explain whether a click activated a real player, opened an ad page, or moved the run into a different browser target.

5.3.3.5 `harvest`: stream URL capture

`harvest` is the second tool worth opening up because the stream URL is rarely waiting in the first DOM read. On many sites it appears only after a play click, a server switch, or a player script creates a request. The tool keeps a Chrome DevTools Protocol network session open during a capture window and combines several sources of evidence:

- outgoing requests from `Network.requestWillBeSent`;
- response headers from CDP and Puppeteer response events;
- failed-request diagnostics, which help explain blocked or geo-restricted media;
- direct `<video>` and `<source>` values when the page exposes them;
- iframe URLs and player objects such as hls.js, Video.js, JWPlayer, and DASH players;
- a retrospective scan of `performance.getEntriesByType("resource")` for resources the browser had already requested.

The output is shaped for review, not just extraction. It deduplicates candidates, classifies HLS, DASH, MP4, WebM, segment, and Smooth Streaming URLs, records which layer found each candidate, returns protocol-specific lists such as `m3u8_urls` and `mpd_urls`, captures a player screenshot, adds iframe diagnostics, and reports the current video state. A hosting or embedded agent can therefore carry the harvest result directly into its final evidence bundle.

5.3.3.6 Ad blocking, proxy rotation, and fingerprint rotation

These controls sit in the browser runtime because they are policy and environment problems, not reasoning tasks for the LLM. The agent can report what happened, but it should not be the component deciding how Chrome looks on the network.

Ad blocking. OWC uses a browser policy around a uBlock Origin Lite-style setup, with the filter sources stored in `tools/puppeteer/shared/filterlists/sources.json`. The default enabled lists are intentionally moderate:

- **advertising and mobile ads:** EasyList, AdGuard Base, AdGuard Mobile Ads, and uBlock Origin Filters;
- **site repair and anti-breakage:** uBlock Origin Unbreak, uBlock Origin Quick Fixes, AdGuard Quick Fixes, and EasyList Anti-Adblock;
- **language support used in the default set:** Liste AR for Arabic-language pages.

Other lists are present but disabled by default, including EasyPrivacy, AdGuard Tracking Protection, AdGuard Social Media, AdGuard Annoyances, uBlock Privacy, uBlock Badware, uBlock Resource Abuse, uBlock Annoyances, Fanboy annoyance lists, HaGeZi lists, OISD, NoCoin, and several regional lists. Keeping them available but off by default mattered because aggressive privacy or annoyance filtering can break embedded players.

The runtime tested the blocking behavior with `https://adblock-tester.com/` and `https://adblock.turtlecute.org/`. Those tests were useful because they expose whether the browser is blocking obvious ad requests, but OWC still treats streaming pages differently

from ordinary pages. In standard mode the policy can load blocking into an isolated Chrome profile and record `blocked_by_client` request failures. In streaming-safe mode, especially for hosting and embedded agents, blocking can be relaxed when the evidence suggests that filtering is breaking the player. That tradeoff is deliberate: a perfect adblock score is less useful than a player trace that still records which requests were blocked.

Proxy rotation. The proxy implementation is validation-first. It can use custom proxies or remote pools from OpenProxyList, Proxifly, monosans, and TheSpeedX, with HTTP tried before the SOCKS sources in the configured order. The pool code normalizes each candidate, removes duplicates by scheme, host, port, and credentials, then validates candidates against `https://api.ipify.org?format=json`. The active runtime uses a 25-candidate cap, an 8-second fetch timeout, a 12-second validation timeout, and a 10-minute cache TTL.

The rotation mode is `sticky`. In other words, OWC does not pick a new proxy for every click. It keeps the first healthy proxy for the current engine/profile and rotates only after failure. That made the browser behavior more stable because cookies, player tokens, and manifest URLs can be tied to the route that created them. If validation fails for all candidates, the configured fallback is direct browsing, so proxy failure does not get confused with a site-down result.

Fingerprint rotation. Fingerprinting was already introduced earlier, but the implementation detail is worth spelling out here. The Puppeteer runtime imports `fingerprint-generator` and `fingerprint-injector`. It generates a browser profile, synchronizes it with the real Chrome version, and injects it into the page before the agent starts tool work. The runtime disables the Puppeteer stealth evasions that would conflict with the explicit fingerprint values, especially user-agent override and navigator plugin handling.

The generated profile is not used blindly. OWC aligns it with a recent Chrome build by checking `OWC_CHROME_VERSION`, the running browser version, Chrome for Testing metadata, the Chrome version-history API, and finally a fallback version. It then writes consistent values for Windows platform data, `User-Agent`, `Accept-Language`, `navigator.userAgentData`, and client hints such as `Sec-CH-UA`, `Sec-CH-UA-Platform`, and `Sec-CH-UA-Full-Version`. Rotation is currently origin-scoped, with a three-minute rotation interval, a six-use maximum, and a recent-pool guard so the same profile is not immediately reused.

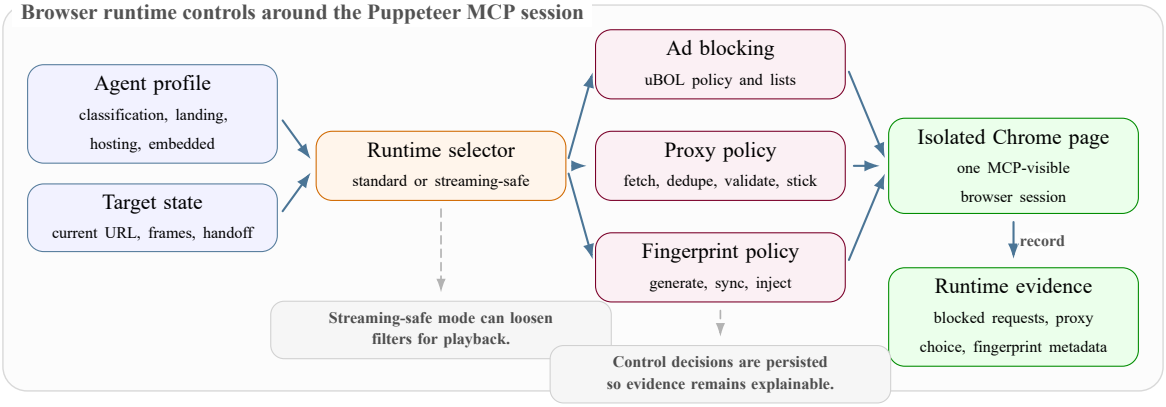


Figure 5.18: Runtime controls applied before a Puppeteer browser page is exposed to the MCP tools.

5.3.4 Agents, Models, Memory, and Prompts

Chapter 3 introduced the general concepts: ReAct loops, prompt contracts, token usage, workflow automation, LangChain, LangGraph, and MCP tool servers. This section is the implementation counterpart. It explains how those concepts are used in OWC's actual agent runtime, model calls, memory layers, prompt compiler, and browser-tool contracts.

OWC does not use one browser agent for everything. That was one of the lessons from the n8n phase: a single broad agent tends to mix routing, clicking, stream extraction, and final reporting in the same context. The implemented version splits the work into specialists. Each one gets its own prompt, its own MCP tool profile, and its own tool budget.

The split is simple:

- **Classification** answers the first routing question: is this URL a landing page, a hosting page, an embedded player, or something else? It mainly uses broad inspection, screenshot context, frame trees, and memory lookup.
- **Landing** works on lists, schedules, tabs, and pagination. Its job is to keep match or channel context attached to candidate URLs instead of returning a loose pile of links.
- **Hosting** is responsible for server buttons, play controls, popups, iframe hints, media state, and stream harvesting. It is the agent that usually decides whether a page can produce stream evidence directly.
- **Embedded** stays inside embedded-player URLs and iframe-local contexts. It does less site navigation and more player-focused verification.
- **Provider analysis** runs after streams are found. It turns recovered network URLs into provider evidence: hostname, IP address, ASN/organization, normalized provider name, country, Whois/RDAP details, and abuse contact when available.
- **Email generation** consumes the provider rows and extraction evidence to draft takedown emails for human review. It does not send them automatically.

The split is enforced in two places. The prompt tells the agent what it owns, but `src/tools/mcp_client.py` also checks the required tool set for that profile before the run starts. If a hosting profile is missing `inspect_hosting`, `interact`, `harvest`, or

`get_media_state`, the session fails early instead of letting the model continue with a broken browser surface. This made agent behavior easier to debug because a tool-availability failure is separated from a model-reasoning failure.

5.3.4.1 LangGraph orchestration and bounded fan-out

LangGraph is used as the workflow backbone, not as an extra reasoning layer. The implementation in `src/agents/orchestrator.py` uses deterministic nodes for classification, landing-page extraction, hosting-page extraction, embedded-page extraction, provider analysis, and email generation. Conditional edges route from classification to the right specialist, then route again based on the queues left in shared state. The state carries the seed URL, classification result, matches, extraction results, pending hosting URLs, pending embedded URLs, provider rows, draft email data, and error information.

The main improvement over n8n is how follow-up work is scheduled:

- in the prototype, candidates moved through explicit handoffs: classify, then landing, then hosting, then embedded;
- in OWC, landing output becomes queues of targets, so several hosting pages can be tested in the same stage;
- the hosting node creates one task per hosting URL, limits concurrency with `asyncio.Semaphore`, and waits for the batch with `asyncio.gather`;
- the embedded node uses the same pattern for player or iframe URLs;
- the setting `max_parallel_hosting_pages` lets the operator raise or lower the number of browser agents that can overlap.

The fan-out is still dependency-aware. If the landing agent already found embedded targets, those can enter the embedded queue. If the embedded URL only appears after a hosting agent opens a server or iframe, OWC waits for that hosting evidence first. That keeps the run faster than the n8n handoff chain without letting embedded agents invent player URLs before the browser has actually seen them.

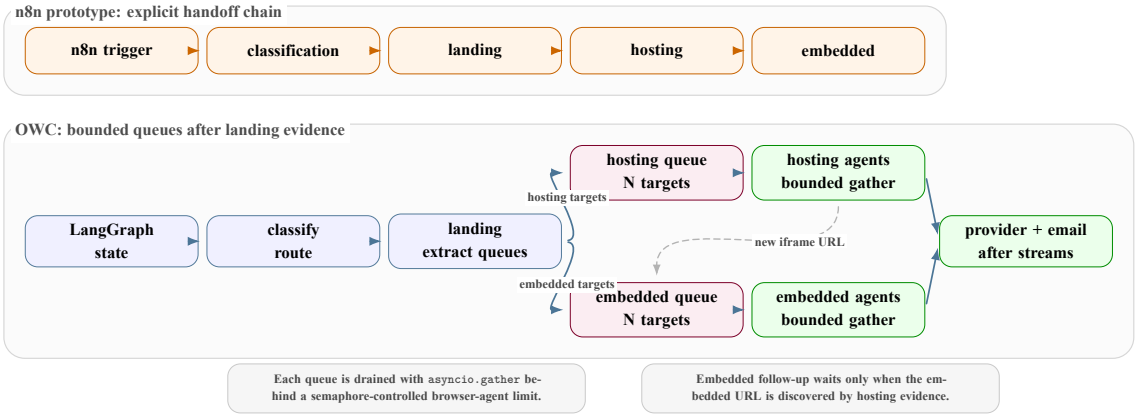


Figure 5.19: n8n handoff chain compared with OWC's LangGraph queues and bounded browser-agent fan-out.

5.3.4.2 Provider analysis and email handoff

Provider analysis is the bridge between browser evidence and an actionable email draft. The browser agents may recover a network URL such as an HLS manifest, DASH manifest, MP4 file, segment URL, or a disguised stream path. That URL is useful evidence, but the email generator needs to know who appears to operate the network endpoint. In the implementation, this is handled after hosting and embedded extraction, before takedown draft generation.

The implemented chain is:

- **Collect stream URLs.** The orchestrator calls `_collect_all_streams`. It reads explicit `StreamURL` rows, server `stream_urls`, `m3u8_urls`, `mpd_urls`, `mp4_urls`, and older metadata fallbacks. It keeps only provider-relevant network URLs using `_looks_like_provider_stream_url`, so ordinary page assets are not sent to provider lookup.
- **Resolve the network owner.** `IPInfoTool` receives those stream URLs, extracts the hostname, resolves it to an IP address, skips localhost or private addresses, and queries IPInfo plus RDAP/Whois data. If an organization string looks like `AS13335 Cloudflare, Inc.`, the helper strips the ASN prefix and keeps the readable provider name, for example `Cloudflare, Inc.`
- **Store provider evidence.** The result is a `ProviderInfo` row with `stream_url`, `ip`, `hostname`, `org`, `provider`, country/region/city fields, `abuse_email`, and raw Whois/RDAP payload when available.
- **Draft the email.** The email generator groups rows by provider/contact, matches each provider row back to the original stream URL, attaches screenshots, server labels, page URL, channel name, and protocol evidence, then creates a `TakedownEmail` draft. The provider name and abuse contact are used in the subject, recipient line, and evidence block.

This step is important because the final reviewer should not receive only "we found a stream". The draft should say which provider endpoint was observed, which URL led to that provider, what screenshot or server action produced the URL, and whether an abuse contact was resolved. If the lookup cannot find an abuse address, the generated email keeps a review note instead of pretending that the contact is known.

5.3.4.3 Design constraints implemented in the agent runtime

The agent runtime was shaped by problems that appeared during the prototype, not by a clean theoretical workflow. Each constraint changed something concrete in the code, prompt contract, or browser tooling.

5.3.4.4 Prompt evolution in the implementation

The prompts became more operational as the project moved from n8n to OWC. The first version sounded like a task request: inspect the page and find the stream. That was not enough. On real sites, the agent has to notice popups, test server buttons, preserve row context, avoid ad redirects, and stop honestly when the page is unavailable.

Constraint	Runtime effect	Implementation response
Changing layouts	Selectors and visible labels differ across languages, clones, and match-list layouts.	Agents use broad DOM context first, then scoped element details and page-type-specific tools.
Nested iframe players	The playable media source may not exist in the top-level page.	Hosting agents can hand off embedded URLs; embedded agents inspect frame trees and player-only pages.
Popups and ad traps	Clicks can open new tabs, steal focus, or hide the original player.	Browser tools restore focus, close popup tabs, and retry with safer JavaScript interactions.
Delayed stream generation	Stream manifests often appear only after a play click or server switch.	Prompts require media-state or screenshot verification before harvesting network evidence.
Cost and context growth	Repeated DOM observations, screenshots, and retries can make runs expensive.	The runtime tracks token usage, uses caching, and enforces bounded tool budgets.
Evidence isolation	The reference system must not depend on company production data.	Persistence is limited to traces, metrics, screenshots, tool calls, model usage, and structured outputs.

Table 5.4: Runtime constraints and their implementation responses.

The implemented prompts use the same ReAct vocabulary–`OBSERVE`, `STATE`, `HYPOTHESIS`, `ACTION`, and `VERIFY`–but each prompt applies it to a different job:

- `classification_v1.md` keeps the first decision small: identify the page type and explain what evidence would change that decision.
- `landing_page_v1.md` treats discovery as frontier management. A candidate can be accepted, rejected, expanded, continued through pagination, or stopped with a named blocker.
- `hosting_page_v1.md` maintains a server/source frontier. After each click or play attempt, the server is marked as activated, failed, blocked, harvested, or needing embedded follow-up.
- `embedded_page_v1.md` protects player ownership. It should not drift back to site navigation when the assigned job is to verify a player frame or embedded URL.
- `orchestrator_v1.md` reads agent outputs as evidence. It decides whether a failure is site-down, popup drift, wrong-route evidence, no-stream evidence, or a real blocker before scheduling another step.

The prompt compiler in `src/agents/prompting.py` is where this becomes operational. It does not concatenate prompt text casually. It builds a layered prompt with a stable prefix and a dynamic tail:

- **Stable prefix:** `BASE POLICY`, `AGENT CONTRACT`, and `RUNTIME CONTEXT`. These sections contain the role rules, evidence rules, tool-use style, output contract, and runtime capabilities that change slowly.
- **Dynamic tail:** `TASK BRIEF`, `SITE MEMORY HINTS`, and `WORKING STATE`. These sections contain the current URL, current objective, remembered domain hints, and latest run state.
- **Trace metadata:** prompt hash, compiled prompt hash, section names, output-contract version, cache mode, provider cache key, and cache eligibility.

That layout was chosen for cache behavior. The static prefix is reused across many runs of the same agent and contract, while the changing page state is placed later. This does not guarantee that every provider will return a cache hit, but it gives the provider a stable reusable prefix and lets OWC record whether cached input tokens were actually reported. It also makes debugging easier: a later trace can show which prompt contract, runtime context, cache mode, and memory injection produced the answer.

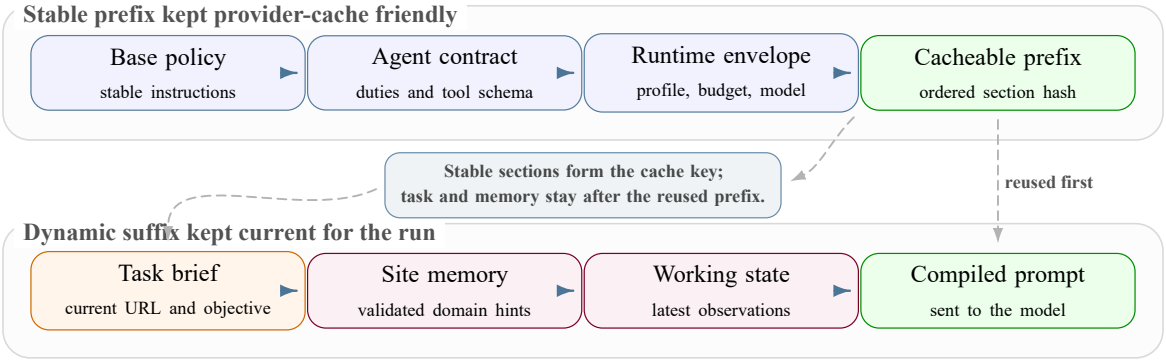


Figure 5.20: Prompt assembly layout used to keep stable instructions cache-eligible while memory and task state remain current.

5.3.4.5 Provider and tool-output caching

Caching is implemented at two levels because OWC repeatedly sends two kinds of input. The first source is the model prompt. In `src/agents/prompting.py`, `compile_agent_prompt` builds the prompt as a stable prefix followed by a dynamic suffix. The stable prefix contains `BASE POLICY`, `AGENT CONTRACT`, and `RUNTIME CONTEXT`; the dynamic suffix contains the current task, site-memory hints, and working state. The compiler stores a local static-prefix cache key and emits provider metadata: cache mode, provider cache key, cache eligibility, prompt hash, compiled prompt hash, and section names.

The Gemini runtime then uses that metadata in the shared agent loop. When provider caching is enabled, the loop marks the run as cache-active if the prompt is eligible and the selected Gemini model can reuse a stable prefix. For calls that can use Gemini's explicit cached-content API, `GeminiCacheManager` creates or refreshes a `cachedContents` resource with a TTL and keeps its name in an in-process registry. When browser tools are bound, the runtime does not force an explicit cached-content handle because Gemini's tool-enabled `GenerateContent` path cannot be combined with that handle. In that case OWC still keeps the prefix stable and records the provider-reported cached-token fields when Gemini applies provider-managed caching.

The second source of repeated input is the tool result. Browser tools can return long page summaries, frame trees, candidate links, media state, and network evidence. Those outputs are not only diagnostic data; after a tool call they become `ToolMessage` content and are sent back to the model on the next turn. That makes them one of the most expensive input blocks in a long run. OWC therefore uses `ToolResultCache` for safe repeated observations such as page context, `query_elements`, element detail, frame tree, media state, and the profile inspectors. A result is reused only after the same tool and arguments have produced identical observations at least twice. Any state-changing browser action, such as navigation, clicking, typing, scrolling, interaction, or media playback, advances the cache generation and invalidates previous observations.

This design is deliberately stricter than a simple LangChain loop that appends whatever message history and tool outputs were produced by the last turn. LangChain still binds the tools and carries the messages, but OWC controls the cache boundary outside the framework: stable instructions first, changing state later, explicit cache metadata for Gemini where compatible, and application-level reuse for stable tool observations. The observer records cache hits, cached input tokens, new input tokens, cache-write tokens, tool-cache hits, and estimated costs, so the operator can see whether the caching policy actually reduced repeated input rather than assuming it did [2, 3].

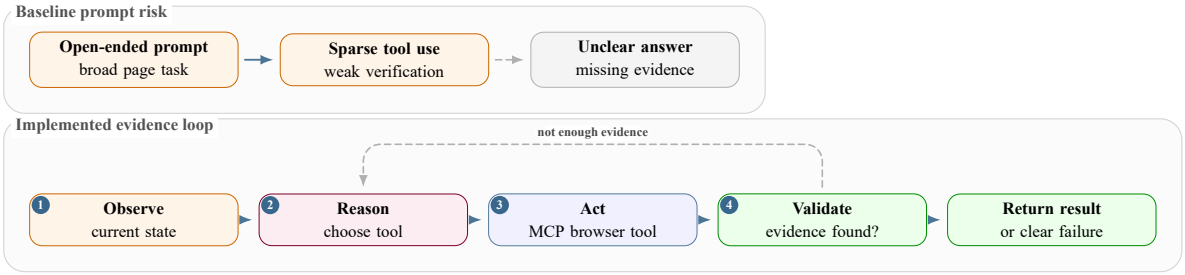


Figure 5.21: Prompt design contrast between an open-ended browser task and the evidence-driven agent loop used in OWC.

5.3.4.6 Memory and bootstrap context

Memory is used as a hint layer, not as proof. The distinction between short memory and long memory matters because they solve different problems.

- **Short-term memory** lives inside one extraction run. It records visited URLs, navigation events, tool calls, selectors, candidate hosting URLs, match records, server frontier entries, activation targets, observed browser changes, screenshots, stream URLs, and compact observations. The agent uses it to avoid repeating the same broad reads and to carry context from one turn to the next.
- **Long-term memory** is the reusable site profile. It is built from confirmed run evidence and can store URL patterns, pagination rules, selectors, server labels, popup dismissals, stream hosts, playbook steps, failure cues, and continuation notes. It is keyed by domain and page type, so a future landing run does not inherit a hosting-only shortcut by accident.

At the start of a profile run, prompts ask for `memory_lookup` before expensive exploration. The returned profile is injected as `SITE MEMORY HINTS`, beside the fresh browser observation, not above it. At the end of useful work, `memory_update` persists only what the current run confirmed: for example a stable server label, a pagination pattern, a popup close selector, a working frame path, or a stream-host pattern.

The guardrails are important:

- remembered concrete URLs are treated as patterns or hints, not as permission to leave the current target page;
- a remembered selector still needs a live page check before the agent trusts it;
- failed routes and popup traps can be remembered too, so future runs avoid known bad moves;
- short-term memory can be summarized into a continuation capsule when context compaction starts.

Memory layer	What it stores	How the agent uses it
Short-term run memory	Current-run URLs, tool attempts, candidate ledgers, server frontier, activation targets, observed changes, screenshots, and stream clues.	Keeps the next turn grounded in what just happened and prevents repeated broad inspection loops.
Long-term site memory	Domain/page-type patterns, stable selectors, pagination rules, server labels, stream hosts, popup controls, playbook steps, and failure cues.	Bootstraps future runs with hints through `memory_lookup`; updates only after current evidence confirms the pattern.

Table 5.5: Short-term and long-term memory responsibilities in OWC.

5.3.4.7 Agent design principles

The resulting agent design follows a few practical rules:

- classify before extracting, because a landing page and an embedded player need different tools;
- prefer browser evidence over textual guesses;
- keep screenshots, traces, tool calls, and structured outputs for review;
- split responsibilities across specialist agents instead of giving one prompt the whole system;
- control cost with budgets, caching, and model/runtime telemetry;
- keep the platform loosely coupled so browser workers, APIs, storage, and queue execution can be tested separately.

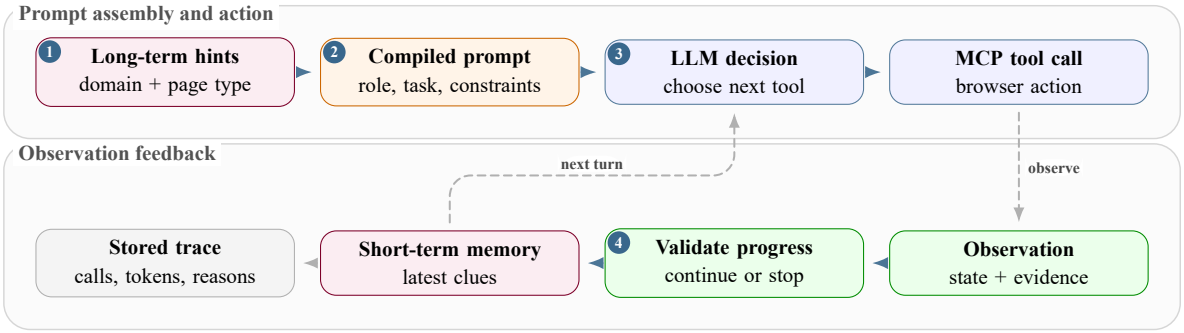


Figure 5.22: Agent reasoning loop with memory and tool feedback.

5.3.4.8 LangChain message loop and tool binding

LangChain is the layer that connects the prompt to callable tools [13]. In OWC, the model never receives the full internal browser registry. The MCP client loads the current profile, then the shared loop binds only that list through `llm.bind_tools(tools)`.

The message flow is explicit:

- `SystemMessage` carries the compiled prompt;
- `HumanMessage` carries the task, URL, and turn context;
- `AIMessage` carries the model response and tool calls;
- `ToolMessage` carries the serialized result returned by the browser or memory tool.

The shared agent loop is also implemented as a small LangGraph graph. It has nodes for the model call, tool dispatch, context compaction, and budget exhaustion. The tool node performs the operational work:

- it refuses tool calls beyond the remaining budget;
- it invokes tools asynchronously with `ainvoke` and applies timeouts;
- it records arguments, durations, cache status, and full tool results;
- it updates short-term memory with navigation and tool events;
- it invalidates cached tool results after state-changing browser actions;
- it warns when the agent repeats broad `query_elements` calls instead of choosing a more specific action.

That loop is also where the system records the operational evidence needed for review: tool arguments, durations, full tool results, cache status, model usage, estimated cost, thinking tokens when the provider exposes them, and context-window pressure. Those traces are not meant to be read line by line, but they explain why the console can show what happened instead of only showing a final label.

5.3.4.9 Context-window compression before continuation

Long browser runs can fill the model context with repeated page summaries, screenshots, tool outputs, and retry history. OWC handles that in the shared agent graph instead of leaving the model to fail at the provider limit. When context usage crosses the configured continuation

threshold, the graph emits `context_compaction_started`, builds a continuation capsule, removes the old message history, and starts a new invocation with only the system prompt, original task, and compact state.

The capsule keeps the parts that are still useful:

- objective, target URL, page type, actor, and continuation index;
- tool budget used and remaining;
- visited URLs, confirmed URL patterns, and pagination patterns;
- pending hosting or embedded frontier items;
- server evidence, activation targets, screenshots, streams, blockers, and the next best move.

It drops the bulk that no longer helps: old repeated tool messages, stale broad inspection payloads, superseded screenshots, and long conversational traces. A simple test case shows the intended shape: before compaction the run can be at 8,500 context tokens in a 10,000-token window, or 85% usage; after compaction the continuation can restart around 1,400 tokens, or 14% usage, while preserving the frontier and evidence needed to continue.

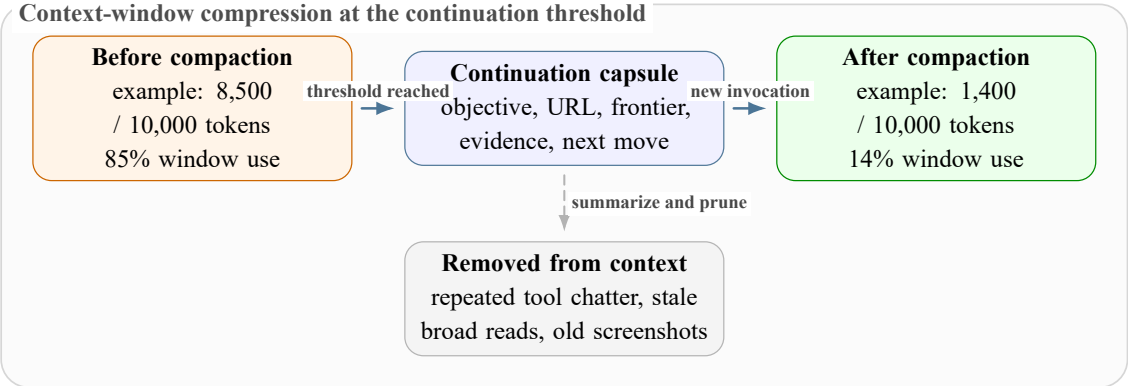


Figure 5.23: Before-and-after view of OWC context compaction when the model context approaches the configured threshold.

5.3.4.10 Model behavior, token usage, and rate limits

The model layer is treated as an operational dependency, not just as a prompting detail. Long page observations, screenshots, retries, and tool traces can grow the context quickly. OWC therefore records context-window metadata, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are tracked across RPM, TPM, and RPD: requests per minute, input tokens per minute, and requests per day [1]. These limits can depend on the selected model and usage tier, so OWC treats them as configuration and observability data instead of hard-coded constants.

5.3.4.11 Prompt contracts and evidence rules

Prompt engineering in OWC means writing executable rules for evidence collection, not just nicer task wording. Each prompt defines page ownership, the allowed evidence, the required JSON output, and the situations where the agent must stop instead of guessing.

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.6: Model-runtime concerns handled by the agent platform.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.7: Prompt engineering rules enforced by the specialist agents.

5.3.4.12 Runtime stopping and continuation control

Stopping behavior is separate from prompt engineering. The prompt tells the model what a good final answer looks like, while the runtime decides when browser work must stop. In `src/agents/base.py`, the shared agent graph tracks tool-call budget, repeated tool batches, low-specificity `query_elements` loops, context-window usage, and site-down errors.

The stop logic combines several signals:

- a fixed tool-call budget for each agent profile;
- repeated tool-batch detection, which catches loops where the same action is tried again without new evidence;
- a no-progress counter for cached or repeated observations;
- context compaction before the model window becomes too large;

- a final budget message that asks the agent to return structured JSON when no more browser tools should run.

This design keeps failures reviewable. A run can end with a clear partial result, `site_down`, `no_progress`, or `budget_exhausted` reason instead of a long trace of repeated broad reads and failed clicks.

5.3.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its job is to preserve what happened during a browser-agent run so that a success, a failure, or a partial result can be inspected later.

5.3.5.1 Two storage views for one run

The main implementation choice was to store each run in two complementary views:

- **a replay view**: `run_snapshots.snapshot_json` keeps the full pipeline result as JSON, so the system can reload the final shape even if the console changes later;
- **an observability view**: normalized rows in `pipeline_runs`, `agent_runs`, `tool_calls`, `llm_calls`, `runtime_events`, `run_streams`, and `run_screenshots` make the run searchable and sortable from the UI.

This split is useful because the final agent JSON is good for replay, but bad for questions such as "which hosting attempt opened this stream?" or "which tool call created this screenshot?" The normalized rows answer those questions without making the operator read one large nested blob.

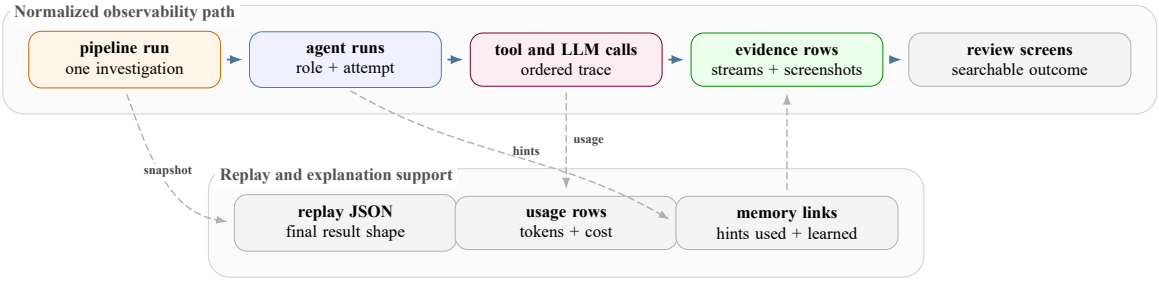


Figure 5.24: How PostgreSQL turns a parallel browser-agent run into replayable and searchable observability records.

5.3.5.2 Attribution across parallel agents

The hard part was attribution. OWC can run several hosting agents at the same time, and each one can return screenshots, iframe URLs, stream rows, popup diagnostics, and final agent JSON. The repository therefore stores contextual fields with the extracted evidence whenever they are available:

- `agent_run_id`, `actor`, `agent_type`, and `invocation_index` identify the specialist and the parallel attempt;
- `tool_name`, `target_url`, and `seq` link a screenshot or stream-like observation back to the trace event that produced it;
- `dedupe_hash` on stream rows prevents the same manifest URL from being counted repeatedly inside one pipeline run;
- token, cache, context-window, and cost fields stay on both agent-level and model-usage rows, so cost can be reviewed by run, agent, provider, and model.

This is the database detail that mattered most in practice. Without attribution, the UI would show only that a run found a screenshot or a stream. With attribution, it can show whether the evidence came from the first hosting URL, the third parallel hosting URL, or an embedded follow-up created by a hosting result.

5.3.5.3 What observability made possible

The schema also supports the review screens used later in the report. Dataset tables group sites and batch runs. Runtime events show the chronological trace. Tool calls show arguments, duration, preview, and errors. Model-usage rows show token and cost behavior. Memory tables preserve reusable domain patterns and record which hints were used by an agent run.

The result is a database designed for explanation. It does not only store the final classification; it stores enough of the route, tool surface, and evidence chain to understand why the system reached that classification.

5.4 Part 3: Deployment

The company deployment decision was intentionally focused. The existing company workflow remained the primary operational system, and the deployment did not attempt to replace it with the complete OWC platform or with all four n8n prototype agents. Technically, the internship produced classification, landing, hosting, and embedded n8n agents, but the deployed unit was only the validated n8n landing-agent backup: a browser-capable agent used when the primary flow cannot extract enough evidence from a dynamic landing page.

Figure 5.25 shows the deployment shape that follows that practical boundary:

- the company workflow submits a fallback task with the target URL and job identifiers to the `agent_tasks` Azure Service Bus (ASB) queue;
- the ASB trigger starts a focused Azure Container Apps Job rather than the full OWC platform;
- the job container runs the n8n landing-agent backup, its Puppeteer MCP endpoint, and a local Chrome profile for rendered-page inspection;
- the agent combines browser evidence with the configured model endpoint and target landing pages;

- the result payload is published to the configured results queue for review and integration [19, 20].

The returned payload is meant for operational review and integration rather than automatic enforcement. It can include:

- matched URLs, match or channel metadata, and screenshots;
- candidate URLs, rejected URLs, logs, and failure reasons;
- reasoning and extraction metadata needed by the company workflow.

This keeps the company workflow, queue interface, and browser-agent backup loosely coupled. The primary process continues normal discovery, while the landing agent handles rendered-page work only for cases that need it.

This deployment choice matches the internship scope. OWC shows how a complete platform could be structured, but the company-side deployment remained focused on the backup landing agent as a cloud-executable single-container unit with queue-based input and queue-based output.

Deployment element	Role	Scope during internship
n8n landing-agent backup	Handles dynamic landing pages when the primary company workflow needs browser reasoning.	Deployed as the focused operational backup.
Azure Service Bus (ASB)	Decouples fallback submission from browser execution.	Used to pass landing-page jobs to the container worker and return results.
Container Apps Job	Runs one containerized agent unit when a message arrives.	Single image for the landing-agent worker, Puppeteer MCP, and Chrome.
Puppeteer runtime	Opens the page and extracts match/host candidates through the local MCP endpoint.	Runs inside the same job container as the landing agent.
Output queue	Publishes matched URLs, metadata, screenshots, rejected URLs, logs, and failure reasons to the configured results queue.	Used for review and integration back into the workflow.

Table 5.8: Landing-agent deployment scope and responsibilities.

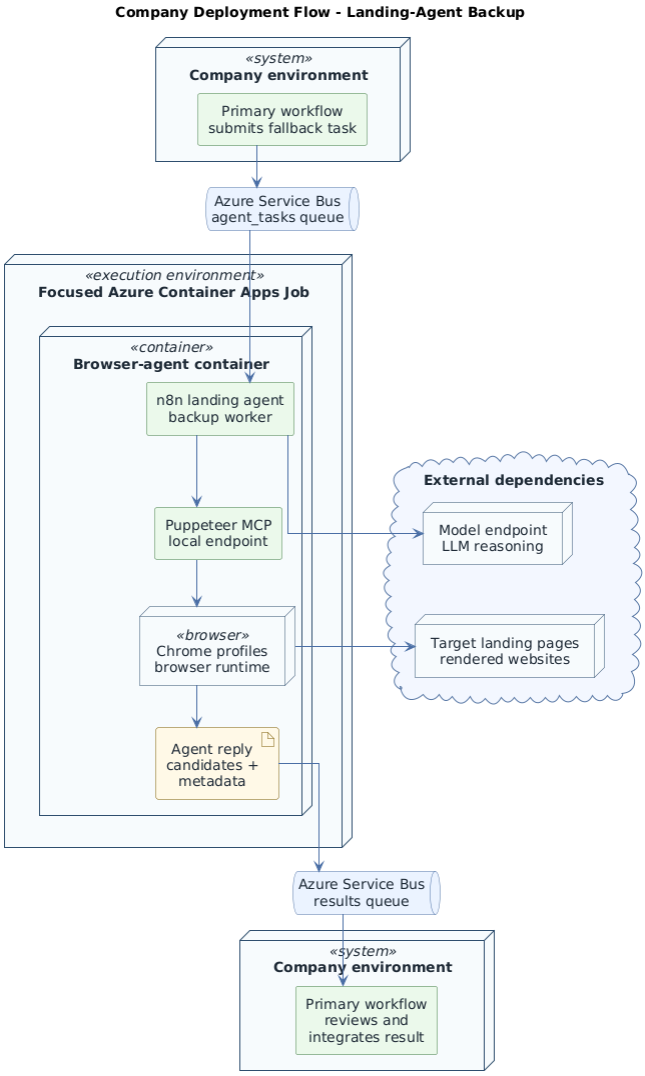


Figure 5.25: Company deployment flow for the landing-agent backup, from the primary workflow and Azure Service Bus queues to the focused Azure Container Apps job, browser runtime, external dependencies, and result payload.

5.5 Conclusion

The implementation moved from a useful but limited n8n prototype to the structured OWC platform. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. OWC addressed those problems with explicit backend routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

The result is a reviewable agent platform rather than a single extraction script: failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

Chapter 6 evaluates whether these implementation choices produced reviewable evidence, controlled tool behavior, and measurable model cost in actual browser-agent runs.

CHAPTER 6

Evaluation and Testing

6.1 Introduction

This chapter evaluates Open Web Catcher (OWC) as an agentic evidence-production system for rights-protection review. It does not claim universal piracy detection, legal judgement, or permanent coverage of every streaming website. The practical question is narrower and more useful: can a suspected streaming website be inspected well enough to produce a traceable evidence package that a human reviewer can understand?

The evaluation is therefore evidence-first. A run is useful when the trace explains what the agents saw, which route was selected, which page states were tested, which player or server path was attempted, and which evidence objects were produced. The evaluation also records when evidence could not be produced because the source site, hosting page, embedded player, stream server, provider lookup, or API quota blocked the run.

The chapter is organized around five evaluation questions:

- Does the system turn unstable websites into reviewable evidence bundles?
- Which agent responsibilities work, fail, or require manual review?
- How does OWC compare with the earlier n8n playground in coverage and operating discipline?
- Which LLM runtime is practical for repeated browser-agent testing?
- How do prompt design and tool families keep browser actions reliable without exhausting context or rate budgets?

6.2 Evaluation Protocol

6.2.1 Reviewable Evidence Bundle

OWC is evaluated as a workflow, not as a single classifier. A reviewable evidence bundle should contain a meaningful subset of the following objects:

- seed URL, persisted run identifier, and route decision;
- specialist-agent trace for classification, landing, hosting, embedded, provider, and email steps when those steps are reached;
- screenshots that support the route, player state, server path, or failure condition;
- stream candidates such as manifest, embedded, media, or playlist URLs;
- provider enrichment for the stream or playlist host;
- abuse-contact and takedown-email draft fields when available;

- tool-call, LLM-call, token, cache, latency, and cost telemetry.

This definition matches the real review workflow. The reviewer needs enough evidence to reconstruct the path from a seed website to a player state, stream artifact, provider row, and draft notice. A terminal status alone is not enough.

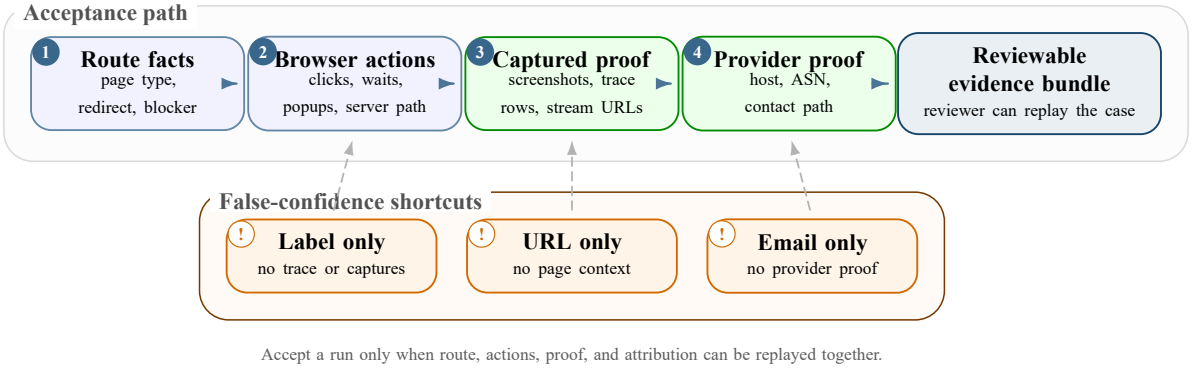


Figure 6.1: Evidence gate used to decide whether an OWC run is reviewable rather than merely labeled.

6.2.2 Acceptance Criteria

Checkpoint	Accepted when
Route decision	The visible browser state supports the selected route: landing, hosting, embedded, inaccessible, redirect, unsupported, or already terminal.
Landing discovery	Match rows, schedule rows, watch buttons, pagination, and hosting candidates are inspected before the system concludes that no hosting path exists.
Hosting interaction	Player controls, popups, overlays, server buttons, and iframe handoffs are attempted in a traceable order.
Embedded inspection	Iframe-local controls and delayed network activity are inspected before the run is marked as no evidence.
Evidence capture	Screenshots and stream candidates support the final claim, or the blocker is recorded as a site, server, embedded, or rate-limit problem.
Provider and email output	Provider rows and draft notices are attached to concrete stream evidence and remain human-reviewable before any enforcement action.

Table 6.1: Acceptance criteria for judging a website run as reviewable evidence.

6.2.3 Failure Taxonomy

Failure categories matter because the same terminal status can hide different causes. A no-stream run may be an agent mistake, but it may also mean that the page was down, the match was not live, the hosting page had disappeared, all servers were inactive, or the model API quota interrupted the test. The evaluation separates these cases before using the run as an agent-quality signal.

Bucket	Evidence signal	Interpretation
Productive evidence	Success or partial run, screenshot evidence, stream record, provider row, or email draft.	The system produced review material on a real website.
No hosting path	Landing page inspected but no useful downstream match or hosting candidate survived.	Usually a page-availability or schedule-state outcome, unless visible candidates were skipped.
No stream evidence	Hosting or embedded page reached, but no useful media or playlist candidate appeared.	Often a server, player, timing, or iframe problem; the trace must show whether the agent tried the available controls.
Agent/runtime pressure	Repeated useless tool calls, wrong route, timeout, redirect loop, or browser/runtime instability.	These rows are the main prompt, tool, and orchestration debugging targets.
Rate or down exclusion	API rate limit, site down, or unavailable external service.	Important for operating throughput, but not a direct browser-agent quality failure.

Table 6.2: Failure taxonomy used before interpreting batch metrics as agent behavior.

6.3 Batch Metrics Used for Evaluation

The dashboard metrics are discussed here once, as a closed snapshot of the current evaluation batch. They are not treated as a generic accuracy score. They are operating evidence that must be read together with the failure taxonomy and agent-level traces.

Strict success is calculated with a fixed eligibility rule:

$$\text{strict success} = \frac{\text{runs marked as strict success}}{\text{terminal persisted runs} - \text{external/no-evidence exclusions}}.$$

The excluded rows are runs where the trace shows that the website could not fairly test the agent: `no_streams`, `no_hosting_pages`, site-down or inaccessible pages, unavailable live events or servers, and API quota/rate-limit interruptions. A no-stream row is excluded only when the trace supports a site, timing, or server availability blocker; if the trace shows that the agent skipped a visible action, it stays in the failure side of the denominator.

Partial evidence, timeout, and failed rows are kept visible in the surrounding metrics instead of being hidden inside the numerator. This is why the strict-success percentage is useful as a headline but still needs the working-website, stream, provider, and failure-category rows beside it.

The strongest interpretation is that OWC is measurable. It records whether a run ended with evidence, where no evidence appeared, how reliable the tools were, how much model context was consumed, and what the average run cost. The weakest interpretation would be to present 73.7% as if it were a final legal or web-wide detection accuracy. The project is evaluated by reconstructable evidence and repeatable agent behavior.

Metric	Current value	Evaluation meaning
Persisted runs	126 orchestrator runs; 0 queued; 0 running.	The snapshot is not distorted by unfinished rows.
Strict success	73.7% after excluding external/no-evidence rows.	Useful headline status, but not sufficient without no-hosting, no-stream, and trace review.
Working websites	24 distinct success or partial websites.	Concrete evidence coverage in the persisted OWC batch.
Stream evidence	217 stream records; 31.7% stream-producing run rate.	Successful runs produce review objects rather than only terminal labels.
Provider and email evidence	103 email records.	Provider enrichment and draft notice creation are attached to stream evidence often enough to be operationally useful.
No-stream or no-hosting rows	31 runs.	Excluded from strict success when the trace shows site, live-event, hosting-page, or server unavailability rather than an agent mistake.
Recent failed rows	44 failed rows in the last 24 hours.	Main source for regression, prompt, and runtime debugging.
Rate/down exclusions	5 rows.	Quota and down-site blockers are tracked outside the strict success denominator.
Tool reliability	97.6% observed tool-call success.	Generic tool execution is not the main bottleneck; page state and agent strategy are.
Latency	525.8 seconds average wall-clock time.	Evidence collection is slow because it requires browser interaction, waits, screenshots, and network observation.
Model usage	2,781 LLM calls; 117,469,858 tokens; 53.7% cache hit share.	Browser agents are large-context workloads where caching materially controls cost.
Cost	$18.696308 total; $0.152003 average per persisted run.	Low enough for iterative engineering tests, but full-batch review still needs daily budget control.

Table 6.3: Single Chapter 6 metric snapshot used to evaluate OWC as an evidence system.

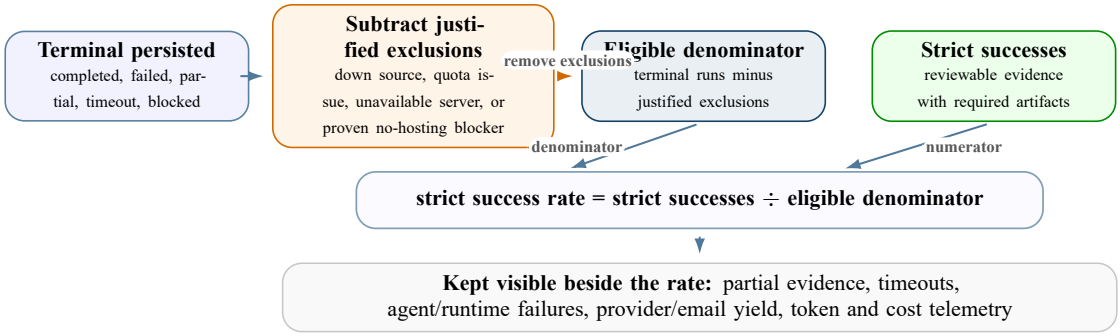


Figure 6.2: Strict-success denominator rule after external and no-evidence exclusions.

6.4 n8n Playground, OWC Coverage, and Cost Discipline

6.4.1 Prototype Coverage and Website Diversity

The n8n playground phase is best read as a feasibility baseline. Before OWC existed as a full platform, the prototype reached more than 150 working websites across MENA and global sources. The MENA group was mostly Arabic and many domains reused the same WordPress-style layout, so one successful navigation pattern often covered several cloned websites. The global subset was smaller, around 50 sites, and a noticeable number went down during the internship. That site drift made batch testing noisy, but it also confirmed that the crawler needed to handle unstable targets rather than only clean demo pages.

The important lesson from n8n is not that 150+ is an accuracy score. It is that the browser-navigation strategy was viable across real layouts, languages, popups, and hosting handoffs. A prototype run was considered useful when it reached the final interaction step correctly, inspected the page state, and extracted network evidence or a clear blocker. OWC reuses that feasibility lesson, but evaluates it with stricter persistence, trace review, and cost accounting.

6.4.2 Strict OWC Evidence, Projection, and Cost Discipline

OWC keeps the same core idea–browser agents that inspect, interact, and extract evidence–but adds stronger routing, structured persistence, prompt regression, tool-output caching, cost telemetry, and long-memory consistency. These additions change the meaning of the count. The current OWC number is stricter because it counts persisted evidence-producing websites, not every website family that a prototype could reach.

The tradeoff is that deeper successful runs can cost more than shallow failures. A better agent discovers more hosting pages, activates more servers, waits for more page states, and records more screenshots and network evidence before stopping. That higher activity is acceptable only because OWC records the model spend, token use, cache hit share, and tool behavior attached to the run.

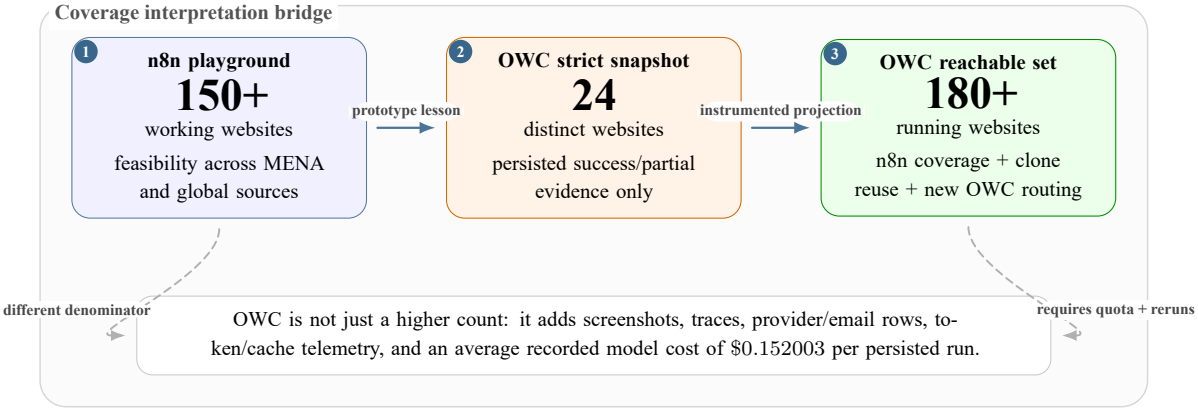


Figure 6.3: Coverage bridge from n8n feasibility to strict OWC evidence and projected OWC reach. The middle value is intentionally stricter because it counts persisted evidence-producing websites rather than broad prototype reach.

The current strict OWC count is 24 distinct evidence-producing websites in the persisted snapshot. That is not a replacement for the n8n 150+ feasibility count because the denominator and instrumentation are different. The better comparison is:

- n8n showed that many real websites could be reached, but it was weaker on repeatability, trace review, and cost accounting;
- OWC shows stricter persisted evidence on 24 distinct websites, with run-level cost, token, screenshot, trace, provider, and email records;
- combining n8n coverage, OWC's newer routing, and known clone families supports a projected reachable set above 180 running websites, provided that quota and testing time are available.

The request-per-day limit remains an operating constraint. A nominal 500 request-per-day budget becomes roughly five or six full website cycles per day once debugging reruns, regression checks, failed attempts, implementation tests, and final validation share the same quota.

6.5 Agent Evaluation

6.5.1 Why Whole-Run Labels Are Not Enough

The agent pipeline cannot be evaluated only with a final success percentage. A run can fail to produce a stream because the match was not live, the hosting server was down, or the provider blocked playback, even if the classification, landing, and hosting agents made the correct decisions. The opposite is also possible: a run can reach a terminal state while hiding a weak route decision, a lost schedule row, or repeated clicks that only worked by accident.

The useful evaluation unit is therefore the agent handoff. Each stage is reviewed by asking three questions: did the agent understand the visible state, did it pass the right context to the next stage, and did it stop without wasting unnecessary tool calls? This keeps strict success useful as a headline metric while preventing it from becoming a black-box score.

6.5.2 Trace Review and Optimization Loop

Agent subtests are not reduced to percentages because their samples are uneven. Some websites have no live match during the test window, some expose several cloned hosting pages, and some only become meaningful after a popup or server switch. The review therefore inspects traces manually and uses the number of required tool calls as a quality signal: if two runs reach the same evidence, the better run is the one that reaches it with fewer unnecessary actions.

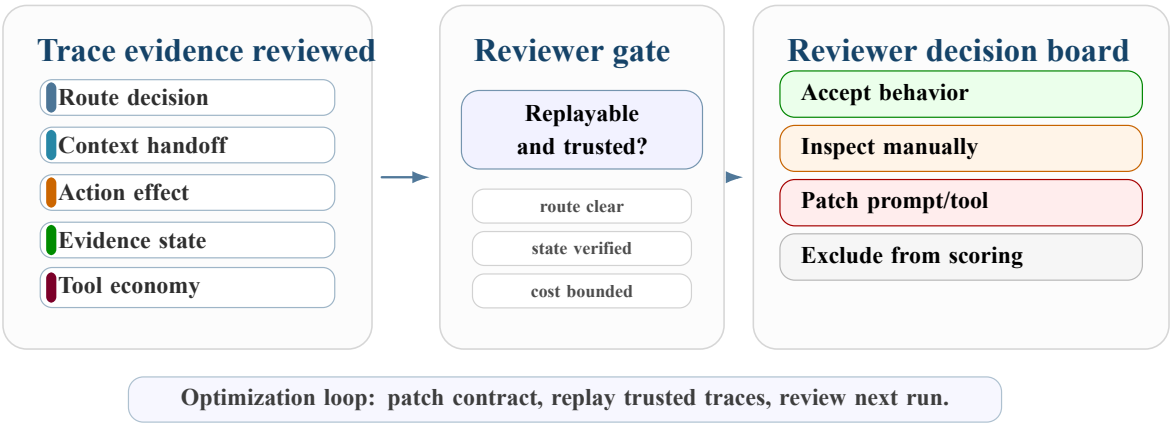


Figure 6.4: Trace-review gate and reviewer decision board used for agent evaluation. The review separates route, context, action, evidence, and tool-use signals before accepting a run, inspecting it manually, patching the prompt/tool contract, or excluding the run from scoring.

The loop also protects regression quality. Prompt or tool changes are checked against websites that previously worked, then the trace is classified into route, context, interaction, evidence, or quota/site availability. Only real prompt or tool failures are used to change the agent behavior. This avoids optimizing against a dead server, a non-live match, or a temporary provider block.

6.5.3 Classification Agent Test

The classification test checks whether the first agent understands the page state before handoff. It should distinguish a landing page from a direct hosting page, an embedded player, a redirect, an inaccessible page, or an unsupported state. The test also checks whether the decision is grounded in visible evidence such as page title, body content, buttons, iframe presence, or navigation outcome.

Agent	Passing behavior	Metrics used during review
Classification	Selects the route supported by visible browser evidence and gives the next specialist a clear handoff.	Route correctness, confidence, redirect/blocker handling, and unnecessary reroute count.
Landing	Reads the body before header/footer fallback, preserves match or schedule context, and returns useful hosting candidates.	Candidate count, missed visible links, no-hosting justification, pagination handling, and duplicate suppression.
Hosting	Activates the player, survives popups, tests server controls, records screenshots, and hands off iframe-local targets when needed.	Server attempts, player-state evidence, screenshot quality, stream candidates, and repeated useless clicks.
Embedded	Inspects frame-local controls and delayed media activity before declaring no evidence.	Frame path correctness, playback attempt, manifest/media observation, and blocker explanation.
Provider/email	Enriches concrete stream hosts and prepares human-reviewable notice drafts.	Provider rows, contact fields, email records, stream-to-provider linkage, and manual-send readiness.

Table 6.4: Agent evaluation matrix used to convert traces into behavior-level findings.

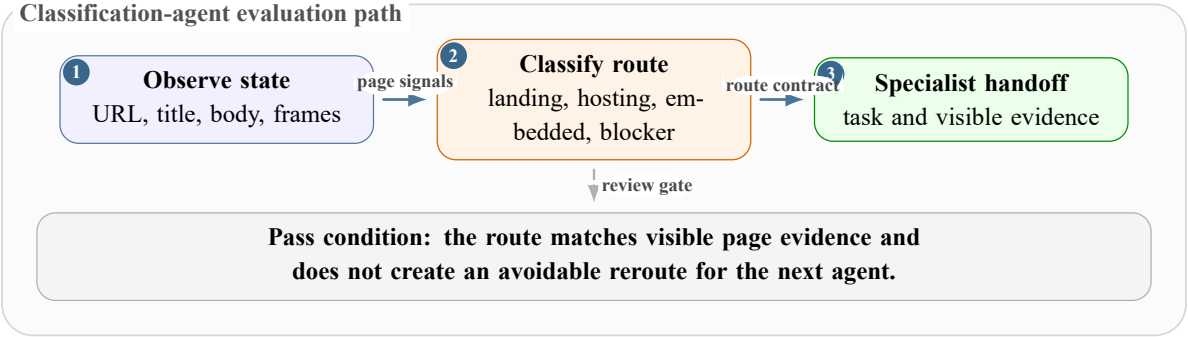


Figure 6.5: Classification-agent test: the decision is accepted only when route, handoff, and next state are supported by observable browser evidence.

This test matters because a wrong route wastes the rest of the run. A landing page sent to the hosting agent usually misses schedule context. A hosting page sent back to landing search often loops over irrelevant links. Classification is therefore judged by route accuracy, handoff quality, and the absence of avoidable reroutes.

6.5.4 Landing Agent Test

The landing test measures whether the agent can move from a listing page to a useful watch path. It should inspect the page body first, preserve match rows or schedule-row context, avoid treating paginator URLs as final hosting targets, and only use header or footer navigation as a fallback. When no hosting path exists, the trace should explain why the page was marked as no hosting rather than hiding the result as a generic failure.

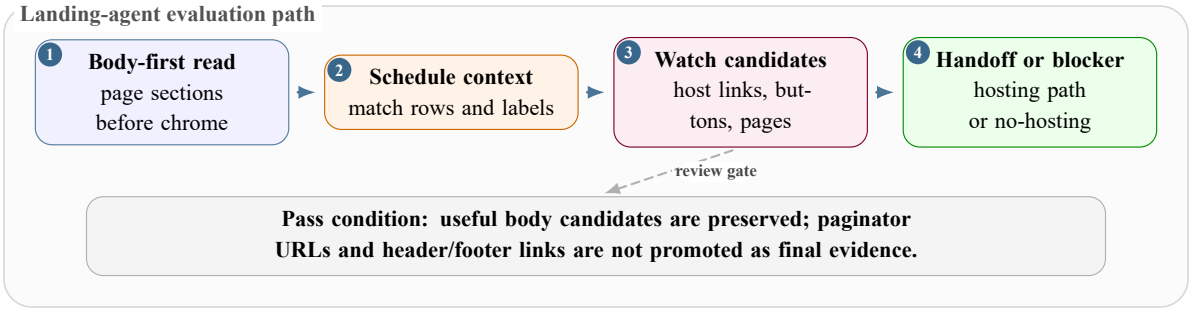


Figure 6.6: Landing-agent test: candidate extraction must preserve body and schedule-row context before declaring that no hosting path exists.

This test is especially important for MENA websites with repeated WordPress-like layouts. A correct landing behavior on one clone family can raise practical coverage across several domains, but only if the agent keeps row context and does not promote navigation chrome as evidence.

6.5.5 Hosting Agent Test

The hosting test measures interaction quality. The agent should handle popups and overlays, click player controls, return focus to the useful page, detect server buttons, switch servers in a traceable order, wait for network activity, and record screenshots that show the tested state. A hosting page that exposes an iframe should not be treated as solved only because the iframe exists; the useful target must be handed to embedded inspection or activated in frame context.

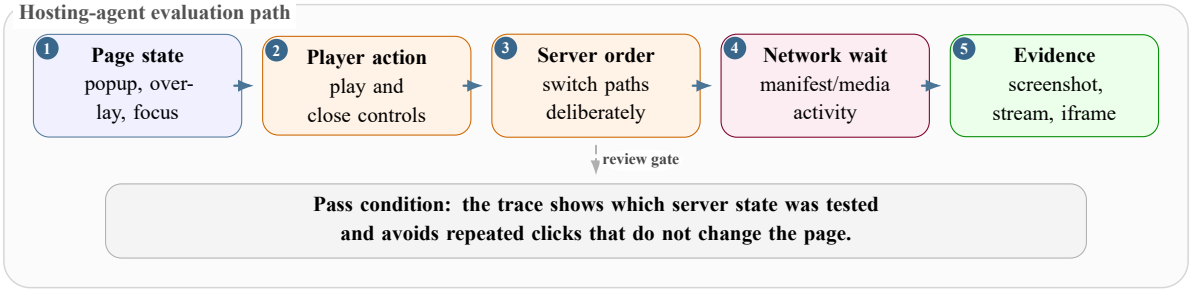


Figure 6.7: Hosting-agent test: interaction quality is evaluated through popup handling, player activation, server switching, screenshots, and stream evidence.

Hosting is where OWC improves most clearly over the n8n playground. The newer system is better at stateful tool use, server switching, popup survival, and keeping evidence attached to the run. Better hosting behavior can increase cost because the agent reaches deeper work instead of stopping early.

6.5.6 Embedded Agent Test

The embedded test checks frame-local inspection. The agent should use the embedded URL or frame path, try relevant controls inside that context, wait for delayed media requests, and record whether the player was unavailable, blocked, or inactive. A no-stream result is acceptable only when the trace shows that the useful embedded state was inspected.

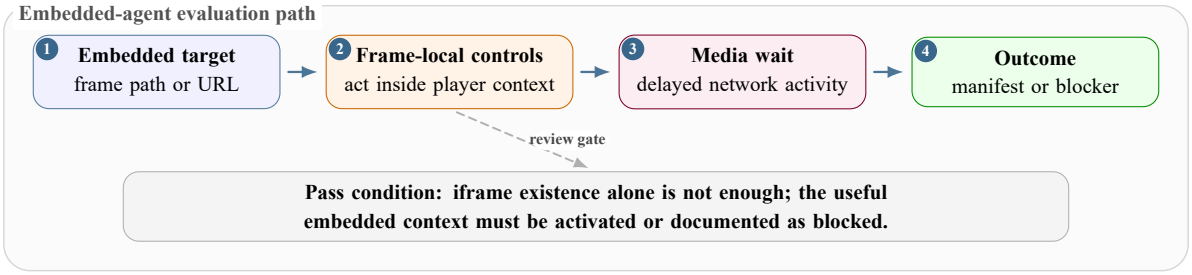


Figure 6.8: Embedded-agent test: iframe-local inspection must happen before the run is accepted as media evidence or as a documented no-stream blocker.

This distinction prevents false agent blame. A hosting page may correctly expose an embedded player while the embedded server itself is down or produces no manifest during the test window. The trace must separate hosting success from embedded availability.

6.5.7 Provider and Email Agent Test

Provider and email evaluation begins only after concrete stream evidence exists. The provider step should extract the playlist or media host, enrich it with DNS, IP, organization, autonomous-system, or country fields where available, and attach the row to the correct stream record. The email step is evaluated as a draft-generation task, not as automatic enforcement.

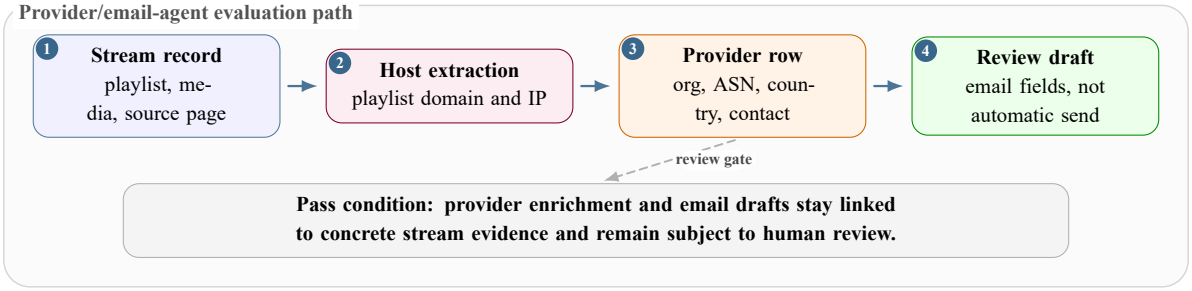


Figure 6.9: Provider/email-agent test: provider enrichment and draft notices must stay linked to concrete stream evidence and remain subject to human review.

A draft is review-ready when it contains the infringing page, stream or playlist host, provider/contact fields, screenshot references, and clear wording that still requires a human reviewer before sending. This preserves the system's role as evidence production, not legal decision-making.

6.6 LLM Runtime Selection

6.6.1 Price and Capability Frontier

The model comparison is scoped to OWC's browser-agent needs: long context, tool use, prompt-cache visibility, stable pricing, and enough throughput for repeated website tests. It is not a universal benchmark. [2, 3, 1, 4, 5, 6].

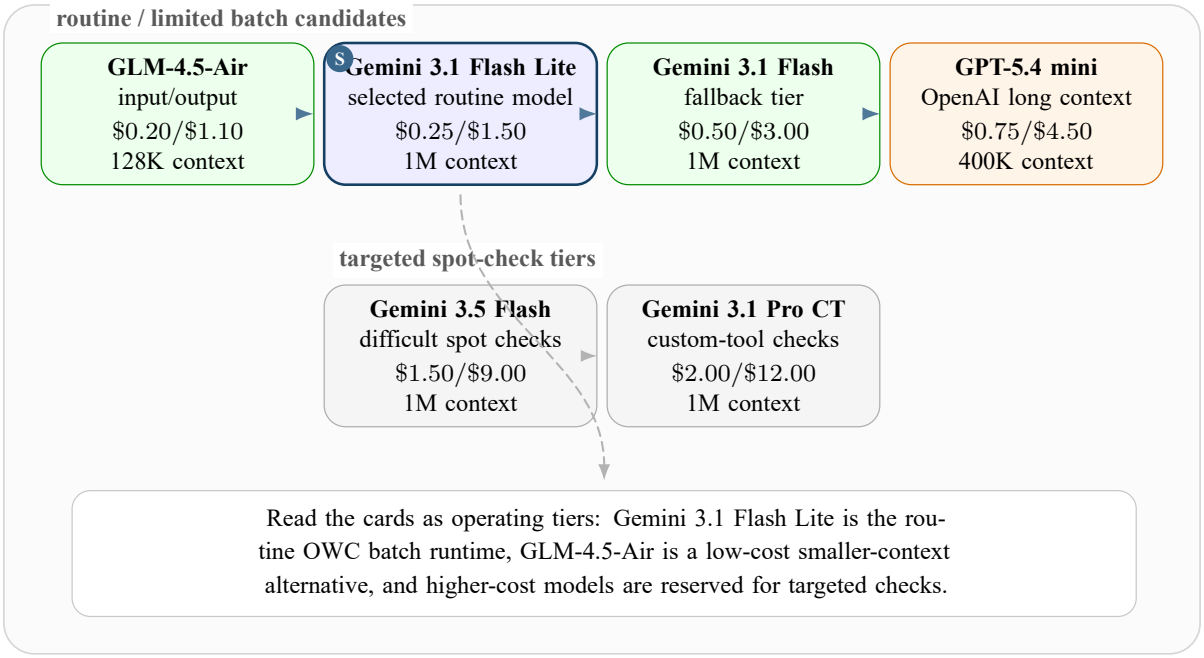


Figure 6.10: Runtime cost map for OWC model selection. Gemini 3.1 Flash Lite is the routine batch point, while higher-cost models are kept for spot checks.

Figure 6.10 should be read as a runtime cost map, not as a universal quality leaderboard. The top row contains models that can plausibly fit routine or limited batch testing. The lower row separates higher-cost spot-check models. The table below adds the missing operational context: context window, cached-input price, and the reason each model is or is not appropriate for repeated browser-agent evaluation.

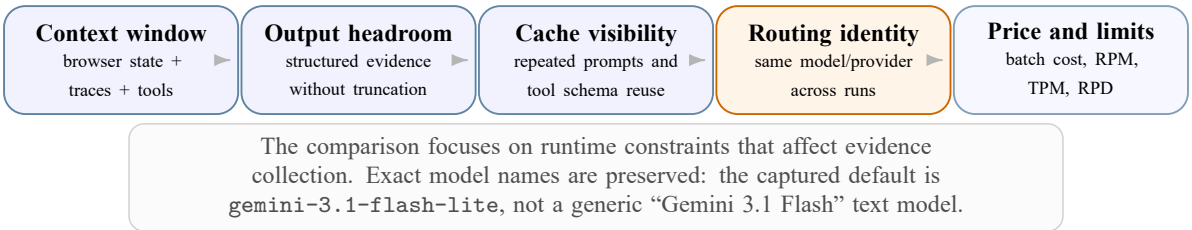


Figure 6.11: Runtime-selection lenses used to compare models for OWC browser-agent evaluation.

Model	Context	Input / 1M	Cached input / 1M	Output / 1M	Evaluation decision
Gemini 3.1 Flash Lite	1M	$0.25	$0.025	$1.50	Selected for routine OWC regression because it combines large context, tool use, cache accounting, and low repeated-run cost.
Gemini 3.1 Flash	1M	$0.50	$0.05	$3.00	Useful Flash-class alternative, but twice the selected input and output price.
Gemini 3.5 Flash	1M	$1.50	$0.15	$9.00	Higher-cost option for difficult spot checks, not attractive for daily batches.
Gemini 3.1 Pro Preview Custom Tools	1M	$2.00	$0.20	$12.00	Relevant for custom-tool behavior, but too expensive for routine browser-agent regression.
GLM-4.5-Air	128K	$0.20	$0.03	$1.10	Attractive low-cost tool model with 106B total and 12B active parameters, but smaller context than Gemini.
GPT-5.4 mini	400K	$0.75	$0.075	$4.50	Strong long-context OpenAI option, but more expensive than Gemini 3.1 Flash Lite for OWC's repeated workload.

Table 6.5: Runtime model comparison for OWC. Prices are per one million tokens where the provider lists token pricing.

6.6.2 Why Gemini 3.1 Flash Lite Was Selected

Gemini 3.1 Flash Lite was selected because OWC's main bottleneck is not one-off model quality. The bottleneck is repeated, long-context, tool-heavy browser evaluation under a real cost and request budget. The selected model gives enough reasoning and tool discipline for the agent loop while keeping input, output, and cached-input prices low enough for prompt regression and website reruns.

The remaining limitation is quota. The same model budget is consumed by final tests, debugging, reruns, failed attempts, prompt-contract tests, and implementation verification. For that reason, the evaluation reports external/no-evidence exclusions separately from agent failures, and known-success regression is prioritized before large exploratory batches.

6.7 Prompt Engineering and Tool-Family Evaluation

6.7.1 Why This Follows the Model Evaluation

The model evaluation does not stand alone. OWC uses the selected model through a prompt contract and a family of browser tools. If the prompt asks for the wrong evidence, the model wastes calls. If the context tools return too much page state, the context window and tokens-per-minute (TPM) budget are consumed before the agent can act. If the interaction tools receive a selector or coordinate that does not match the inspected state, the model may reason correctly but the browser action still fails.

For this reason, prompt engineering and tool-family evaluation were treated as connected parts of the same runtime system. A prompt revision is useful only when it improves the sequence of inspect, decide, act, and verify steps on websites that were already known to work, without breaking output contracts or increasing unnecessary tool calls.

6.7.2 Prompt Engineering Evaluation

6.7.2.1 Prompt-Technique Trials and ReAct Selection

Several prompt styles were tested during the browser-agent design work. The comparison was qualitative and trace-based: the question was not only whether the model produced a convincing sentence, but whether it used the right browser input, selected the correct tool, preserved page context, and verified the result after the action.

Figure 6.12 summarizes the tested prompt families and the result observed in browser-agent traces. The figure keeps the distinction between foundational prompting, reasoning prompts, and operational prompt structures, but makes the selection logic visible at a glance instead of spreading the result across several tables.

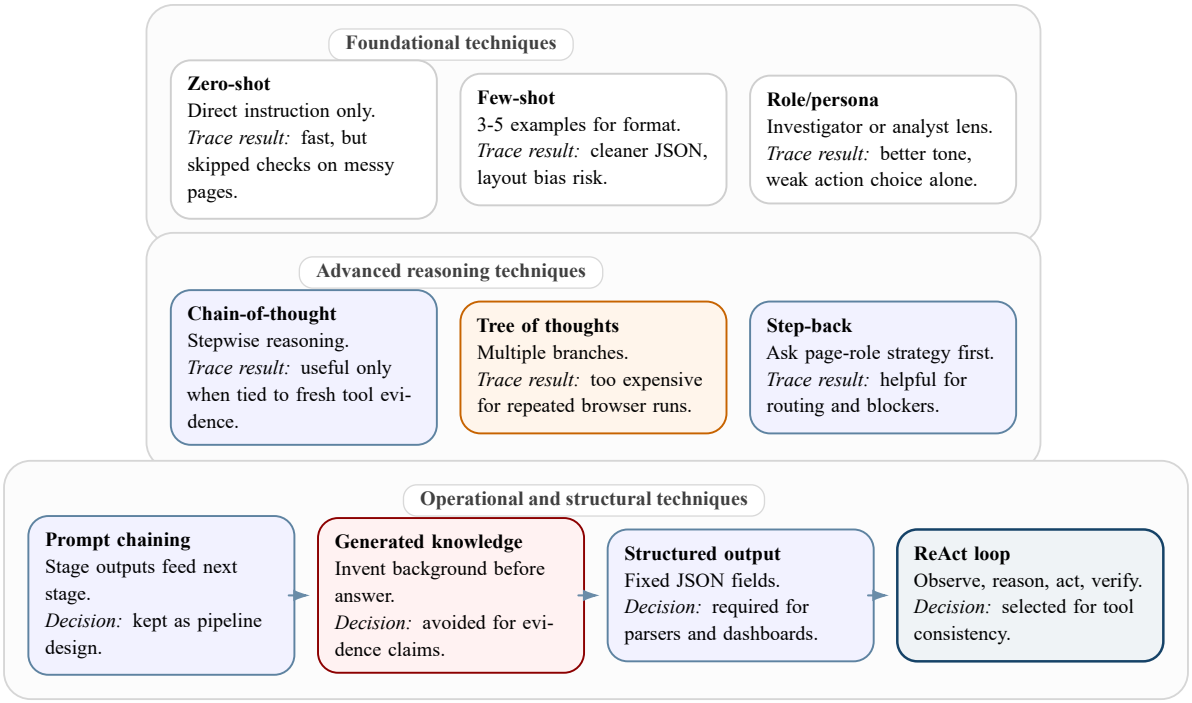


Figure 6.12: Prompt-engineering techniques tested during browser-agent evaluation and the reason ReAct was selected.

ReAct was selected because it matched the real shape of the task. A streaming page is not solved by one answer; it is solved by repeated observation and interaction. The best traces were the ones where the agent first read the current page state, chose a tool based on that state, checked whether the browser actually changed, and then decided whether to continue, switch server, inspect an iframe, harvest network evidence, or stop with a clear blocker. This made ReAct more dynamic than zero-shot or few-shot prompts and more cost-controlled than tree-style branching. Figure 6.13 shows the same reasoning pattern as an evidence loop rather than a code listing.

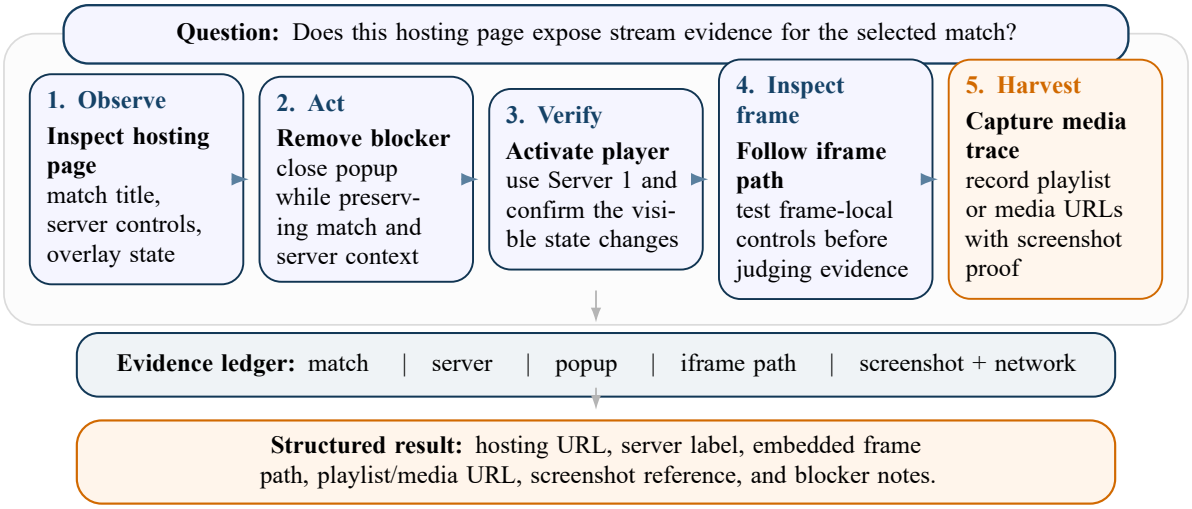


Figure 6.13: ReAct-style hosting prompt example represented as an evidence loop instead of a code listing.

6.7.2.2 Regression Across Known Successful Websites

Prompt changes were evaluated against websites where OWC had already produced useful evidence. This avoids a common mistake: testing a new prompt only on fresh websites where site availability, schedule timing, or server downtime can hide whether the prompt actually improved. A prompt change is considered acceptable when it preserves the known successful route, keeps the expected JSON fields, and still reaches the same kind of evidence with a comparable or smaller number of tool calls.

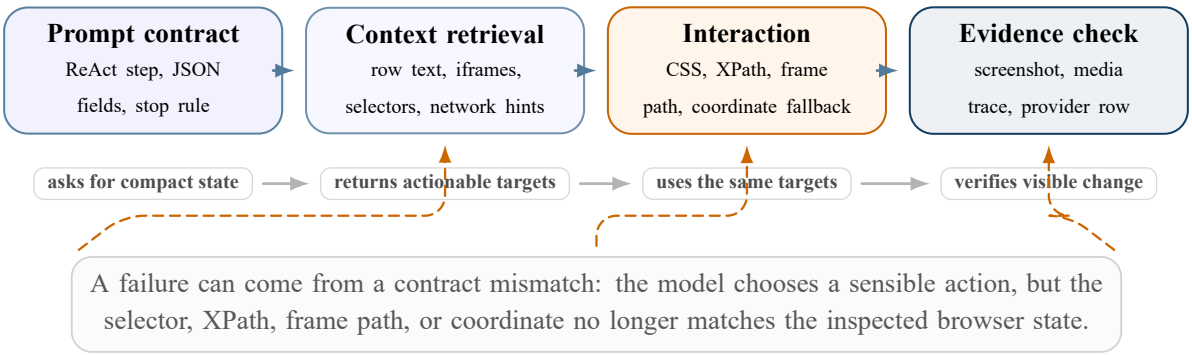


Figure 6.14: Prompt and tool-family alignment contract used during browser-agent evaluation.

Prompt change	Regression target	Pass signal
ReAct loop framing	Classification, landing, hosting, and embedded prompts still use observe, state, hypothesize, act, and verify steps.	The trace explains the page state before action and verifies whether the action changed the browser state.
Output-contract preservation	Existing JSON fields such as `hosting_pages`, `servers`, `protocol_details`, `down_reason`, and `not_embedded_player`.	Runtime parsers continue to receive the fields needed for downstream agents and dashboard review.
Landing body-first behavior	Known successful landing pages with schedules, body buttons, pagination, tabs, and cloned WordPress-style layouts.	The agent preserves row context and does not promote header/footer or paginator links as final evidence.
Hosting and iframe behavior	Hosting pages with popups, server controls, and iframe players.	The agent tries useful frame-local targets and server switches before declaring no stream or handing off too early.
Tool economy	Previously successful websites used for prompt regression.	The same evidence is reached with no increase in repeated useless clicks or context-retrieval loops.

Table 6.6: Prompt-regression evidence used to decide whether a prompt change is safe.

6.7.2.3 ReAct Discipline and Failure Recovery

The prompt contract follows a compact ReAct loop: observe the page state, state the working hypothesis, choose the next action, then verify whether the action changed the page in the expected way. The important part is verification. In this evaluation, a failure does not only mean that the final run failed. It also means that an intended action was not effectively done: a click did not hit the useful element, a popup remained on top of the player, a server switch did not change the player state, or an iframe was handed off without usable frame-local targets.

6.7.3 Tool-Family Evaluation

6.7.3.1 Context Retrieval Tools

The context family exists to describe the website to the agent without dumping the whole DOM into the model. These tools return compact, action-oriented summaries: page title, body sections, candidate links, visible buttons, row text, iframe groups, popup state, server controls, and relevant network or media hints. The evaluation checks whether this summary is small enough to protect the context window and TPM budget, but detailed enough for the agent to choose the next useful action.

Good context retrieval reduces failures because the agent receives stable targets instead of vague page descriptions. For example, a schedule-table page should return the match row around a provider button, not only the provider button text. A hosting page should return iframe-local controls when they exist, not only the fact that an iframe exists. This is why context tools are evaluated by both compactness and usefulness.

6.7.3.2 Interaction Tools

The interaction family executes the action selected by the agent. It can use CSS selectors, XPath expressions, text targets, frame paths, or coordinate fallbacks. These inputs are only reliable when they come from the same context snapshot that informed the agent's decision. A selector returned by inspection, a frame path exposed by hosting context, and the click command that uses it must describe the same browser state.

Interaction is therefore evaluated by action-state matching. A successful interaction is not merely a tool call that returned without an exception. It should visibly close the popup, activate the player, reveal a server list, switch the selected server, navigate to a useful hosting page, or produce new network/media evidence. Repeated clicks that do not change state are counted as prompt/tool inefficiency even if the browser process itself remains healthy.

6.7.3.3 Evidence, Memory, and Enrichment Tools

The remaining tool families verify and preserve the result. Screenshot and network tools show whether the interaction produced evidence. Short-memory and long-memory features help the agents reuse reliable routes across clone families without re-learning the same layout from scratch. Provider and email tools attach stream hosts to enrichment rows and draft notices only after concrete stream evidence exists.

Tool family	Evaluation question	Practical pass signal
Context retrieval	Does the agent receive enough website state without filling the context window or wasting TPM?	Compact summaries include visible candidates, row context, iframe/frame targets, and server controls.
Interaction	Do CSS selectors, XPath targets, frame paths, and coordinates match the inspected state?	The intended page change happens with few retries and no repeated useless clicks.
Evidence capture	Can the reviewer see what changed after the action?	Screenshots, network traces, and media candidates support the final route or blocker.
Memory and routing	Does the system stay consistent across clones and reruns?	Known successful routes are reused carefully, while stale or pagination-only targets are not promoted as final evidence.
Provider and email	Are downstream records attached to concrete stream evidence?	Provider rows and draft notices remain reviewable and linked to the stream or playlist host that produced them.

Table 6.7: Tool-family evaluation contract used after model selection.

6.7.4 Connection Between Prompts and Tools

The strongest OWC runs are the ones where these layers agree. The prompt asks for a small but sufficient context snapshot, the context tool returns targets that can be acted on, the interaction tool uses those exact targets, and the evidence tools verify the result. When one layer drifts from the others, the failure often appears as a browser problem even though the real issue is contract mismatch.

This is why the evaluation does not reduce prompt engineering to wording quality. It is runtime behavior. Better prompts lower cost when they avoid unnecessary context retrieval and repeated actions. Better context tools lower failure rate when they expose the right selectors, frame paths, and row context. Better interaction tools make the model's plan observable in the browser. The complete evaluation is therefore model, prompt, context, interaction, and evidence working as one system.

6.8 Limitations

Limitation	Effect on evaluation	Review action
API quota	Full website cycles are limited because debugging and final runs share the same request budget.	Track external/no-evidence exclusions separately and prioritize known-success regression.
Source site drift	Websites disappear, redirect, change popups, or change server lists during the internship.	Treat traces and screenshots as time-bound evidence.
Live-event timing	A page can be structurally correct while no match or active server is available at test time.	Rerun when the event is live before marking agent behavior as failed.
Clone-heavy coverage	Many MENA domains share the same layout, so domain count can overstate diversity.	Report distinct evidence-producing websites and broader clone family coverage separately.
Provider lookup gaps	CDN, proxy, or hosting intermediaries can hide the true operator.	Use enrichment for triage, then require human verification before notice sending.
Legal interpretation	Evidence packages do not decide infringement, rights ownership, or jurisdiction.	Keep generated notices as drafts for human review.

Table 6.8: Limitations that remain after evidence-first agent evaluation.

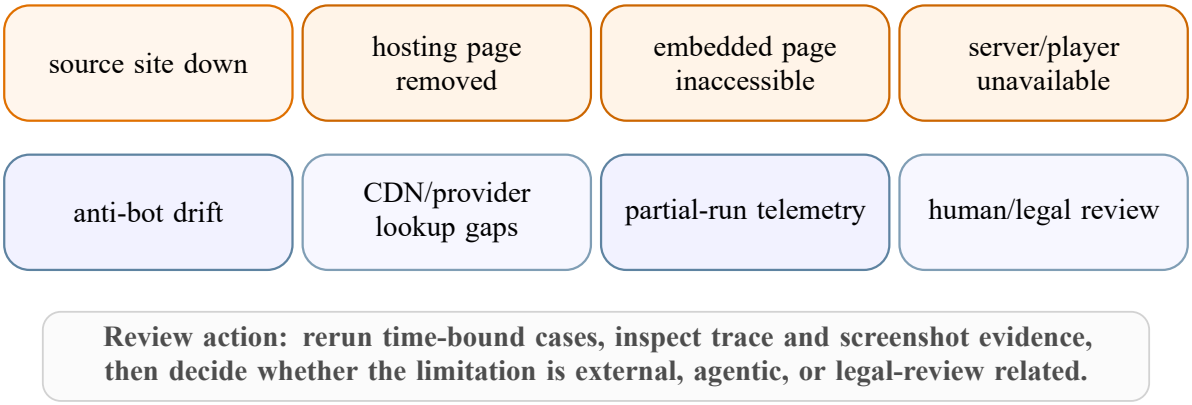


Figure 6.15: Operational risk map for interpreting evaluation results.

6.9 Conclusion

OWC is stronger than the earlier n8n playground because it turns browser-agent behavior into measurable, reviewable evidence. The current batch records 126 persisted runs, 73.7% strict success after external/no-evidence exclusions, 24 distinct evidence-producing websites, 217 stream records, 103 email records, 97.6% tool-call success, and an average model cost of about

$0.152003 per persisted run. These values show an observable evidence-production platform, not merely a demo automation.

The result should still be interpreted conservatively. The metrics do not prove web-wide coverage or legal correctness. They show that OWC can inspect difficult websites, preserve useful traces, expose failure causes, and make cost visible. The most important evaluation contribution is the measurement framework: strict success is read together with working websites, no-hosting and no-stream outcomes, agent-specific behavior, prompt regression, provider enrichment, draft-email readiness, rate-limit pressure, and runtime cost. The prompt and tool-family evaluation explains why these numbers are connected: the selected model is only useful when prompts request compact context, context tools expose actionable targets, interaction tools execute those targets, and evidence tools prove that the intended browser state was reached.

The final chapter uses these evaluation results to summarize the project contribution and identify the next technical perspectives.

CHAPTER 7

General Conclusion and Perspectives

This PFE internship addressed a practical evidence-collection problem in the sports rights-protection domain. Soft Stars needed more than a list of suspicious URLs. It needed a way to preserve what happened in the browser and why the collected evidence could be reviewed later.

The main difficulty was that useful evidence rarely appears in the first HTML response. In practice, it often appears only after:

- the page finishes rendering JavaScript-heavy content;
- a channel group, match card, or server list is opened;
- a player is activated and the media state changes;
- an iframe or embedded page requests a short-lived media manifest.

The work followed Design Science Research Methodology as its methodological backbone. The problem was identified through the limits of static collection and manual review. The solution objectives then focused on classification, browser interaction, screenshots, stream evidence, channel metadata, provider enrichment, traceability, and human review.

The artifact was built in two steps:

- **n8n prototype**: four browser-agent workflows for classification, landing, hosting, and embedded-player extraction. Only the landing agent was deployed as the focused company backup for dynamic targets.
- **Open Web Catcher**: the complete platform built around FastAPI, Next.js, PostgreSQL, MCP browser tools, Puppeteer/Chrome, specialist agents, model telemetry, and reviewable evidence outputs.

The main engineering contribution is the separation of the investigation into clear page roles and agent responsibilities:

- the orchestrator controls the run, prepares handoffs, manages queues, and aggregates results;
- the classification agent decides whether the URL is a landing, hosting, embedded, or irrelevant page;
- the landing agent discovers match, event, channel, hosting, and embedded candidates;
- the hosting agent activates server and player controls while preserving screenshots and diagnostics;
- the embedded agent stays inside direct player contexts and harvests final media evidence.

Provider enrichment and takedown-email drafting remain downstream review stages. They are not automatic enforcement actions, which keeps the system aligned with the need for human verification.

OWC also contributes a more reliable browser-tool layer. Tool profiles separate broad inspection, precise context reads, navigation, interaction, screenshot capture, media-state checks, stream harvesting, and memory hints. Runtime controls then handle profile-scoped sessions, fingerprint behavior, proxy validation, sticky proxy selection, popup recovery, iframe recovery, and media verification.

This makes failures easier to understand. A failed run can be linked to a specific cause: classification uncertainty, missing hosting candidates, player activation failure, iframe drift, proxy failure, missing streams, or skipped provider analysis. It is no longer reduced to a vague failed scrape.

The implementation also produced a review-oriented operator experience. The console shows run status, traces, screenshots, streams, model usage, cached tokens, costs, tool calls, provider results, persistence health, and runtime events. The database supports that review by storing runs, agent runs, runtime events, LLM calls, tool calls, screenshots, streams, provider analyses, email drafts, prompt compilations, and memory hints.

The deployment decision remained pragmatic. The complete OWC platform defines the long-term architecture, while the operational deployment focuses on the validated n8n landing-agent backup:

- target URLs arrive through the `agent_tasks` Azure Service Bus queue;
- a focused container runs the landing-agent worker, Puppeteer MCP, and Chrome;
- matched URLs, metadata, screenshots, rejected URLs, and failure information are returned through the configured results queue.

This keeps the primary company workflow and the AI browser agent decoupled while still covering dynamic pages that need rendered-state reasoning.

The project remains a foundation rather than a finished enforcement platform. Some pages will still fail because of anti-bot behavior, CAPTCHA or challenge pages, unstable player code, server-switching variance, misleading channel labels, popup-heavy navigation, or unavailable providers. Human review is still necessary before evidence is used operationally, and larger controlled batches are needed before making strong accuracy or cost claims.

7.1 Perspectives

The next iteration should focus on concrete improvements:

- CAPTCHA and challenge-page handling, with clear blocked-state evidence instead of silent retries;
- larger labeled website batches and stronger regression checks;
- better reuse of successful selectors and server patterns without hardcoding individual sites;

- OCR-assisted screenshot review for channel labels, player overlays, and blocked pages;
- screenshot annotation and reviewer notes inside the operator console;
- provider-enrichment validation before email drafting, especially when CDN or intermediary hosts hide the true origin;
- gradual deployment from the landing-agent backup toward more queue-driven specialist agents when the operational risk and cost are acceptable.

Taken together, the internship demonstrated that a constrained browser-agent workflow can complement static monitoring when it is constrained, observable, and designed around evidence. OWC does not claim to permanently solve every illegal streaming website. Its value is more practical. It gives Soft Stars a foundation for:

- classifying pages and following controlled handoffs;
- interacting with the browser and verifying player state;
- preserving traces and making the result understandable for human review.

Bibliography

[1] Google, *Gemini API Rate Limits*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/rate-limits. Consulted on: May 20, 2026.

[2] Google, *Gemini Developer API Pricing*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/pricing. Consulted on: May 21, 2026.

[3] Google, *Context Caching in the Gemini API*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/caching. Consulted on: May 22, 2026.

[4] Z.ai, *GLM-4.5 API Pricing*, Z.ai documentation, 2026. Available: https://docs.z.ai/guides/llm/glm-4.5. Consulted on: May 27, 2026.

[5] Z.ai, *GLM-4.5 Model Documentation*, Z.ai documentation, 2026. Available: https://docs.z.ai/guides/llm/glm-4.5. Consulted on: May 27, 2026.

[6] OpenAI, *API Pricing*, OpenAI documentation, 2026. Available: https://openai.com/api/pricing/. Consulted on: May 29, 2026.

[7] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: https://pptr.dev. Consulted on: February 6, 2026.

[8] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: https://chromedevtools.github.io/devtools-protocol/tot/Network/. Consulted on: January 23, 2026.

[9] A. R. Hevner, S. T. March, J. Park, and S. Ram, *Design Science in Information Systems Research*, *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: https://www.jstor.org/stable/25148625. Consulted on: December 9, 2025.

[10] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, *A Design Science Research Methodology for Information Systems Research*, *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: doi:10.2753/MIS0742-1222240302. Consulted on: December 10, 2025.

[11] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: https://fastapi.tiangolo.com. Consulted on: April 8, 2026.

[12] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: https://nextjs.org/docs. Consulted on: April 9, 2026.

[13] LangChain, *LangChain Agents and Tools Documentation*, Official documentation, 2026. Available: https://docs.langchain.com/oss/python/langchain/agents. Consulted on: March 5, 2026.

[14] LangChain, *LangGraph Overview*, Official documentation, 2026. Available: https://docs.langchain.com/oss/python/langgraph/overview. Consulted on: March 7, 2026.

[15] Model Context Protocol, *Model Context Protocol Specification*, Official documentation, 2026. Available: https://modelcontextprotocol.io/specification/2025-06-18. Consulted on: April 14, 2026.

[16] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: https://docs.n8n.io. Consulted on: December 18, 2025.

[17] n8n GmbH, *LangChain Concepts in n8n*, n8n documentation, 2026. Available: https://docs.n8n.io/advanced-ai/langchain/langchain-n8n/. Consulted on: January 12, 2026.

[18] n8n GmbH, *AI Agent Node Documentation*, n8n documentation, 2026. Available: https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/. Consulted on: January 12, 2026.

[19] Microsoft, *Jobs in Azure Container Apps*, *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/container-apps/jobs. Consulted on: May 16, 2026.

[20] Microsoft, *Azure Service Bus Messaging Service*, *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/service-bus-messaging/. Consulted on: May 15, 2026.

[21] Cloudflare, *How Cloudflare Challenges Work*, Official documentation, 2026. Available: https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/. Consulted on: January 29, 2026.

[22] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: https://www.rfc-editor.org/rfc/rfc8216. Consulted on: January 14, 2026.

[23] ISO/IEC, *Information technology—Dynamic adaptive streaming over HTTP (DASH)—Part 1: Media presentation description and segment formats*, ISO/IEC 23009–1, 2019. Available: https://www.iso.org/standard/79329.html. Consulted on: January 16, 2026.

[24] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: https://github.com/gorhill/uBlock. Consulted on: January 27, 2026.

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Chargula II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454